

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-09-16

Table of contents

Sensor Data Analysis Method Module	6
Prerequisites	7
Learning Objectives	8
1 The Principle of Measurements	9
1.1 Measurement	11
1.1.1 Direct and indirect measurement	11
1.1.2 Accuracy and precision	12
1.2 Measurement series	12
1.3 Variance	13
1.4 Standard deviation	13
1.4.1 Experiment: Distribution characteristics	15
1.5 Results	15
1.6 graphical representation	15
1.7 Code	16
1.7.1 Exercise Distribution Statistics	18
1.7.2 Variance and Standard Deviation with NumPy and Pandas	20
2 The Normal Distribution	22
2.1 One Die	22
2.2 two dice	23
2.3 three dice	25
2.4 Plot	30
2.5 Code	30
2.6 absolute frequency	34
2.7 Relative Frequency	35
2.8 Frequency Density	37
2.9 3 Bins	38
2.10 5 Classes	38
2.11 7 Classes	39
2.12 Adjusting a Normal Distribution	41
2.13 The SciPy Package	43
2.14 Normal Distribution Exercises	45
2.15 Confidence Intervals	46

2.16	Standard Normal Distribution	47
2.17	Single Measurement	47
2.18	Sample $N = 12$	48
2.19	Code	49
2.20	The t-Distribution	51
2.21	Plot	52
2.22	Code	53
2.22.1	Example: Weight of female penguins	54
2.23	Graphics	55
2.24	Code	55
2.25	Exercise: Confidence Intervals	57
3	Linear Parameter Estimation	61
3.1	Measurement Series: Hooke's Law	61
3.1.1	Descriptive Statistics	63
3.1.2	Exploratory Statistics	68
3.2	Determining the Spring Constant	72
3.2.1	Linear Regression	74
3.3	Plot	75
3.4	Code	75
3.5	Partial derivative with respect to the intercept	77
3.6	Partial derivative with respect to the slope	78
3.6.1	Quantifying Measurement Deviation	84
3.6.2	The SciPy Module	87
3.6.3	Spring Constant Result	88
4	Error Calculation	89
4.1	Before we begin	89
4.1.1	The Concept of Error	89
4.1.2	Confidence Level	90
4.1.3	Notation	90
4.2	Types of Errors	90
4.3	Fundamental Concepts According to DIN 1319	91
4.3.1	Measurement Deviation	92
4.3.2	Measurement Uncertainty	93
4.3.3	Complete Measurement Result	94
4.3.4	Overview	95
4.4	Absolute and Relative Errors	97
4.5	Maximum Error	98
4.6	Significant Figures	98
4.7	Methods of Error Analysis	100
4.7.1	1. Error Estimation	101
4.7.2	2. Error Statistics	106

4.7.3	3. Error Propagation	109
4.8	Complete Measurement Result	117
4.9	95% Confidence Interval	117
4.10	Linear Error Propagation	120
4.11	Gaussian Error Propagation	120
5	Exercise: Hooke's Law	123
5.1	Reading Files	123
5.2	Task: Reading Files	125
5.3	Sample Solution: Data Preparation	133
5.3.1	Checking for Outliers	134
5.4	Determining Outliers	134
5.5	Combined Output of Measurement Series	136
5.5.1	Conversion of Measurement Quantities	139
5.5.2	Graphical Representation	141
5.6	Graphical Representation	143
5.7	Calculating Means and Standard Errors	144
5.8	Code	144
5.9	Evaluation	145
5.10	Sample Solution: Determining Spring Constants	146
5.11	$\alpha = 0.10$	146
5.12	$\alpha = 0.05$	147
5.13	Code	147
5.14	Evaluation	148
6	Calibration	150
6.1	Calibration and Adjustment	150
6.2	Calibration Methods According to DIN 1319	151
6.2.1	Correction Table	152
6.2.2	Determination of Calibration Factors	152
6.2.3	Determination of a (Empirical) Calibration Function	152
6.3	Additive Errors: The Zero Offset Error	152
6.3.1	Quantifying the Zero-Offset Error	153
6.3.2	Correcting the zero-point error	156
6.3.3	Exercise: Zero-Point Error	156
6.4	Multiplicative Error: Span Error	156
6.4.1	Quantifying Span Error	158
6.4.2	Correcting Span Error	159
6.4.3	Exercise: Span Error	160
6.5	Zero-Point and Span Errors	160
6.5.1	Quantifying Zero-Point and Span Errors	162
6.5.2	Correcting Zero-Point and Span Errors	163

6.6	Nonlinearity: Linearity Error	165
6.6.1	Example Pt100	165
6.6.2	Task: Reading the Pt100 Table	166
6.6.3	Representing Nonlinearity	173
6.6.4	Quantifying Nonlinearity	177
6.7	Types of Errors and Practical Calibration Methods	180
6.7.1	Nonlinearity	180
6.8	Other	181
6.9	Occurrence Together with Nonlinearity	181
6.10	Residual Ramp	181

Sensor Data Analysis Method Module



Building Blocks of Computer-Assisted Data Analysis by Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, and Sebastian Seipel. Sensor Data Analysis Method Module by Maik Poetzsch is licensed under [CC BY 4.0](#). The work is available on [GitHub](#). Excluded from the license are all third-party logos and other content marked otherwise. 2025

Suggested Citation

Arnold, Lukas, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, and Sebastian Seipel. 2025. “Building Blocks of Computer-Assisted Data Analysis. Sensor Data Analysis Method Module.” <https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse-en>.

BibTeX Template

```
@misc{BCD-m-sensordatenanalyse-2025,  
  title={Building Blocks of Computer-Assisted Data Analysis. Sensor Data Analysis Method Modu.  
  author={Arnold, Lukas and Arnold, Simone and Bagemihl, Florian and Baitsch, Matthias and Fel  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse-en}}
```

Prerequisites

The processing time for this module is approximately 30 hours.

The following modules are required to work through this learning unit:

- Tool module Python
- Tool module NumPy
- Tool module Pandas
- Tool module matplotlib

The following modules and packages are used in this unit:

- numpy
 - numpy.polynomial
- pandas
- matplotlib
- glob
- scipy version 1.14.1

The following data sets are used in this module:

- Tooth growth in guinea pigs [CSV file](#)
- Penguin measurements at Palmer Station [GitHub](#)
- Electrical resistance values of a Pt100 thermometer from DIN 60751 and from [here \(PDF\)](#)
- Distance measurements with an ultrasonic sensor (measurements from FH Dortmund)
- Temperature measurements with a resistance thermometer (measurements from FH Dortmund)

Cross-reference to:

- Methods module Data Fitting and Data Optimization

Learning Objectives

In this module you will learn ...

- Statistical basic concepts
- Tools for analyzing and graphically visualizing sensor data
- Methods for processing multiple files
- Error calculation and analysis
- Sensor characteristic curves
- Characteristic curve errors and their correction

1 The Principle of Measurements

“In physics, only that which has been measured truly exists” (Merz 1968, 14).

Merz, Ludwig. 1968. *Grundkurs der Messtechnik. Teil I: Das Messen elektrischer Größen*. 2nd edition. Munich; Vienna. R. Oldenbourg Verlag.

In this module, the following packages are used:

```
import numpy as np
import numpy.polynomial.polynomial as poly
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import glob
```

Physical quantities are determined with the help of measuring instruments. These assign a numerical value to the actual characteristic relative to a reference system.

An example: “Johanna measures 173 centimeters on the measuring board.”

- The actual characteristic is Johanna’s height.
- The measuring instrument is the measuring board.
- The numerical value is 173.
- The reference system is the metric system.

Measured values can only ever be approximations of the true value of the underlying physical quantity for various reasons. On the one hand, a person’s [height varies over the course of the day](#). On the other hand, the measurement result also depends on the scale used. If the measurement had been taken in the imperial system, Johanna’s height would have been determined as 68 inches, which corresponds to 172.72 centimeters.

The measurement result is therefore not an exact representation of the actual characteristic. A well-known example of the uncertainty associated with the measurement process is the coastline paradox: The result of measuring irregular coastlines becomes larger the smaller the chosen measurement segments are.

Britain-fractal-coastline-200km, Britain-fractal-coastline-100km, and Britain-fractal-coastline-50km by Maksim are licensed under [CC BY-SA 3.0](#) and available on Wikipedia ([200km](#), [100km](#), [50km](#)). 2006



(a) Straight measuring segments of 200 km, total length approx. 2350 km
 (b) Straight measuring segments of 100 km, total length approx. 2775 km
 (c) Straight measuring segments of 50 km, total length approx. 3425 km

Figure 1.1: Coastline paradox

1.1 Measurement

! Important 1: Measurement

“A measurement is the experimental process by which a specific value of a physical quantity is determined as a multiple of a unit or reference value.

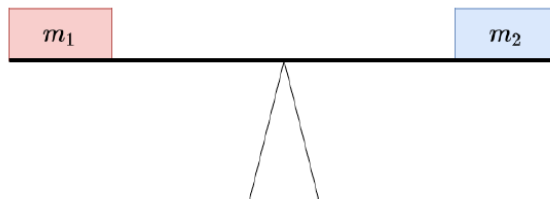
The measurement first yields a measured value. However, due to external influences, this value practically never agrees exactly with the true value of the measured quantity but shows some measurement deviation. A *complete measurement result* is obtained when the measured value is supplemented with quantitative statements about the expected magnitude of the measurement deviation. In metrology, this is regarded as part of the measurement task and thus part of the measurement itself.”

Measurement, by various [authors](#), is licensed under [CC BY-SA 4.0](#) and available on [Wikipedia](#). 2025

- An ideal measurement is a direct measurement or the desired value depends linearly on the measured value.
- An ideal measurement is *accurate* and *precise*.

1.1.1 Direct and indirect measurement

In a direct measurement, the measured quantity is obtained by direct comparison with a standard or a defined reference system.



(a) Beam balance



(b) Folding rulers

Figure 1.2: Direct measurement

Folding rulers by Fst76 are licensed under [CC-BY-SA 3.0](#) and available on [Wikimedia](#). 2014

In an indirect measurement, the measured quantity is related to another physical quantity.



(a) Spring scale



(b) Laser distance measurement

Figure 1.3: Indirect measurement

Spring scale by Amada44 is licensed under [CC-BY-SA-3.0 unported](#) and available on [Wikimedia](#). 2016

Observe the Moon was published by NASA and is available at [nasa.gov](#). 2010

1.1.2 Accuracy and precision

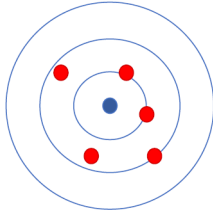
1.2 Measurement series

To reduce measurement uncertainty, a measured value can be repeatedly recorded as part of a measurement series. The best estimate of the measured quantity is given by the arithmetic mean.

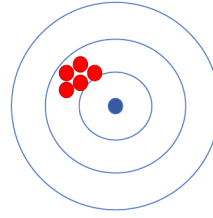
The arithmetic mean of a measurement series $\bar{x}$ is the sum of all individual measured values x_i divided by the number of measurements N .

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Using the arithmetic mean, we can assess the scatter of measured values and the precision of the measurement by calculating the variance and standard deviation.



(a) Accuracy



(b) Precision

The accuracy of a measurement describes how close the measured values are to the real value. Accuracy can only be determined when accepted reference values exist.

The precision of a measurement describes how well the individual measured values agree with each other. The precision of a measurement is determined using the standard deviation of the sample.

Figure 1.4: Accuracy and precision

1.3 Variance

Variance is the mean of the squared deviations from the mean.

$$\text{Var}(x_i) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

1.4 Standard deviation

The square root of the variance is the standard deviation. It has the same unit as the measured values and is therefore easier to interpret. The standard deviation s is calculated as:

$$s_N = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

For samples, the sample variance is used. The following applies to the standard deviation of a sample:

$$s_{N-1} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Since the variance is the square of the standard deviation s , it is often denoted as s^2 .

⚠ Warning 1: Standard deviation and variance in the population

In statistics, formulas are often written with Greek letters when referring to the entire population.

The population mean is also called the expected value and is denoted by the Greek letter μ (mu). The standard deviation of the expected value is denoted by σ (sigma).

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

Using the standard deviation, the *standard error of the mean* can be determined. The standard error of the mean indicates how closely the arithmetic mean of the sample approximates the population mean (more on this shortly) and is also called the *sampling error*. It is calculated from the standard deviation and the square root of the sample size.

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}}$$

Since the population standard deviation is usually unknown, the standard error of the mean is estimated using the sample standard deviation.

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}}$$

Sometimes, the standard error is rewritten in long form:

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

The standard error becomes smaller (the measurement becomes more precise) the smaller the population variance and the larger the sample size.

This can be illustrated using a simulated dice experiment. For an ideal, fair die, each face appears equally often. The expected value of a six-sided die is:

$$\frac{1}{6} \sum_{i=1}^{i=6} (x_i) = 3.5$$

The standard deviation of a fair six-sided die is:

$$\sqrt{\frac{1}{6} \sum_{i=1}^{i=6} (x_i - 3.5)^2} \approx 1.71$$

Since the population variance is known, the standard error of the mean depends only on the sample size.

1.4.1 Experiment: Distribution characteristics

In a simulated experiment, 100 people each roll a die 3, 10, and 50 times and compute the mean of their rolls. Because a fair die is simulated, the standard error can be calculated using the population standard deviation.

1.5 Results

Rolls per person: 3	Sample size: 300
Smallest mean: 1.00	Largest mean: 6.00
Sample mean: 3.57	Standard error: 0.10

Rolls per person: 10	Sample size: 1000
Smallest mean: 1.70	Largest mean: 4.60
Sample mean: 3.49	Standard error: 0.05

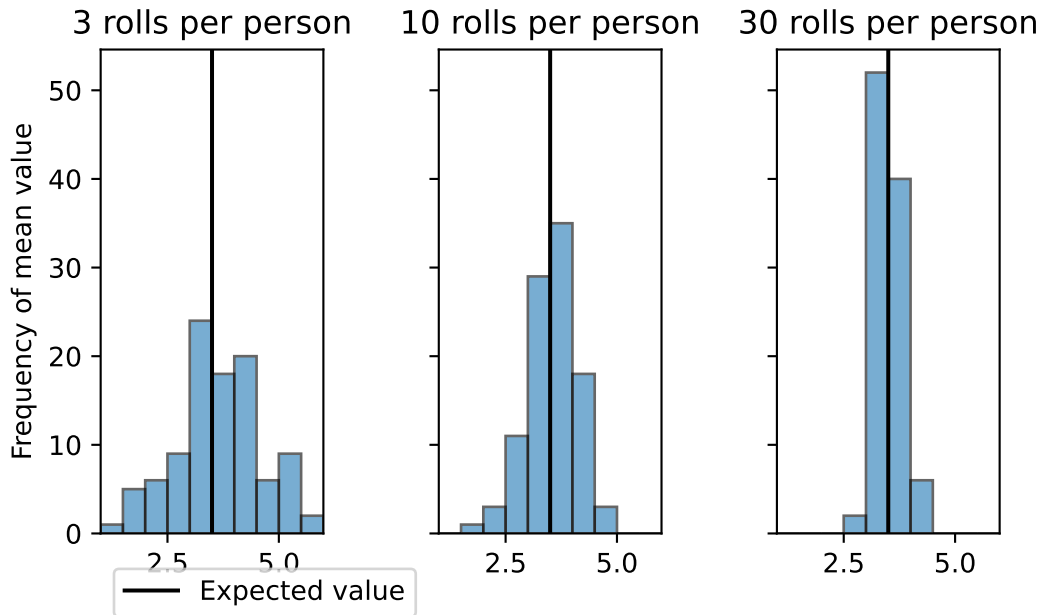
Rolls per person: 50	Sample size: 5000
Smallest mean: 2.98	Largest mean: 4.36
Sample mean: 3.50	Standard error: 0.02

As the number of rolls increases, the minimum and maximum of the individual average values as well as the sample mean approach the expected value.

Note: Since the script is generated dynamically, the random numbers were created using a fixed seed value.

1.6 graphical representation

The frequency of the individual mean values is shown in the following histograms.



1.7 Code

Calculation

```
people = 100
population_std_dev = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 rolls
rolls = 3

## People are in rows (axis = 0), rolls in columns (axis = 1)
dice3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = (people, rolls))

## Compute row-wise mean with np.array.mean(axis = 1)
print(f"Rolls per person: {rolls}\t\t\t\t",
      f"Sample size: {rolls * people}\n",
      f"Smallest mean: {dice3.mean(axis = 1).min():.2f}\t\t",
      f"Largest mean: {dice3.mean(axis = 1).max():.2f}\n",
      f"Sample mean: {dice3.mean():.2f}\t\t",
      f"Standard error: {population_std_dev / (dice3.size ** (1/2)):.2f}\n",
      sep = "")
```

```

# 10 rolls
rolls = 10

## People are in rows (axis = 0), rolls in columns (axis = 1)
dice10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(people, rolls))

## Compute row-wise mean with np.array.mean(axis = 1)
print(f"Rolls per person: {rolls}\t\t\t",
      f"Sample size: {rolls * people}\n",
      f"Smallest mean: {dice10.mean(axis = 1).min():.2f}\t\t",
      f"Largest mean: {dice10.mean(axis = 1).max():.2f}\n",
      f"Sample mean: {dice10.mean():.2f}\t\t",
      f"Standard error: {population_std_dev / ( dice10.size ** (1/2) ):.2f}\n",
      sep = "")

# 50 rolls
rolls = 50

## People are in rows (axis = 0), rolls in columns (axis = 1)
dice50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(people, rolls))

## Compute row-wise mean with np.array.mean(axis = 1)
print(f"Rolls per person: {rolls}\t\t\t",
      f"Sample size: {rolls * people}\n",
      f"Smallest mean: {dice50.mean(axis = 1).min():.2f}\t\t",
      f"Largest mean: {dice50.mean(axis = 1).max():.2f}\n",
      f"Sample mean: {dice50.mean():.2f}\t\t",
      f"Standard error: {population_std_dev / ( dice50.size ** (1/2) ):.2f}\n",
      sep = "")

people = 100
population_std_dev = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 rolls
rolls = 3
dice3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(people, rolls))

# 10 rolls
rolls = 10
dice10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(people, rolls))

```

```

# 50 rolls
rolls = 50
dice50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = rolls)

# plotting
bins = 10

# 3 rolls
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey = True)

ax1.hist(dice3.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax1.set_xlim(1, 6)
ax1.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black', label = 'Expected value')
ax1.set_ylabel('Mean dice result')
ax1.set_ylabel('Frequency of mean')
ax1.set_title("3 rolls per person")
ax1.legend(loc = 'lower left', bbox_to_anchor = (0, -0.2))

# 10 rolls
ax2.hist(dice10.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax2.set_xlim(1, 6)
ax2.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax2.set_ylabel('Mean dice result')
ax2.set_title("10 rolls per person")

# 50 rolls
ax3.hist(dice50.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax3.set_xlim(1, 6)
ax3.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax3.set_ylabel('Mean dice result')
ax3.set_title("50 rolls per person")

plt.tight_layout()
plt.show()

```

1.7.1 Exercise Distribution Statistics

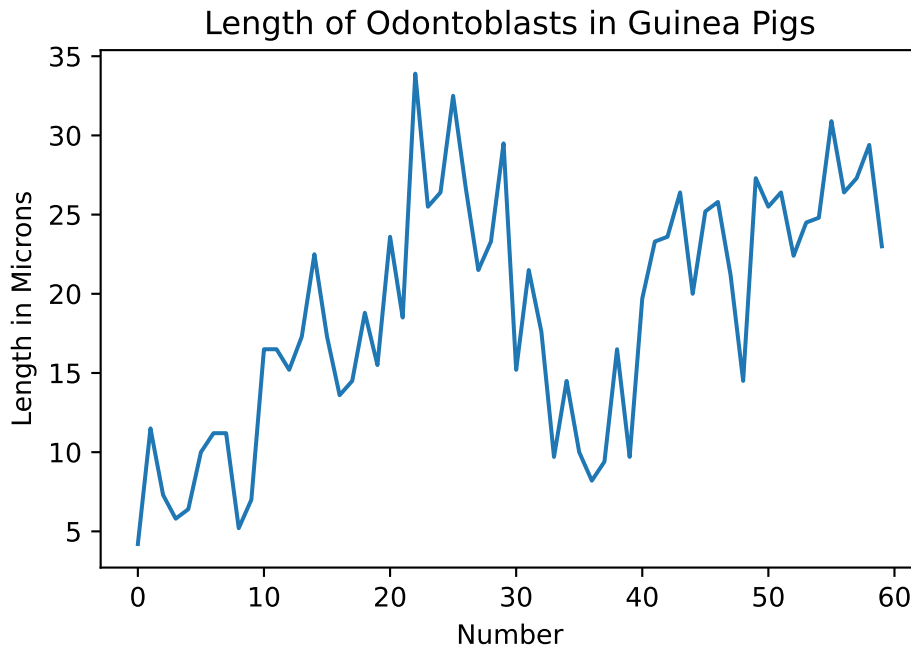
The dataset `ToothGrowth.csv` contains a series of measurements for the length of tooth-forming cells in guinea pigs. The animals received Vitamin C either directly (VC) or in the form of orange juice (OJ) at different doses.

Code Block 1.1

```
file_path = "01-daten/ToothGrowth.csv"
guinea_pigs = pd.read_csv(filepath_or_buffer = file_path, sep = ',', header = 0, \
    names = ['ID', 'len', 'supp', 'dose'], dtype = {'ID': 'int', 'len': 'float', 'dose': 'float'})
```

Crampton, E. W. 1947. "THE GROWTH OF THE ODONTOBLASTS OF THE INCISOR TOOTH AS A CRITERION OF THE VITAMIN C INTAKE OF THE GUINEA PIG." The Journal of Nutrition 33 (5): 491–504. <https://doi.org/10.1093/jn/33.5.491>

The dataset can be accessed in R using the command `ToothGrowth`.



Calculate the arithmetic mean, variance, standard deviation, and sampling error of the measurement series for tooth length (`len`). Use the formulas provided.

The result could look like this:

```
N: 60
Arithmetic mean: 18.81
Sampling error: 0.99
Sample variance: 58.51
Sample standard deviation: 7.65
```

💡 Tip 1: Sample Solution: Distribution Statistics

```
def distribution_statistics(x, output = True):

    # Determine the number of measurements
    N = len(x)

    # Calculate the arithmetic mean
    sample_mean = sum(x) / N

    # Calculate the sample variance
    sample_variance = sum((x - sample_mean) ** 2) / (N - 1)

    # Calculate the sample standard deviation
    sample_std_dev = sample_variance ** (1/2)

    # Calculate the sampling error
    sampling_error = sample_std_dev / (N ** (1/2))

    # Output
    if output: # output = True
        print(f"N: {N}\n",
              f"Arithmetic mean: {sample_mean:.2f}\n",
              f"Sampling error: {sampling_error:.2f}\n",
              f"Sample variance: {sample_variance:.2f}\n",
              f"Sample standard deviation: {sample_std_dev:.2f}",
              sep = '')

    else: # output = False
        return N, sample_mean, sampling_error, sample_variance, sample_std_dev

distribution_statistics(guinea_pigs['len'])
```

1.7.2 Variance and Standard Deviation with NumPy and Pandas

The NumPy and Pandas modules have built-in functions for calculating variance and standard deviation. Variance and standard deviation are calculated using the functions `np.var()` and `np.std()`, or the methods `pd.var()` and `pd.std()`. The parameter `ddof` (delta degrees of freedom) controls which denominator is used for calculating variance in the form $N - \text{ddof}$. While NumPy's default is `ddof=0`, Pandas calculates the sample variance with the default value `ddof=1`.

```
print("Variance:")
print(f"NumPy:\t{np.var(guinea_pigs['len']):.2f}")
print(f"Pandas:\t{guinea_pigs['len'].var():.2f}")

print("\nStandard Deviation:")
print(f"NumPy:\t{np.std(guinea_pigs['len']):.2f}")
print(f"Pandas:\t{guinea_pigs['len'].std():.2f}")
```

Variance:

NumPy: 57.54

Pandas: 58.51

Standard Deviation:

NumPy: 7.59

Pandas: 7.65

2 The Normal Distribution

As the sample size increases, the estimate of the expected value becomes more accurate. Mathematically, this observation is explained by the [central limit theorem](#). For example, when rolling multiple dice, outcomes far from the expected value become increasingly unlikely. This can already be demonstrated with just a few dice (see example).

i Note 1: Frequency Distribution of Dice Outcomes

For a single die, there are 6 possible outcomes; for 2 dice, $6 * 6$ possible combinations; for 3 dice, $6 * 6 * 6$ combinations, and so on. Because many combinations have the same total value, results near the expected value occur more frequently than, for example, rolling all ones.

2.1 One Die

```
one_die = []

for i in range(1, 7):
    one_die.append(i)

one_die = pd.Series(one_die)

print("Frequency distribution of dice rolls:")
print(one_die.value_counts(), "\n")
print(f"Average: {one_die.mean():.1f}")

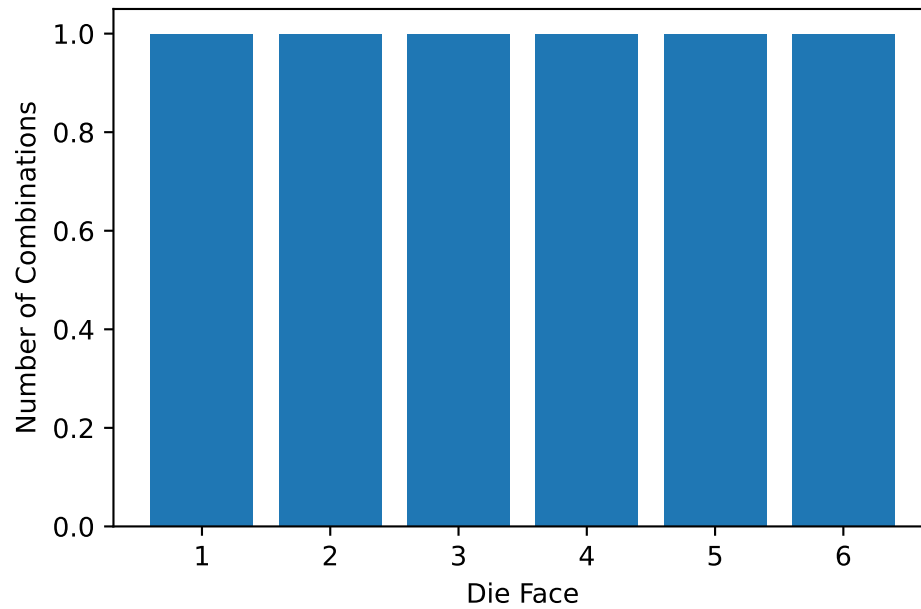
plt.bar(one_die.unique(), one_die.value_counts())
plt.xlabel('Die Face')
plt.ylabel('Number of Combinations')
plt.show()
```

Frequency distribution of dice rolls:

1	1
2	1
3	1

```
4    1
5    1
6    1
Name: count, dtype: int64
```

Average: 3.5



2.2 two dice

```

two_dice = []

for i in range(1, 7):
    die_1 = i

    for j in range(1, 7):
        die_2 = j
        two_dice.append(die_1 + die_2)

two_dice = pd.Series(two_dice)

print("Frequency distribution of the sum of dice:")
print(two_dice.value_counts().sort_index(ascending=True), "\n")
print(f"Average: {two_dice.mean():.1f}")
print(f"Average per die: {two_dice.mean() / 2:.1f}")

plt.bar(two_dice.unique(), two_dice.value_counts().sort_index(ascending=True))
plt.xlabel('Sum of dice')
plt.ylabel('Number of combinations')
plt.grid()
plt.show()

```

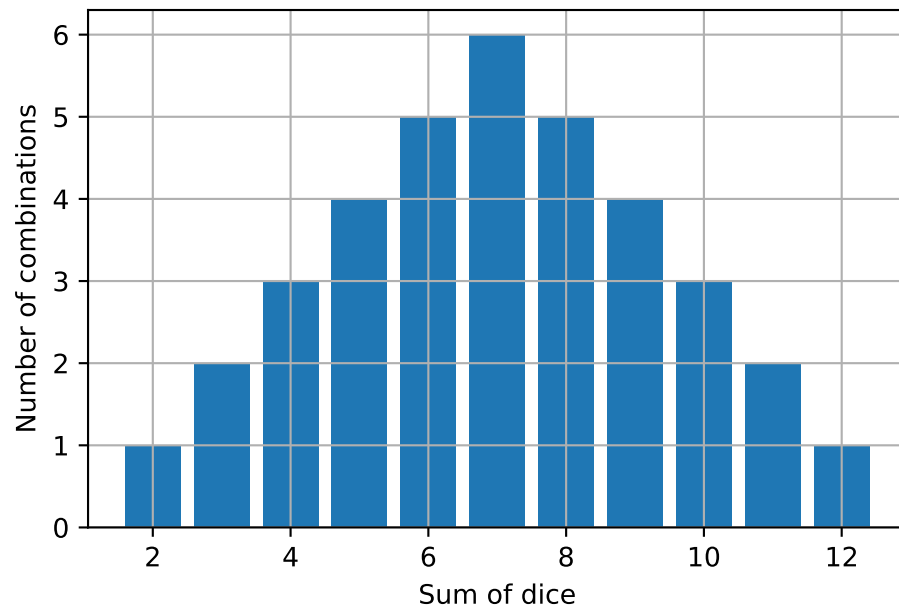
Frequency distribution of the sum of dice:

2	1
3	2
4	3
5	4
6	5
7	6
8	5
9	4
10	3
11	2
12	1

Name: count, dtype: int64

Average: 7.0

Average per die: 3.5



2.3 three dice

```

three_dice = []

for i in range(1, 7):
    die_1 = i

    for j in range(1, 7):
        die_2 = j

        for k in range(1, 7):
            die_3 = k
            three_dice.append(die_1 + die_2 + die_3)

three_dice = pd.Series(three_dice)

print("Frequency distribution of the sum of dice:")
print(three_dice.value_counts().sort_index(ascending=True), "\n")
print(f"Average sum: {three_dice.mean():.1f}")
print(f"Average per die: {three_dice.mean() / 3:.1f}")

plt.bar(three_dice.unique(), three_dice.value_counts().sort_index(ascending=True))
plt.xlabel('Sum of dice')
plt.ylabel('Number of combinations')
plt.grid()
plt.show()

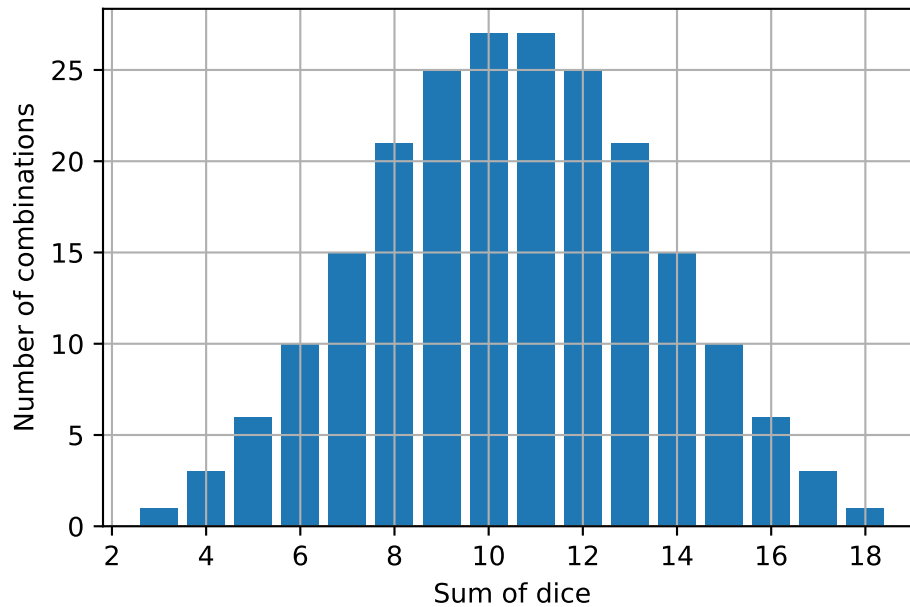
```

Frequency distribution of the sum of dice:

3	1
4	3
5	6
6	10
7	15
8	21
9	25
10	27
11	27
12	25
13	21
14	15
15	10
16	6
17	3

```
18      1
Name: count, dtype: int64
```

```
Average sum: 10.5
Average per die: 3.5
```



The convergence of measurements toward a population mean as the sample size increases also holds when the population mean and variance are unknown. As the sample size grows, the measurements approach the [normal distribution](#), which is also known as the Gaussian bell curve after its discoverer, Carl Friedrich Gauss.

The convergence of the distribution of measurements to the normal distribution for larger samples can be illustrated using the body mass of penguins from the palmerpenguins dataset.

palmerpenguins

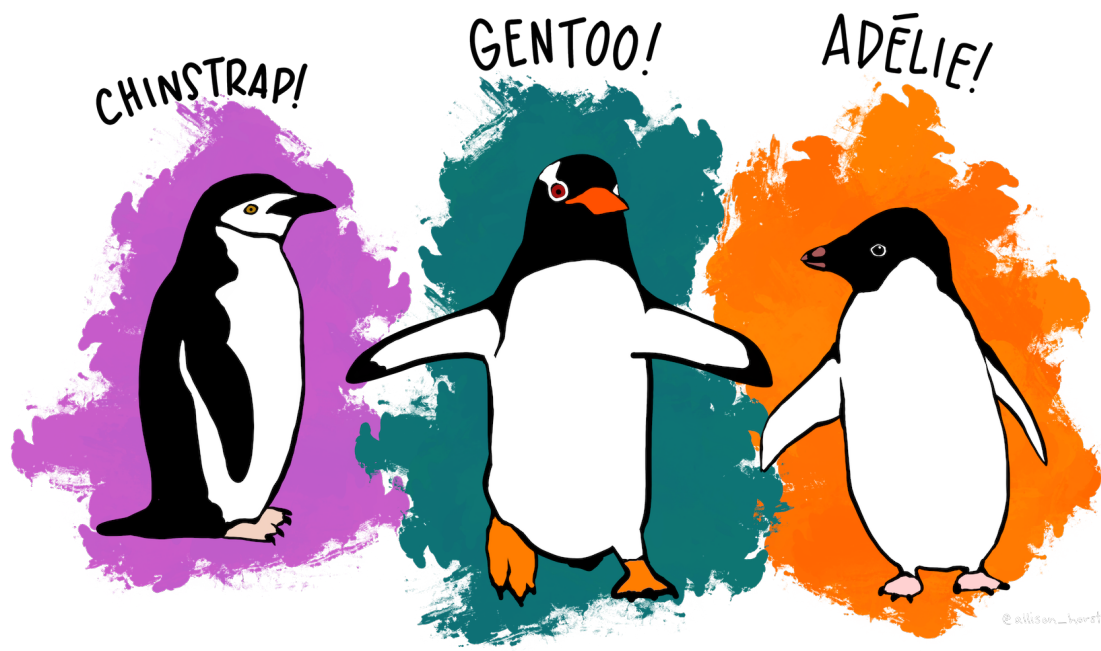


Figure 2.1: Penguins from the Palmer Station dataset

Meet the Palmer Penguins by @allison_horst is licensed under [CC0-1.0](#) and is available on [GitHub](#). 2020

The dataset is released under the [CC0](#) license and is available in R as well as on [GitHub](#). 2020

```
# R commands to load the dataset
install.packages("palmerpenguins")
library(palmerpenguins)
```

Horst AM, Hill AP, and Gorman KB. 2020. palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. <https://allisonhorst.github.io/palmerpenguins/>. doi: 10.5281/zenodo.3960218.

```
penguins = pd.read_csv(filepath_or_buffer="01-daten/penguins.csv")

# Remove animals with incomplete entries

penguins.drop(np.where(penguins.apply(pd.isna).any(axis=1))[0], inplace=True)
```

```
print(penguins.info(), "\n")
```

```
<class 'pandas.DataFrame'>
Index: 333 entries, 0 to 343
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                333 non-null   str
1   island                 333 non-null   str
2   bill_length_mm         333 non-null   float64
3   bill_depth_mm          333 non-null   float64
4   flipper_length_mm      333 non-null   float64
5   body_mass_g            333 non-null   float64
6   sex                    333 non-null   str
7   year                   333 non-null   int64
dtypes: float64(4), int64(1), str(3)
memory usage: 23.4 KB
None
```

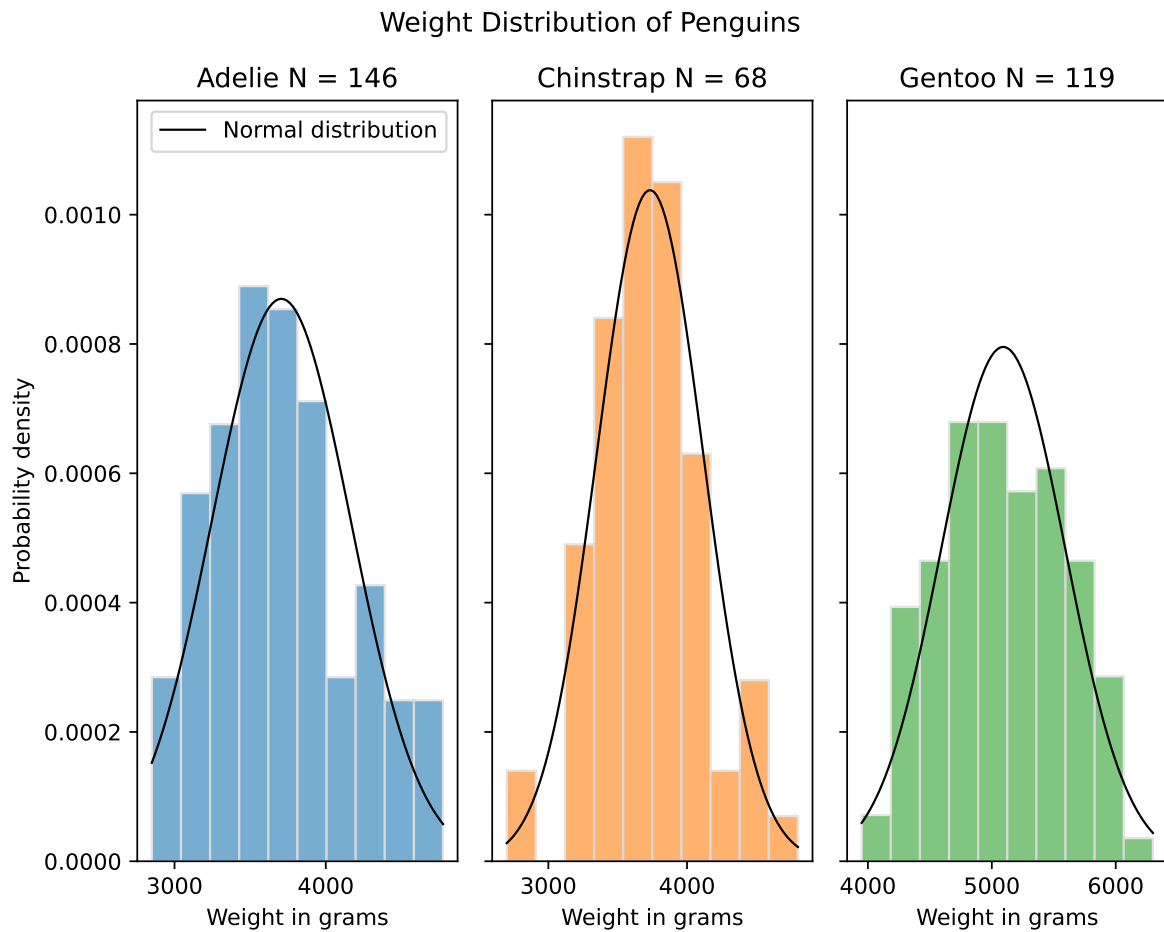
The dataset contains data for three penguin species.

```
print(penguins.groupby(by = penguins['species']).size())
```

```
species
Adelie      146
Chinstrap   68
Gentoo     119
dtype: int64
```

Among other things, body weight was measured in grams, which is recorded in the column `body_mass_g`. The weight distribution of the three species is displayed using a histogram for each species. Additionally, the sample mean and sample standard deviation are calculated for each species. These values can be used to compute a normal distribution curve and overlay it on the histogram (as shown in [Example 2](#)). This allows a visual check of whether the empirical distribution of the sample values corresponds to a normal distribution with the same mean and standard deviation. (For measurement data, this is often less clear than for the distribution of dice rolls.)

2.4 Plot



2.5 Code

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Weight Distribution of Penguins')

# Adelie

species = 'Adelie'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogram
```

```

ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
ax1.set_xlabel('Weight in grams')
ax1.set_ylabel('Probability Density')
ax1.set_title(label = str(species) + " N = " + str(data.size))

## Normal Distribution Curve

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (sample_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - sample_mean) ** 2 /

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 'Normal Distribution')
ax1.legend()

# Chinstrap

species = 'Chinstrap'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogram

ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Weight in grams')
ax2.set_title(label = str(species) + " N = " + str(data.size))

## Normal Distribution Curve

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (sample_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - sample_mean) ** 2 /

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo

species = 'Gentoo'
data = penguins['body_mass_g'][penguins['species'] == species]

```

```

## Histogram

ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Weight in grams')
ax3.set_title(label = str(species) + " N = " + str(data.size))

## Normal Distribution Curve

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (sample_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - sample_mean) ** 2 /

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

The normal distribution is a density curve that a histogram approaches as the number of measurements goes to infinity and the class width approaches zero.

! Important 2: Histogram

A histogram is a graphical representation of the frequency distribution of cardinally scaled variables (i.e., with numerical, ordered values). The data are divided into classes, which can have either constant or variable widths. Adjacent rectangles are drawn with the width of each class, and the area of each rectangle represents the (relative or absolute) class frequency. The height of each rectangle then represents the (relative or absolute) frequency density, i.e., the (relative or absolute) frequency divided by the width of the corresponding class.

Example 2.2: Calculating and Visualizing a Histogram

As an example, we use the length of the tooth-forming cells of guinea pigs that received a vitamin C dose of 2.

```

dose2 = guinea_pigs.loc[guinea_pigs['dose'] == 2, 'len']
print(*list(dose2)) # * = print without commas
print("N", len(dose2), "Minimum:", dose2.min(), "Maximum:", dose2.max(), "Range", dose2

```

```

23.6 18.5 33.9 25.5 26.4 32.5 26.7 21.5 23.3 29.5 25.5 26.4 22.4 24.5 24.8 30.9 26.4 2
N 20 Minimum: 18.5 Maximum: 33.9 Range 15.399999999999999

```

The function `np.histogram(a, bins = 10, range = None, density = None)` can be used to compute a histogram.

- `a` is the data to be analyzed
- `bins` specifies the number of bins (classes); by default, 10 bins are used.
- `range = (float, float)` allows setting the lower and upper limits of the bins.
- `density = True` allows representing the value as the probability density function instead of absolute frequencies. This is calculated as follows:
 - $\text{relative frequency} = \text{number of values per bin} / \text{total number of values}$
 - $\text{probability density} = \text{number of values per bin} / (\text{total number of values} * \text{bin width})$
 - $\text{bin width} = (\text{maximum value} - \text{minimum value}) / \text{number of bins}$

For a manageable number of values, a histogram with 5 bins is computed. For comparison, the probability density is also displayed.

```
print(np.histogram(dose2, bins = 5))
print("Probability density:", np.histogram(dose2, bins = 5, density = True)[0])
```

```
(array([2, 5, 8, 2, 3]), array([18.5 , 21.58, 24.66, 27.74, 30.82, 33.9 ]))
Probability density: [0.03246753 0.08116883 0.12987013 0.03246753 0.0487013 ]
```

The function `np.histogram()` returns an array of absolute frequencies or frequency densities for each bin as the first output. The second output is an array of the x-positions of the bin edges — for each bin, the position of the left edge is given, and for the last bin, the position of the right edge is also included. For 5 bins, this results in 6 edge positions.

The bin width can be obtained, for example, using the `np.diff()` method.

```
hist_abs, bin_edges = np.histogram(dose2, bins=5)
bin_width = np.diff(bin_edges)
print(bin_width)
```

```
[3.08 3.08 3.08 3.08 3.08]
```

By multiplying the frequency density by the class width, the relative frequencies can be calculated.

```
hist_density = np.histogram(dose2, bins=5, density=True)[0]
hist_relative = hist_density * bin_width
print(hist_relative)
```

```
[0.1  0.25 0.4  0.1  0.15]
```

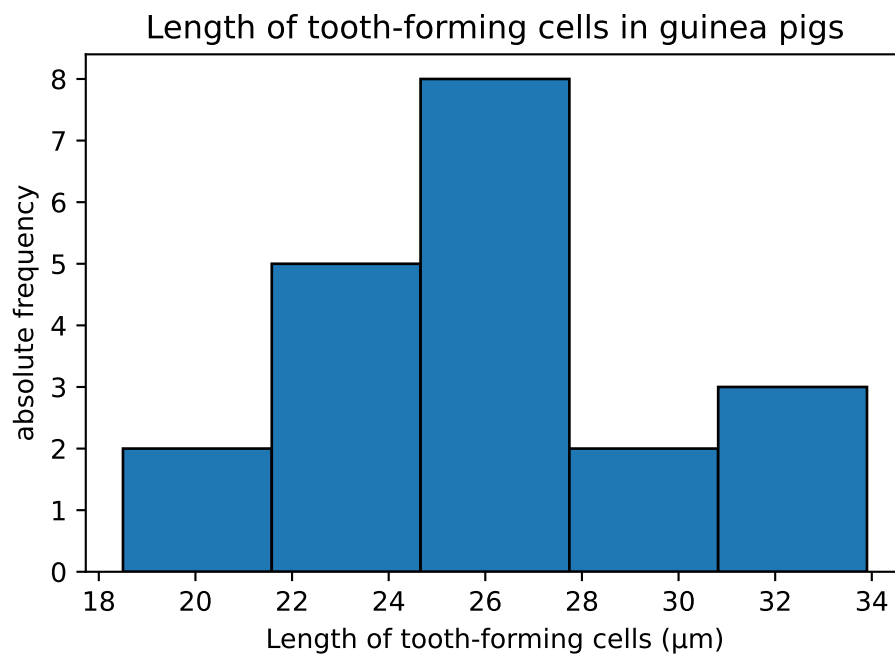
The sum of the relative frequencies is 1. A histogram can be created using the function `plt.hist(x, bins=None, *, range=None, density=False)`, which internally calls `np.histogram()` for calculations. The function parameters correspond to those of the NumPy function, with the argument `x` specifying the data to be plotted. Additional graphical parameters can also be passed. The function returns 3 values: the absolute frequency of the bins (or, if `density=True`, the probability density), the x positions of the rectangles, and the objects used for plotting (the latter are stored in the variable `ignore` in the code below and are not used further).

2.6 absolute frequency

```
plt.hist(dose2, bins=5, edgecolor='black')
plt.title('Length of tooth-forming cells in guinea pigs')

# Axis labels
plt.xlabel('Length of tooth-forming cells (m)')
plt.ylabel('absolute frequency')

plt.show()
```



2.7 Relative Frequency

A representation of relative frequencies is not directly possible.

```

hist_density, bins, ignore = plt.hist(dose2, bins=5, density=True, edgecolor='black')
plt.title('Length of tooth-forming cells in guinea pigs')

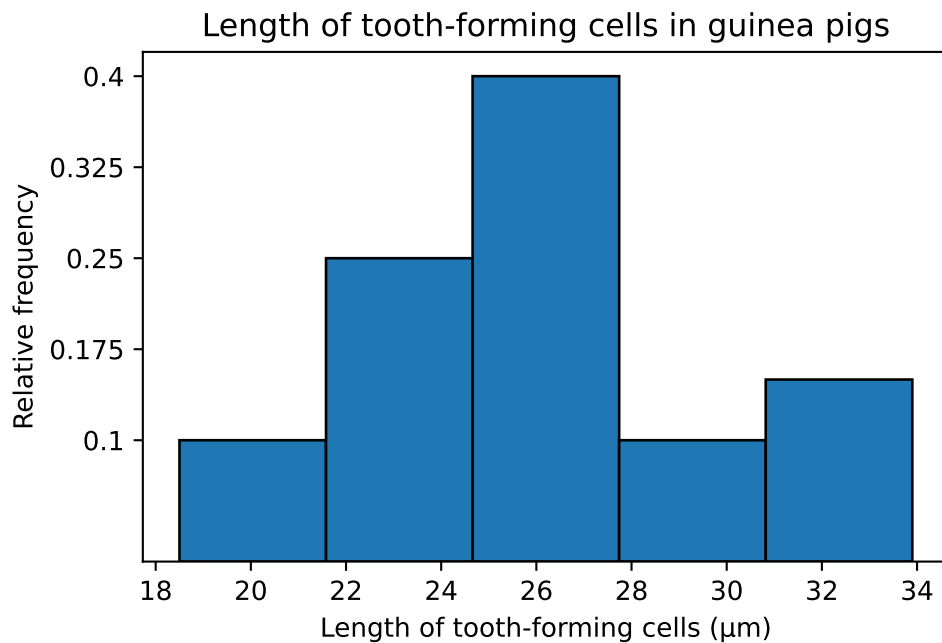
# calculate relative frequency
bin_width = np.diff(bins)[0]
hist_relative = hist_density * bin_width

# generate yticks at positions from min(hist_density) to max(hist_density)
# but with values from hist_relative
plt.yticks(
    ticks=np.linspace(min(hist_density), max(hist_density), len(hist_relative)),
    labels=np.linspace(hist_relative.round(2).min(), hist_relative.round(2).max(), len(hist_relative))
)

# axis labels
plt.xlabel('Length of tooth-forming cells (μm)')
plt.ylabel('Relative frequency')

plt.show()

```

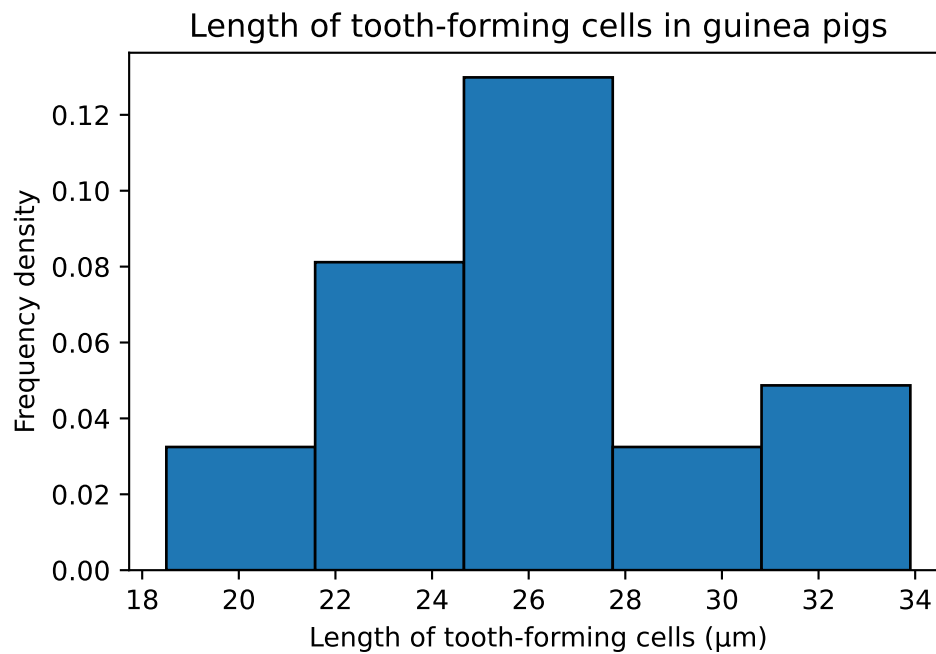


2.8 Frequency Density

```
plt.hist(dose2, bins = 5, density = True, edgecolor = 'black')
plt.title('Length of tooth-forming cells in guinea pigs')

# Axis labels
plt.xlabel('Length of tooth-forming cells (m)')
plt.ylabel('Frequency density')

plt.show()
```



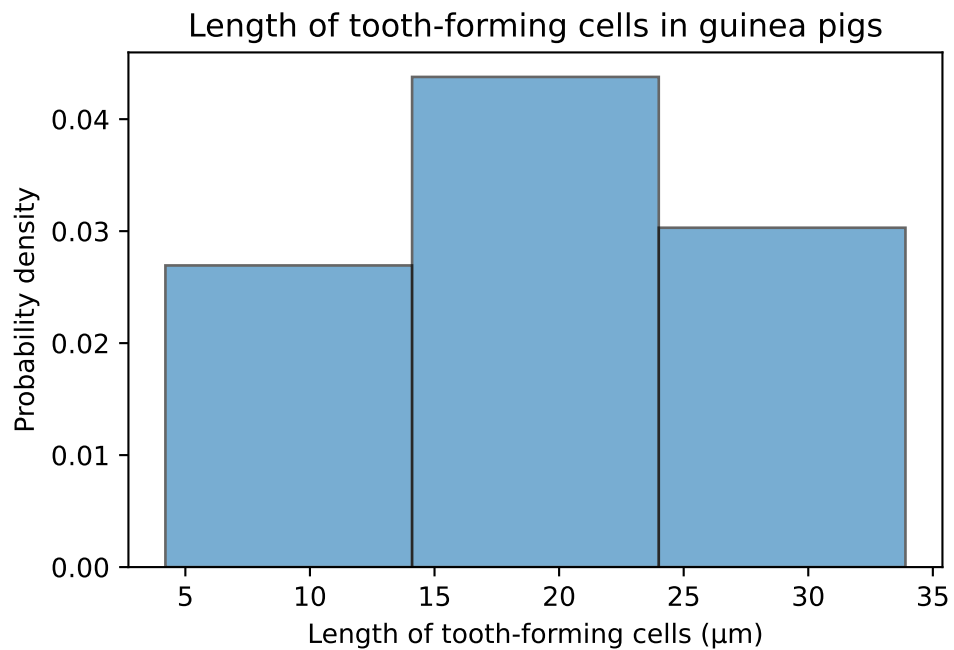
Histograms are not always the best choice to characterize the distribution of a sample. The visual impression depends on the number of bins chosen – for example:

2.9 3 Bins

```
plt.hist(guinea_pigs['len'], bins = 3, density = True, edgecolor = 'black', alpha = 0.5)
plt.title('Length of tooth-forming cells in guinea pigs')

# Axis labels
plt.xlabel('Length of tooth-forming cells (m)')
plt.ylabel('Probability density')

plt.show()
```

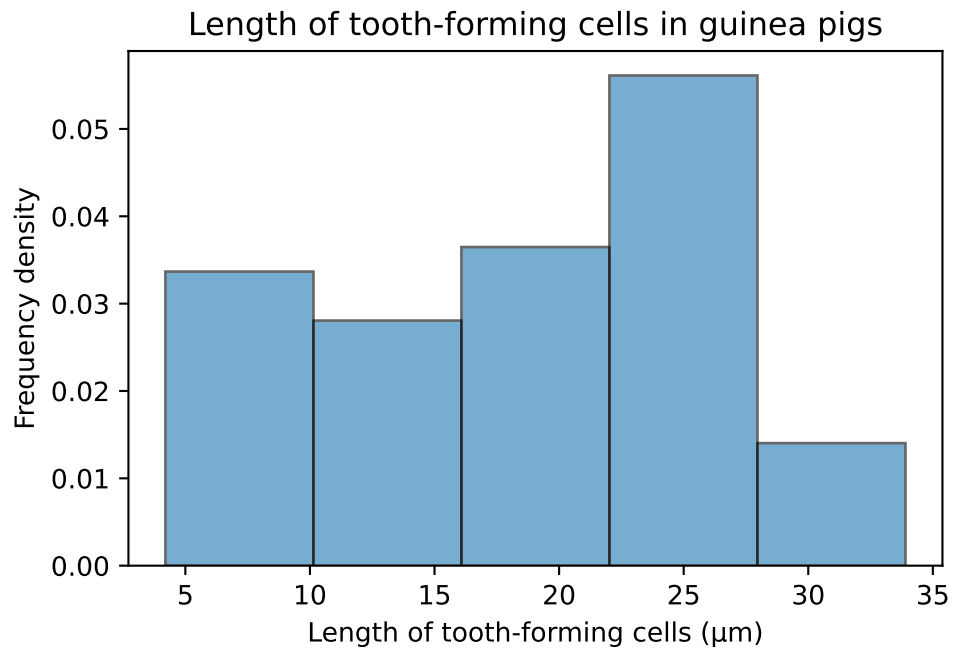


2.10 5 Classes

```
plt.hist(guinea_pigs['len'], bins = 5, density = True, edgecolor = 'black', alpha = 0.5)
plt.title('Length of tooth-forming cells in guinea pigs')

# Axis labels
plt.xlabel('Length of tooth-forming cells (μm)')
plt.ylabel('Frequency density')

plt.show()
```

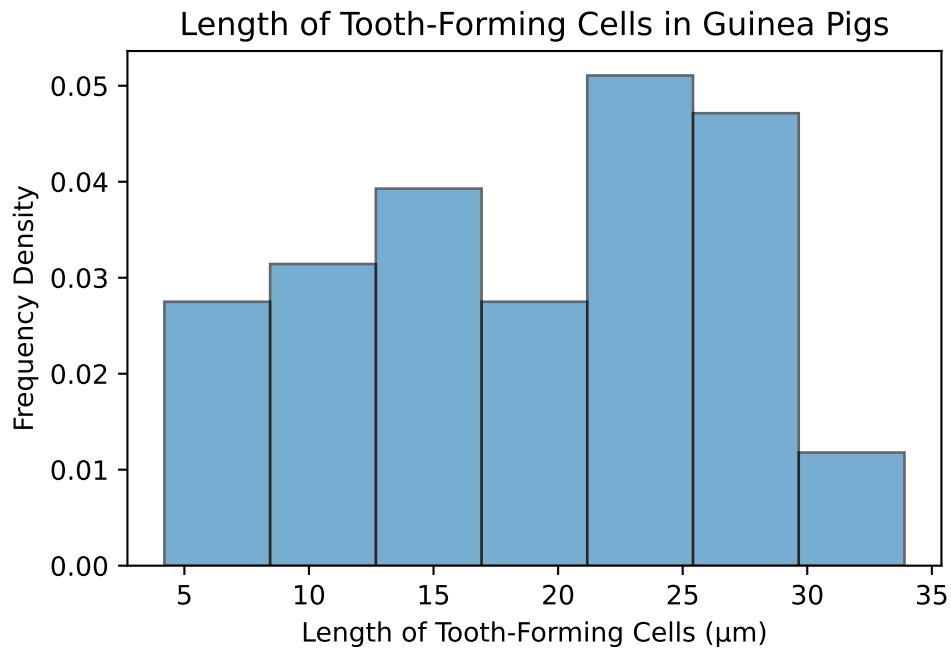


2.11 7 Classes

```
plt.hist(guinea_pigs['len'], bins = 7, density = True, edgecolor = 'black', alpha = 0.7)
plt.title('Length of Tooth-Forming Cells in Guinea Pigs')

# Axis labels
plt.xlabel('Length of Tooth-Forming Cells (μm)')
plt.ylabel('Frequency Density')

plt.show()
```



The probability density function (PDF) of the normal distribution describes the proportion of values that lie within a specific range. When calculating relative frequencies in Example 2, we saw that the sum of relative frequencies is 1. This corresponds to the area under the density curve.

The probability density function of the normal distribution is defined as:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

The shape of the normal distribution is determined by the factor $e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$ in the function. The maximum of the function occurs at $x = \mu$. From there, it decreases symmetrically and

approaches the x-axis. The decline of the function is faster when σ is smaller. The inflection points of the curve are located one standard deviation away from the mean.

A normal distribution with mean $\mu = 0$ and standard deviation $\sigma = 1$ is called the standard normal distribution.

(Baitsch, Matthias 2019, pp. 51–54)

2.12 Adjusting a Normal Distribution

To approximate the distribution of a dataset with a normal distribution, its mean and standard deviation are inserted into the normal distribution's function. In Python, these calculations can be performed directly. More convenient are the functions provided by the SciPy package, which will be presented in detail in the next section. The following example shows calculation and visualization using Python and SciPy.

i Note 2: Calculating and plotting density curves

Let's consider the distribution statistics of the group of guinea pigs that received a dose of 2 milligrams of vitamin C.

```
print(distribution_characteristics(dose2), "\n");

dose2_mean = distribution_characteristics(dose2, output = False)[1]
dose2_std = distribution_characteristics(dose2, output = False)[4]

print("Exact mean:", dose2_mean)
print("Exact standard deviation:", dose2_std)
```

```
N: 20
Arithmetic mean: 26.10
Sample error: 0.84
Sample variance: 14.24
Sample standard deviation: 3.77
None
```

```
Exact mean: 26.1
Exact standard deviation: 3.7741503052098744
```

If we insert the standard deviation and the arithmetic mean into the normal distribution function, we get:

$$f(x) = \frac{1}{3.7742\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

$$f(x) = 0.1057 \cdot e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

In Python, these calculations can be implemented and visualized:

```
# Plot histogram of probability density

plt.hist(dose2, bins=7, density=True, edgecolor='black', alpha=0.6)
plt.title('Length of tooth-forming cells in guinea pigs')

# Axis labels

plt.xlabel('Length of tooth-forming cells (m)')
plt.ylabel('Probability density')

# Calculate normal distribution

hist, bin_edges = np.histogram(dose2, bins=7)

x_values = np.linspace(min(bin_edges), max(bin_edges), 100)

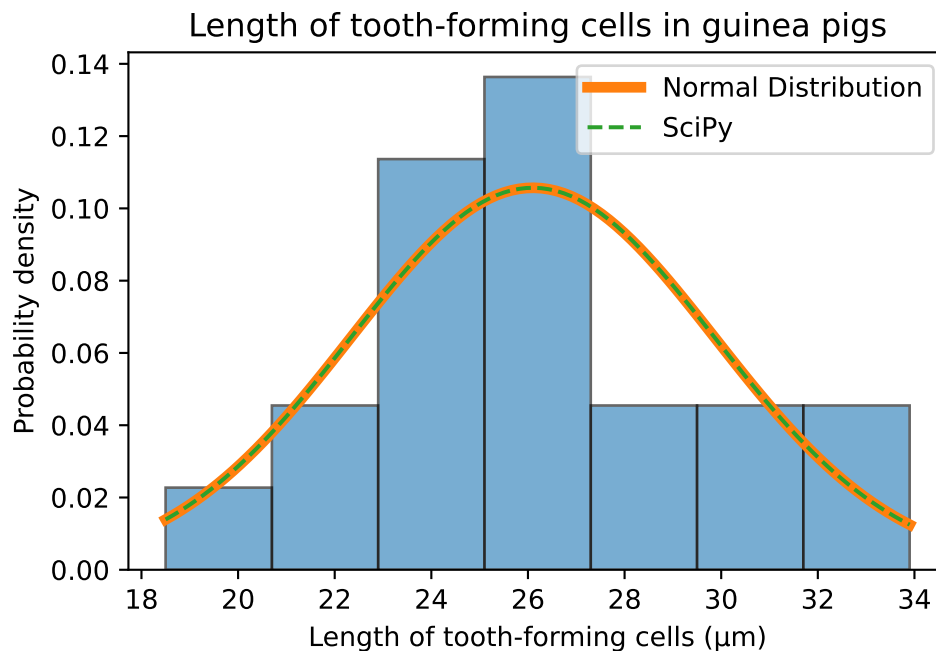
## Compute normal distribution function in Python

y_values = 1 / (dose2_std * np.sqrt(2 * np.pi)) * np.exp(- (x_values - dose2_mean) ** 2 / (2 * dose2_std ** 2))
plt.plot(x_values, y_values, label='Normal Distribution', lw=4)

## Using SciPy

y_values_scipy = scipy.stats.norm.pdf(x_values, loc=dose2_mean, scale=dose2_std)
plt.plot(x_values, y_values_scipy, label='SciPy', linestyle='dashed')

plt.legend()
plt.show()
```



The distribution of the length of tooth-forming cells in guinea pigs that received a dose of 2 milligrams of vitamin C could follow a normal distribution. However, due to the small sample size, this is difficult to assess.

2.13 The SciPy Package

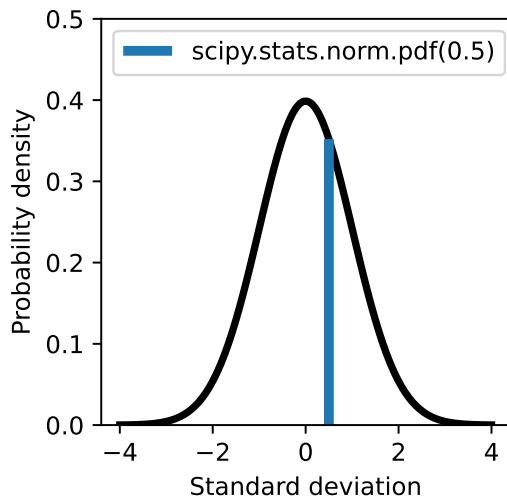
Functions for calculating density curves can be imported via the SciPy package. The `stats` module (statistical functions) includes numerous functions for hypothesis testing. Functions for the normal distribution are called as follows:

```
import scipy
print("Probability density of the normal distribution at x = 0:", scipy.stats.norm.pdf(0), "
```

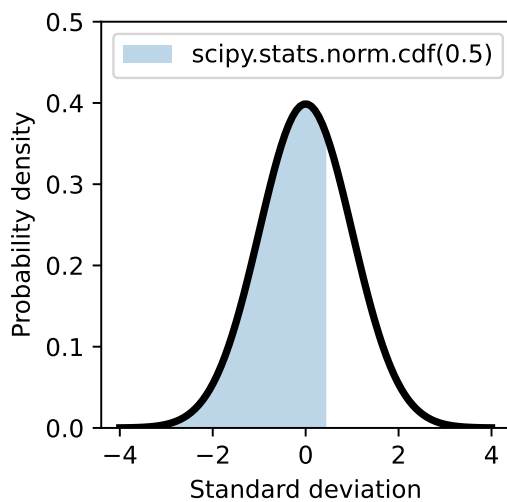
```
Probability density of the normal distribution at x = 0: 0.3989422804014327
```

For the normal distribution, four functions are relevant:

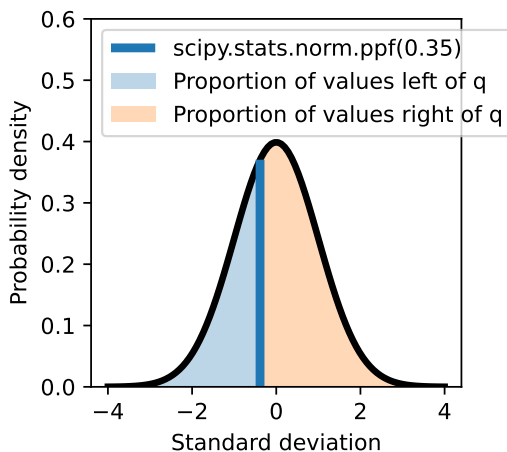
The shape of the normal distribution can be adjusted using the parameters `loc = mean` and `scale = standard_deviation`. By default, the standard normal distribution with `loc = 0` and `scale = 1` is used. The function parameters can be single values (scalars) or arrays/lists.



Description The function `scipy.stats.norm.pdf(x)` calculates the density of the normal distribution at the point `x` (pdf = probability density function). `x` can also be an array — the curve on the left was calculated using the command `scipy.stats.norm.pdf(np.linspace(-4, 4, 100))`.



Description The function `scipy.stats.norm.cdf(x)` calculates the proportion of values to the left of `x` (cdf = cumulative density function).



Description The function `scipy.stats.norm.ppf(q)` is the

2.14 Normal Distribution Exercises

You may have heard of Mensa International, an organization for highly gifted individuals. To become a member of this organization, one must have a higher intelligence quotient (IQ) than 98% of the population in their country of origin ([Wikipedia](#)).

1. If the average IQ is 100 and the standard deviation is 15, what IQ would you need to be accepted into Mensa International?
2. Mensa International is not the only organization of its kind. Other organizations have even stricter criteria. What IQ is required to become a member of these organizations?
 - Intertel (criterion: IQ in the top 1%)
 - Triple Nine Society (criterion: IQ in the top 0.1%)
 - Prometheus Society (criterion: IQ in the top 0.003%)
3. IQ is not equivalent to innate intelligence and also depends on how much opportunity someone has had for mental training, e.g., through schooling. The lowest average IQ was measured at 71 [in the country Niger](#). Assuming you have an IQ of 100, would you meet the Mensa International criterion in Niger?

Sample Solution Normal Distribution

Exercise 1: An IQ of more than ...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.98))
```

130.80623365947733

Exercise 2:

You need an IQ of at least...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.99))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.999))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 1 - (0.003 / 100)))
```

134.8952181106126

146.3534845925172

160.19216216677682

Exercise 3: Not quite.

```
print(scipy.stats.norm.cdf(loc=71, scale=15, x=100))
```

0.9734024259789904

By the way, as [Der Spiegel reports](#), students with average IQs perform just as successfully as highly gifted students, provided they are curious enough and work diligently.

2.15 Confidence Intervals

Inferential statistics is based on the principle of drawing conclusions about the true population value from sample values. The reasoning is as follows:

1. When a sample is drawn from a population, the sample values are normally distributed around the population mean. For a normal distribution:
 - 68.27% of all values lie within $\pm 1 s$,
 - 95.45% of all values lie within $\pm 2 s$, and
 - 99.73% of all values lie within $\pm 3 s$.
2. With the same probability distribution, the unknown population mean is distributed around a random value from the sample.
3. The expected value can be estimated with a certain probability using the standard error of the mean of a sample. A confidence level is chosen, i.e., the probability that the expected value actually lies within the estimated range. The opposite case, where the expected value does not lie within the estimated range, is called the significance or alpha level, denoted by the Greek letter α . α ranges from 0 to 1, and the confidence level is $1 - \alpha$ (see: [Type I and II errors](#)).
 - the expected value lies in 68.27% of cases within $\pm 1 \frac{s}{\sqrt{n}}$,
 - the expected value lies in 95.45% of cases within $\pm 2 \frac{s}{\sqrt{n}}$, and
 - the expected value lies in 99.73% of cases within $\pm 3 \frac{s}{\sqrt{n}}$.

Often, the alpha level $\alpha = 0.05$ or a 95% confidence interval is chosen, which corresponds to $\pm 1.96 \frac{s}{\sqrt{n}}$. This, however, applies only to large samples. For small sample sizes, sample means follow the t-distribution, which will be introduced in the next section.

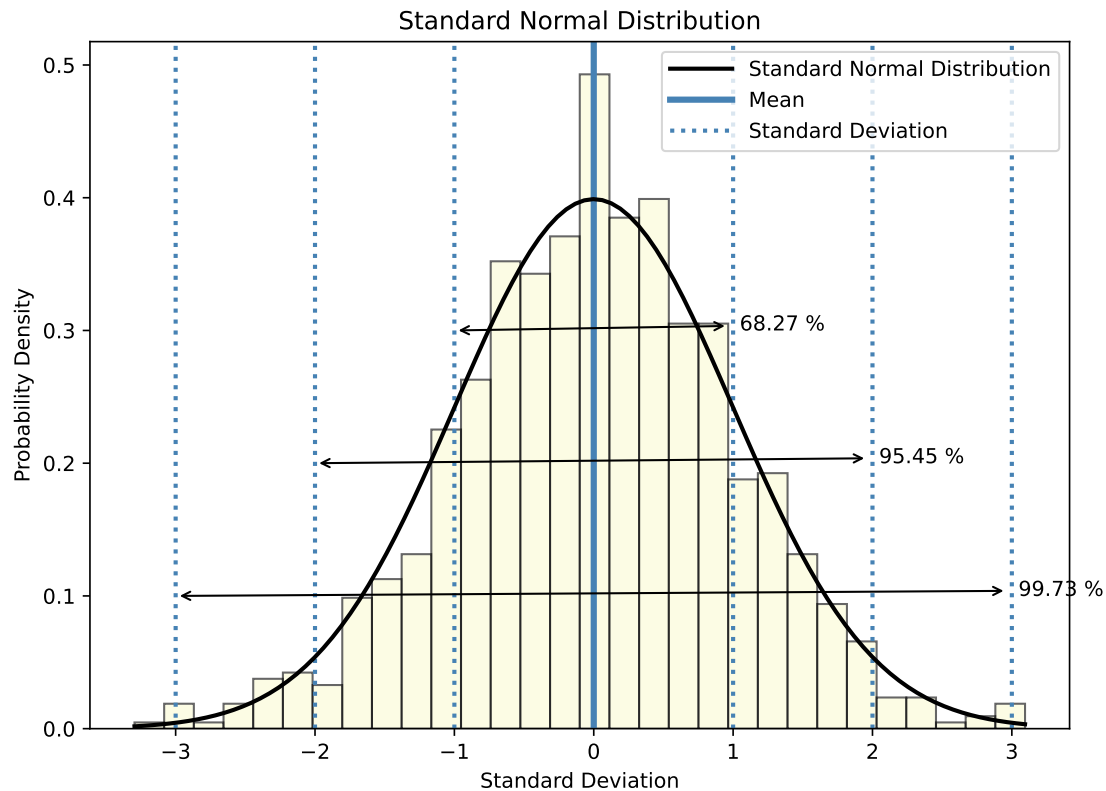
to do Maik: here one could / should introduce one-sided and two-sided hypothesis tests and the concept of “alpha-half”. This could also be illustrated nicely with arrows going only to the right or both sides.

The following example visually demonstrates the idea that with a certain probability, one can infer the population mean (expected value) from the sample mean.

2.16 Standard Normal Distribution

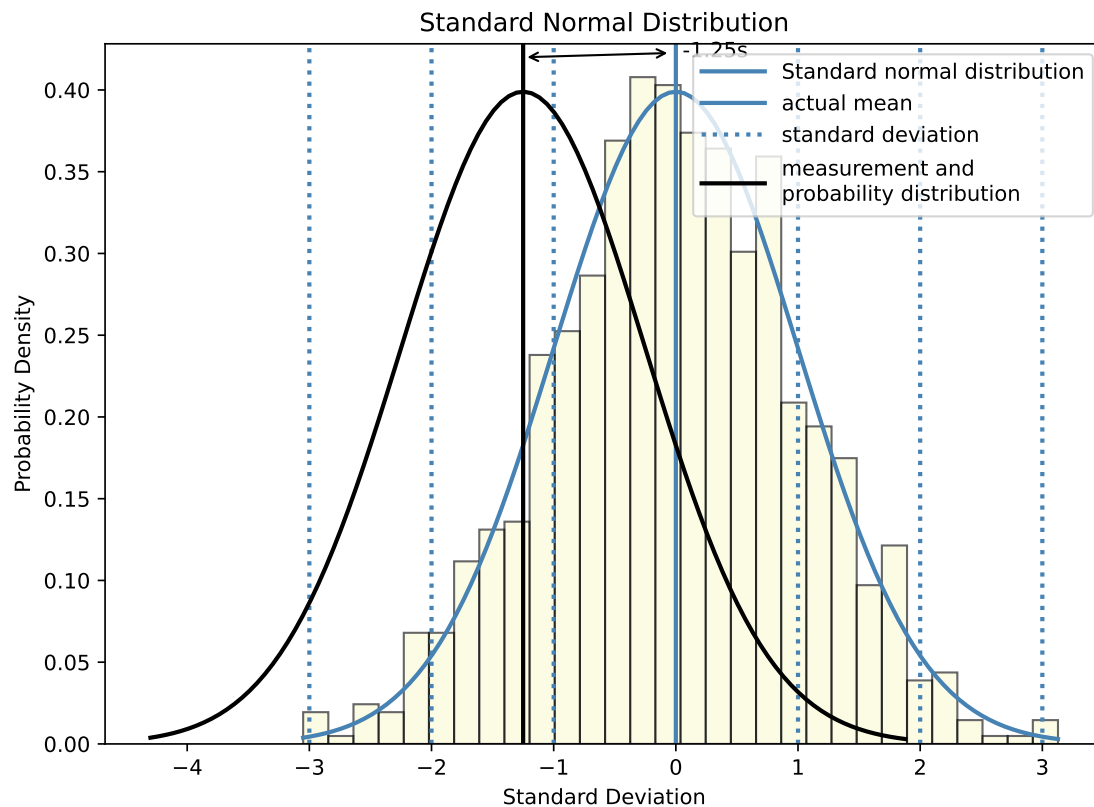
In a normal distribution, values near the expected value occur more frequently than values far from it. How often or rarely a value occurs relative to the mean can be expressed using the standard deviation.

The chart below shows randomly drawn values and a normal distribution curve.



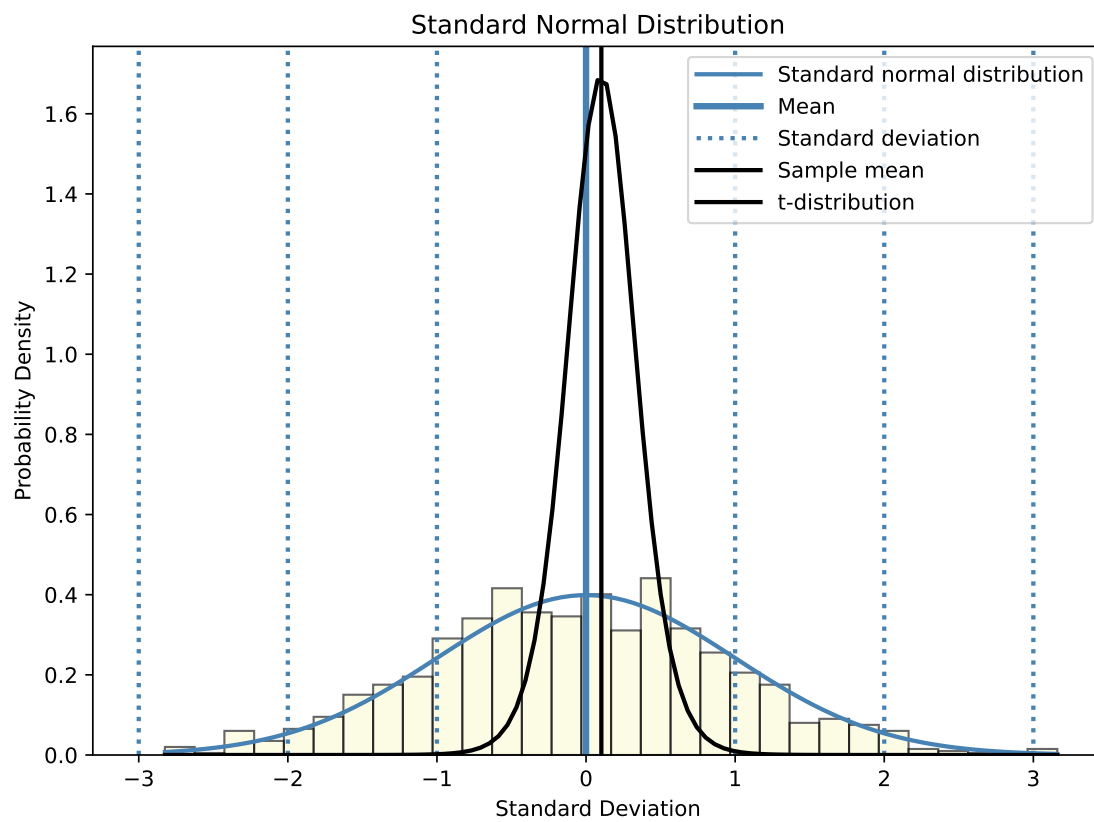
2.17 Single Measurement

A single measurement from the distribution almost never equals the expected value. However, one knows that values close to the expected value occur more frequently than those far away.



2.18 Sample N = 12

The mean of a sample almost never equals the expected value. However, the expected spread, expressed in units of the standard error of the mean, is significantly narrower than that of a single observation. This is also clearly visible in the graph. The density curve is much narrower and taller than the normal distribution curve of a single measurement. This is due to the small standard deviation in the sample of ~ 0.2 , which compresses the curve.



2.19 Code

Code for the panel Sample $N = 12$

```

import numpy as np
import matplotlib.pyplot as plt
import scipy

# Parameters of the standard normal distribution
mu, sigma = 0, 1 # mean and standard deviation

# Generate data
seed = 4
np.random.seed(seed = seed)
data = np.random.default_rng().normal(mu, sigma, 1000)

# Plot
plt.figure(figsize = (8.5, 6))

# Plot histogram
array, bins, patches = plt.hist(data, bins = 30, density = True, alpha = 0.6, color = 'lightblue')

# Draw mean line
mean_line = plt.axvline(mu, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# Draw positive and negative standard deviations
pos_std_lines = [plt.axvline(mu + i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]
neg_std_lines = [plt.axvline(mu - i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]

# Normal distribution curve
x_values = np.linspace(min(bins), max(bins), 100)
y_values = 1 / (sigma * np.sqrt(2 * np.pi)) * np.exp(- (x_values - mu) ** 2 / (2 * sigma ** 2))
normal_dist_curve = plt.plot(x_values, y_values, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# Sample
N = 12
np.random.seed(seed = 4)
sample = np.random.default_rng().normal(mu, sigma, N)

sample_std = sample.std(ddof = 1)
sample_mean = sample.mean()
standard_error = sample_std / np.sqrt(len(sample))

# Histogram calculation for sample
# hist, bins = np.histogram(sample, bins = 30, density = True)

# Standard error curve for sample
# x_values = np.linspace(min(bins), max(bins), 100)
x = np.linspace(sample_mean - 4 * sample_std, sample_mean + 4 * sample_std, 100)
y_values = scipy.stats.t.pdf(x = x_values, df = N - 1, loc = sample_mean, scale = standard_error)

# Draw sample mean
mean_sample = plt.axvline(sample_mean, color = 'black', linestyle = 'solid', linewidth = 2)

# Draw sample distribution curve
sample_dist_curve = plt.plot(x_values, y_values, color = 'black', linestyle = 'solid', linewidth = 3)

```

2.20 The t-Distribution

The t-distribution was discovered by William Sealy Gosset (though not the very first) and popularized. It is also known as Student's t-distribution: Since Gosset's employer, the Guinness Brewery, did not allow him to publish the discovery, Gosset published under the pseudonym Student. ([Wikipedia](#))

The t-distribution describes the distribution of sample means with unknown population variance, where the standard error is estimated using the sample standard deviation. Compared to the normal distribution, the t-distribution has the [degrees of freedom](#) as an additional parameter.

! Important 3: Degrees of Freedom

“The number of independent pieces of information that go into estimating a parameter is called the degrees of freedom. In general, the degrees of freedom of an estimate of a parameter equal the number of independent individual pieces of information used in the estimate minus the number of parameters that are estimated as intermediate steps in calculating the parameter itself. For example, n values are used to compute the sample variance. Nevertheless, the degrees of freedom are $n - 1$, because the mean is estimated as an intermediate step, losing one degree of freedom.”

Degrees of freedom (statistics), by various [authors](#), licensed under [CC BY-SA 4.0](#), retrieved from [Wikipedia](#). 2025

The general probability density function of the t-distribution is:

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

- ν (nu) is the degrees of freedom.
- Γ is the [Gamma function](#), which for integer arguments n satisfies $\Gamma(n) = (n-1)!$.

Since the degrees of freedom for calculating the sample mean is $n - 1$, it can also be written as:

$$f(x) = \frac{\Gamma\left(\frac{n}{2}\right)}{\sqrt{(n-1)\pi} \Gamma\left(\frac{n-1}{2}\right)} \left(1 + \frac{x^2}{n-1}\right)^{-\frac{n}{2}}$$

- n is the sample size.

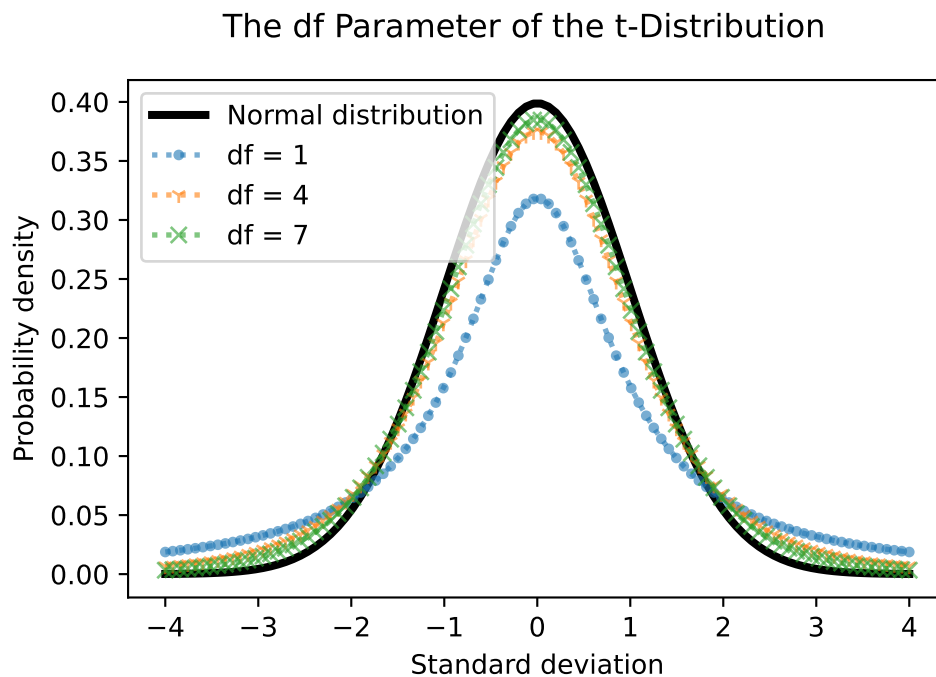
The `scipy.stats` module provides functions to compute the t-distribution:

- `scipy.stats.t.pdf(x, df, loc=0, scale=1)` computes the probability density for values `x` for a t-distribution with `df` degrees of freedom, mean `loc`, and standard deviation `scale` (pdf = probability density function).
- `scipy.stats.t.cdf(x, df, loc=0, scale=1)` computes the cumulative probability left of `x` (cdf = cumulative density function).
- `scipy.stats.t.ppf(q, df, loc=0, scale=1)` is the quantile function of the t-distribution (ppf = percentile point function).
- `scipy.stats.t.rvs(df, loc=0, scale=1, size=1)` generates `size` random samples from the t-distribution.

Function parameters can be single values (scalars) or arrays/lists.

As the sample size increases, the t-distribution approaches the normal distribution. A common rule of thumb is $n > 30$. The graphic below shows the convergence of the t-distribution to the normal distribution.

2.21 Plot



2.22 Code

```
x_values = np.linspace(-4, 4, 100)

# Normal distribution

y_values = scipy.stats.norm.pdf(x_values)
plt.plot(x_values, y_values, color='black', lw=3, label='Normal distribution')

# plt.ylim(bottom=0, top=0.5)

# t-distributions

marker = [".", "1", "x"]

[plt.plot(x_values, scipy.stats.t.pdf(x_values, df=(i + (i - 1) * 2)), linestyle='dotted', m

plt.suptitle('The df Parameter of the t-Distribution')
plt.xlabel('Standard deviation')
plt.ylabel('Probability density')
plt.legend(loc='upper left')
plt.show()
```

The maximum of the t-distribution is less dense, but the tails of the distribution are denser than those of the normal distribution.

Thus, for the t-distribution of sample means, we have:

$$\bar{x} \pm t_{n-1} \cdot \frac{s}{\sqrt{n}}$$

- t is the return value of the function `scipy.stats.t.ppf(q, df = n - 1, loc = 0, scale = 1)`
- q is the chosen alpha level, or for a two-sided hypothesis test $\frac{\alpha}{2}$ and $1 - \frac{\alpha}{2}$.
- The result is the return value of the functions:
 - `scipy.stats.t.ppf(q = alpha/2, df = n - 1, loc = sample_mean, scale = sample_standard_error)`
 - `scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 1, loc = sample_mean, scale = sample_standard_error)`

💡 Tip 2: t-distribution or normal distribution?

On a computer, the t-distribution is just as easy (or difficult) to compute as the normal distribution. Since the t-distribution approaches the normal distribution for larger sample sizes anyway, it is recommended to always use the t-distribution.

2.22.1 Example: Weight of female penguins

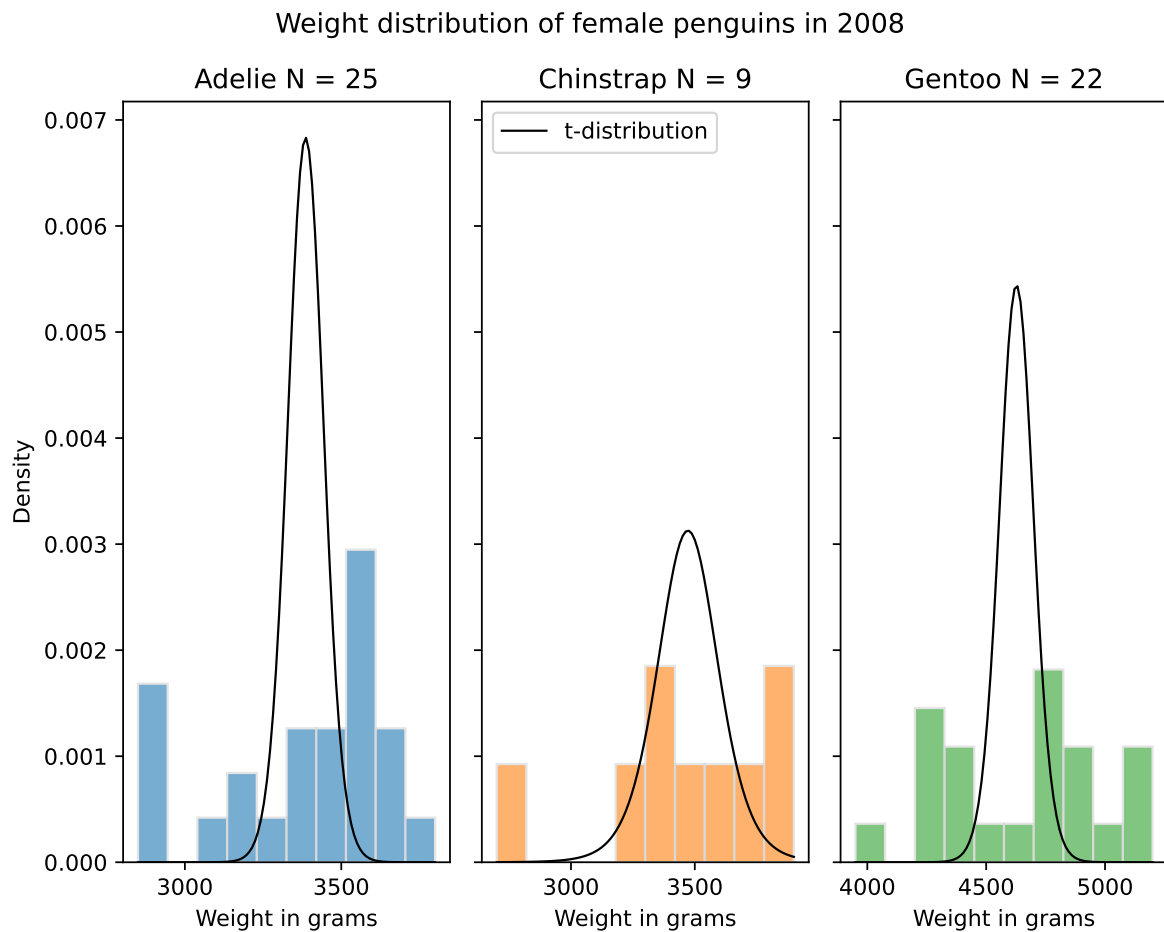
The t-distribution of the estimated sample mean for small sample sizes is shown for female penguins observed in the year 2008.

```
print(penguins.groupby(by = [penguins['species'], penguins['sex'], penguins['year']]).size())
```

species	sex	year	
Adelie	female	2007	22
		2008	25
		2009	26
	male	2007	22
		2008	25
		2009	26
Chinstrap	female	2007	13
		2008	9
		2009	12
	male	2007	13
		2008	9
		2009	12
Gentoo	female	2007	16
		2008	22
		2009	20
	male	2007	17
		2008	23
		2009	21

dtype: int64

2.23 Graphics



2.24 Code

```
year = 2008
sex = 'female'
species = 'Adelie'

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Weight distribution of female penguins in 2008')

# Adelie
```

```

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
sample_size = data.size

## Histogram

ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
ax1.set_xlabel('Weight in grams')
ax1.set_ylabel('Density')
ax1.set_title(label = str(species) + " N = " + str(sample_size))

## t-distribution of the sample mean

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
standard_error = sample_std / np.sqrt(sample_size)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = sample_mean, scale = standard_error, df = sample_size)

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 't-distribution')
ax1.legend(loc = 'upper left')

# Chinstrap

species = 'Chinstrap'

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
sample_size = data.size

## Histogram

ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Weight in grams')
ax2.set_title(label = str(species) + " N = " + str(sample_size))

## t-distribution of the sample mean

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
standard_error = sample_std / np.sqrt(sample_size)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)

```

```

y_values = scipy.stats.t.pdf(x_values, loc = sample_mean, scale = standard_error, df = sample_size)

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo

species = 'Gentoo'

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
sample_size = data.size

## Histogram

ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Weight in grams')
ax3.set_title(label = str(species) + " N = " + str(sample_size))

## t-distribution of the sample mean

sample_mean = data.mean()
sample_std = data.std(ddof = 1)
standard_error = sample_std / np.sqrt(sample_size)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = sample_mean, scale = standard_error, df = sample_size)

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

2.25 Exercise: Confidence Intervals

1. Estimate the body weight of female penguins of the species Adelie, Chinstrap, and Gentoo observed in the year 2008.
2. Which confidence interval can you report for the mean values if a confidence level of 90% is required?

💡 Tip 3: Tip and Sample Solution

The following steps will help you solve the task:

1. Determine the sample mean $\bar{x}$.
2. Determine the sample standard deviation s , the sample size N , and the standard error $\frac{s}{\sqrt{N}}$.
3. Determine the z- or t-values from the normal or t-distribution for the chosen confidence level — for a two-sided hypothesis test use $\frac{\alpha}{2}$ and $1 - \frac{\alpha}{2}$.
4. Compute the confidence interval $\bar{x} \pm t_{\alpha/2} \frac{s}{\sqrt{n}}$.

Sample Solution

Define alpha level and select penguins

```
alpha = 1 - 0.9
data = penguins[(penguins['sex'] == sex) & (penguins['year'] == year)]
```

1. Determine sample means.

```
penguin_means = data['body_mass_g'].groupby(by=data['species']).mean()
print(penguin_means)
```

```
species
Adelie      3386.000000
Chinstrap   3472.222222
Gentoo      4627.272727
Name: body_mass_g, dtype: float64
```

2. Determine sample standard deviation, sample size, and standard error

```
penguin_stds = data['body_mass_g'].groupby(by = data['species']).std(ddof = 1)
penguin_sizes = data['body_mass_g'].groupby(by = data['species']).size()
penguin_stderrors = penguin_stds / np.sqrt(penguin_sizes)

print("Sample standard deviations:\n", penguin_stds)
print("\nSample sizes:\n", penguin_sizes)
print("\nStandard errors:\n", penguin_stderrors)
```

Sample standard deviations:

```
species
Adelie      288.862712
```

```
Chinstrap    370.903551
Gentoo       339.722321
Name: body_mass_g, dtype: float64
```

Sample sizes:

```
species
Adelie      25
Chinstrap    9
Gentoo      22
Name: body_mass_g, dtype: int64
```

Standard errors:

```
species
Adelie      57.772542
Chinstrap   123.634517
Gentoo      72.429042
Name: body_mass_g, dtype: float64
```

3. Determining t-Values

```
t_lower = scipy.stats.t.ppf(alpha / 2, loc = 0, scale = 1, df = penguin_sizes - 1)
print("t-value lower interval limit:", t_lower)

t_upper = scipy.stats.t.ppf(1 - alpha / 2, loc = 0, scale = 1, df = penguin_sizes - 1)
print("t-value upper interval limit:", t_upper)
```

```
t-value lower interval limit: [-1.71088208 -1.85954804 -1.7207429 ]
t-value upper interval limit: [1.71088208 1.85954804 1.7207429 ]
```

4. calculating confidence intervals

```

# with scipy.stats.t.ppf
lower_intervals = scipy.stats.t.ppf(alpha / 2, loc = penguin_means, scale = penguin_std)
print("Lower interval bounds:", lower_intervals)

upper_intervals = scipy.stats.t.ppf(1 - alpha / 2, loc = penguin_means, scale = penguin_std)
print("Upper interval bounds:", upper_intervals)

print("\n'Manual' calculation:\n")

# 'manual'

print(penguin_means + t_lower * penguin_stderrors)
print(penguin_means + t_upper * penguin_stderrors)

```

```

Lower interval bounds: [3287.15799233 3242.31789851 4502.64096694]
Upper interval bounds: [3484.84200767 3702.12654593 4751.90448761]

```

```

'Manual' calculation:

```

```

species
Adelie      3287.157992
Chinstrap   3242.317899
Gentoo      4502.640967
Name: body_mass_g, dtype: float64
species
Adelie      3484.842008
Chinstrap    3702.126546
Gentoo      4751.904488
Name: body_mass_g, dtype: float64

```

3 Linear Parameter Estimation

Many physical quantities are measured indirectly. Frequently, there is a linear relationship between the measured quantity and the desired quantity. In practice, the data of a measurement series are never exact and are affected by more or less significant deviations from the true value of the desired quantity. Linear parameter estimation is a method used to determine, from a set of data, the value that minimizes the deviations of individual measured values.

Possible sources:

- <https://messtechnik-und-sensorik.org/2-kennlinien-und-messgenauigkeit/>
- <https://www.stssensors.com/de/characteristic-curve-hysteresis-measurement-error-terminology-in-pressure-measurement-technology/>

3.1 Measurement Series: Hooke's Law

[Hooke's law](#), named after the English scientist Robert Hooke, describes the relationship between the force F and the change in length Δx of a spring by the equation

$$F = k \cdot \Delta x,$$

where k is the spring constant.

The spring constant is a fundamental property of elastic materials and indicates how much force is required to stretch or compress a spring by a certain length. Hooke's law states that the deformation of an elastic body is proportional to the applied force, as long as the spring is not stretched or compressed beyond its elastic range.

In an experiment, Hooke's law was tested. A metal spring hanging from a mount has a (variable) weight attached. Below it, at some distance, an ultrasonic sensor is placed for distance measurement. The distance between the underside of the weight attached to the spring and the ultrasonic sensor is the measured distance.

The weights were determined with an accuracy of $\epsilon_m = 0.5$ g using a kitchen scale.

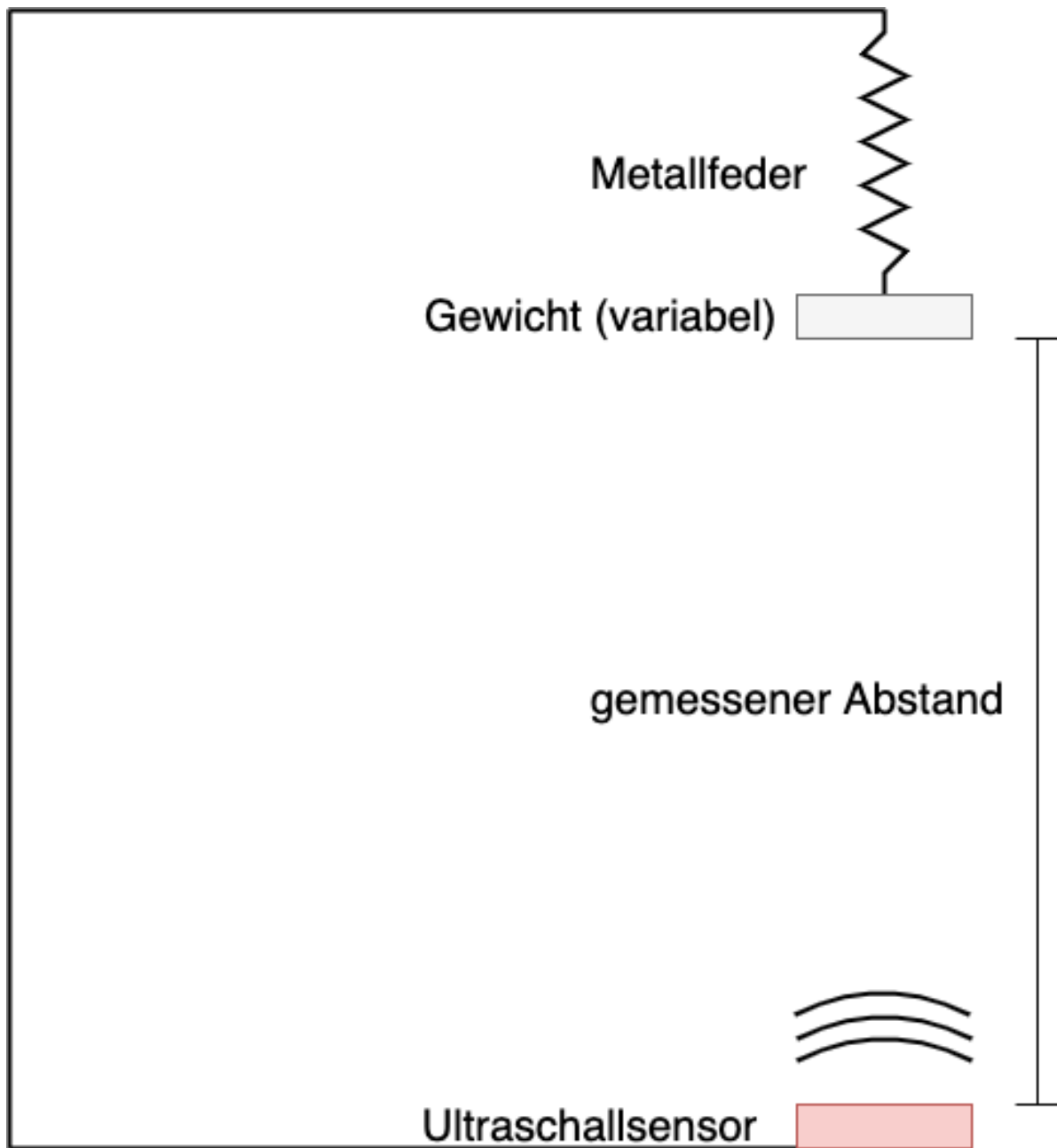


Figure 3.1: Experimental Setup

The measurement series is provided as a CSV file located at the path `01-daten/hooke_data.csv`. The file is read using Pandas.

```
file_path = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer=file_path, sep=';')
```

3.1.1 Descriptive Statistics

After loading the data, you should get an overview of it. With the methods `pd.DataFrame.head()` and `pd.DataFrame.tail()` you can view a snippet from the beginning and the end of the dataset.

```
print(hooke.head(), "\n")
print(hooke.tail())
```

```

      no  mass  distance
0     0   705    153.29
1     1   705    152.74
2     2   705    153.27
3     3   705    152.81
4     4   705    152.77
```

```

      no  mass  distance
109  109     0    173.70
110  110     0    173.44
111  111     0    173.75
112  112     0    173.30
113  113     0    200.00
```

The method `pd.DataFrame.describe()` generates descriptive statistics for the dataset.

In this case, however, the output is not particularly useful yet.

The column 'no' contains only a running experiment number, and the column 'mass' contains different weights.

```
hooke.describe()
```

	no	mass	distance
count	114.000000	114.000000	114.000000
mean	56.561404	394.921053	162.301754
std	33.131552	226.237605	7.483767
min	0.000000	0.000000	152.740000
25%	28.250000	201.000000	156.622500

	no	mass	distance
50%	56.500000	452.000000	160.720000
75%	84.750000	605.000000	167.767500
max	113.000000	705.000000	200.000000

It is more useful to have a descriptive statistic of the measured extension, grouped by the applied weight. For this, the Pandas method `pd.DataFrame.groupby()` can be used. This way, the arithmetic mean and standard deviation can be read for each of the measured weights.

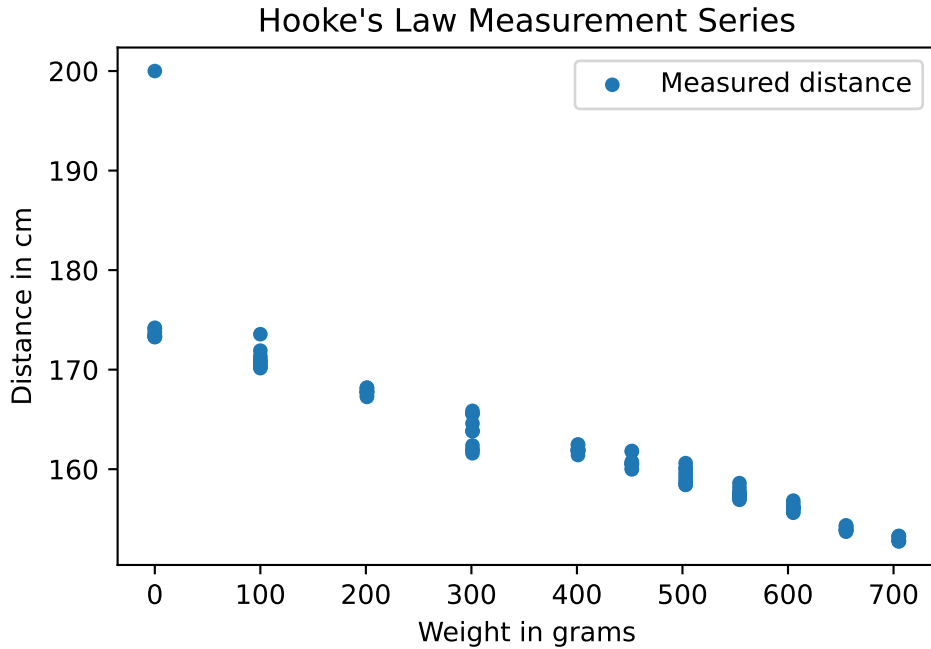
```
hooke.groupby(by='mass')['distance'].describe()
```

	count	mean	std	min	25%	50%	75%	max
mass								
0	12.0	175.828333	7.620157	173.27	173.3150	173.570	174.1125	200.00
100	11.0	171.044545	0.985833	170.15	170.3650	170.800	171.2400	173.56
201	11.0	167.791818	0.296305	167.26	167.7200	167.780	167.9750	168.19
301	10.0	163.710000	1.660977	161.60	162.0575	163.825	165.3250	165.86
401	10.0	161.967000	0.313229	161.42	161.8450	161.915	162.0250	162.48
452	10.0	160.713000	0.627854	159.98	160.4575	160.555	160.7400	161.83
503	10.0	159.314000	0.781099	158.43	158.6400	159.220	159.9650	160.61
554	10.0	157.547000	0.523791	156.92	157.2075	157.435	157.7100	158.60
605	10.0	156.142000	0.354206	155.62	156.0700	156.080	156.2075	156.84
655	11.0	154.022727	0.224414	153.72	153.8800	153.920	154.2400	154.35
705	9.0	153.008889	0.241425	152.74	152.8100	152.910	153.2700	153.29

Even at this point, the high standard deviation in the measurement series with 0 grams could be noticeable. It is, however, easier to spot when looking at the graphical representation.

```
hooke.plot(x='mass', y='distance', kind='scatter', title="Hooke's Law Measurement Series", y=
plt.legend()

plt.show()
```



Graphically, the measurement of 200 cm for the weight 0 grams stands out as strongly deviating from the other measurements.

The measurements for the weight 0 grams will be examined more closely. For this purpose, the values are displayed both in absolute terms and [standardized in units of the standard deviation \(z-scores\)](#).

A variable is standardized by subtracting the mean from each value and dividing the result by the standard deviation.

$$Z = \frac{x - \mu}{\sigma}$$

Since the population mean and standard deviation are usually unknown, the sample mean and sample standard deviation are used instead. This procedure is called *studentization*, named after the pseudonym of William Sealy Gosset, who was mentioned in the previous chapter.

$$z_i = \frac{x_i - \bar{x}}{s}$$

```

weight = 0

# calculate z-transformation manually
mean_extension = hooke[hooke['mass'] == weight].loc[:, 'distance'].mean()
std_extension = hooke[hooke['mass'] == weight].loc[:, 'distance'].std(ddof=1)

z_values = hooke[hooke['mass'] == weight].loc[:, 'distance'].apply(lambda x: (x - mean_extension) / std_extension)
z_values.name = 'z-values'

# z-transformation using scipy
## scipy returns a np.array
## for better display, convert the np.array to a pd.Series with a name attribute
scipy_z_values = pd.Series(scipy.stats.zscore(hooke[hooke['mass'] == weight].loc[:, 'distance']), index=hooke[hooke['mass'] == weight].index)
scipy_z_values.name = 'scipy z-values'

# combined output of the data
print(pd.concat([hooke[hooke['mass'] == weight], z_values, scipy_z_values], axis=1))

```

	no	mass	distance	z-values	scipy z-values
102	102.0	0.0	173.32	-0.329171	NaN
103	103.0	0.0	174.11	-0.225498	NaN
104	104.0	0.0	173.42	-0.316048	NaN
105	105.0	0.0	174.12	-0.224186	NaN
106	106.0	0.0	173.30	-0.331795	NaN
107	107.0	0.0	174.21	-0.212375	NaN
108	108.0	0.0	173.27	-0.335732	NaN
109	109.0	0.0	173.70	-0.279303	NaN
110	110.0	0.0	173.44	-0.313423	NaN
111	111.0	0.0	173.75	-0.272742	NaN
112	112.0	0.0	173.30	-0.331795	NaN
113	113.0	0.0	200.00	3.172069	NaN
0	NaN	NaN	NaN	NaN	-0.329171
1	NaN	NaN	NaN	NaN	-0.225498
2	NaN	NaN	NaN	NaN	-0.316048
3	NaN	NaN	NaN	NaN	-0.224186
4	NaN	NaN	NaN	NaN	-0.331795
5	NaN	NaN	NaN	NaN	-0.212375
6	NaN	NaN	NaN	NaN	-0.335732
7	NaN	NaN	NaN	NaN	-0.279303
8	NaN	NaN	NaN	NaN	-0.313423
9	NaN	NaN	NaN	NaN	-0.272742
10	NaN	NaN	NaN	NaN	-0.331795

11 NaN NaN NaN NaN 3.172069

The value 200 cm in row 113 appears to be incorrect. A natural extension of the spring by an additional 16 centimeters is not plausible. The z-value > 3 also marks the measurement as an [outlier](#). Therefore, this row is removed from the dataset.

! Important 4: Outlier

In statistics, a measurement is called an outlier if it deviates strongly from the rest of the data series. Multiple outliers can occur in a dataset. These values can be removed from the dataset to improve estimation if it is assumed that they are caused by measurement errors or other disturbances.

One way to identify outliers is the z-transformation. A threshold must be chosen, above which a measurement is classified as an outlier, e.g., 2.5 or 3 standard deviations. A variety of outlier tests have been developed in statistics (see [Outlier tests](#)).

However, classifying a measurement as an outlier cannot be based solely on statistical methods; it is always a judgment decision based on your domain knowledge. Not all extreme values are automatically invalid—they can occur with a certain statistical probability (see chapter on the Normal distribution). Such values are referred to as valid extreme values. Outlier by various [authors](#) is licensed under [CC BY-SA 4.0](#) and is available on [Wikipedia](#)

```
hooke.drop(index = 113, inplace = True)

hooke.groupby(by = 'mass')['distance'].describe()
```

	count	mean	std	min	25%	50%	75%	max
mass								
0	11.0	173.630909	0.367409	173.27	173.3100	173.440	173.9300	174.21
100	11.0	171.044545	0.985833	170.15	170.3650	170.800	171.2400	173.56
201	11.0	167.791818	0.296305	167.26	167.7200	167.780	167.9750	168.19
301	10.0	163.710000	1.660977	161.60	162.0575	163.825	165.3250	165.86
401	10.0	161.967000	0.313229	161.42	161.8450	161.915	162.0250	162.48
452	10.0	160.713000	0.627854	159.98	160.4575	160.555	160.7400	161.83
503	10.0	159.314000	0.781099	158.43	158.6400	159.220	159.9650	160.61
554	10.0	157.547000	0.523791	156.92	157.2075	157.435	157.7100	158.60
605	10.0	156.142000	0.354206	155.62	156.0700	156.080	156.2075	156.84
655	11.0	154.022727	0.224414	153.72	153.8800	153.920	154.2400	154.35
705	9.0	153.008889	0.241425	152.74	152.8100	152.910	153.2700	153.29

The highest standard deviation is observed for the measurement series of 301 grams. However, the measured values appear unremarkable.

```
weight = 301

## scipy returns a np.array

## for better display, the np.array is converted to a pd.Series, which has a name attribute

z_values = pd.Series(scipy.stats.zscore(hooke[hooke['mass'] == weight].loc[:, 'distance'], d
z_values.name = 'z-values'

print(pd.concat([hooke[hooke['mass'] == weight], z_values], axis=1))
```

	no	mass	distance	z-values
70	70.0	301.0	162.38	NaN
71	71.0	301.0	161.93	NaN
72	72.0	301.0	161.95	NaN
73	73.0	301.0	161.60	NaN
74	74.0	301.0	164.59	NaN
75	75.0	301.0	165.86	NaN
76	76.0	301.0	163.82	NaN
77	77.0	301.0	163.83	NaN
78	78.0	301.0	165.57	NaN
79	79.0	301.0	165.57	NaN
0	NaN	NaN	NaN	-0.800734
1	NaN	NaN	NaN	-1.071658
2	NaN	NaN	NaN	-1.059617
3	NaN	NaN	NaN	-1.270337
4	NaN	NaN	NaN	0.529809
5	NaN	NaN	NaN	1.294419
6	NaN	NaN	NaN	0.066226
7	NaN	NaN	NaN	0.072247
8	NaN	NaN	NaN	1.119823
9	NaN	NaN	NaN	1.119823

3.1.2 Exploratory Statistics

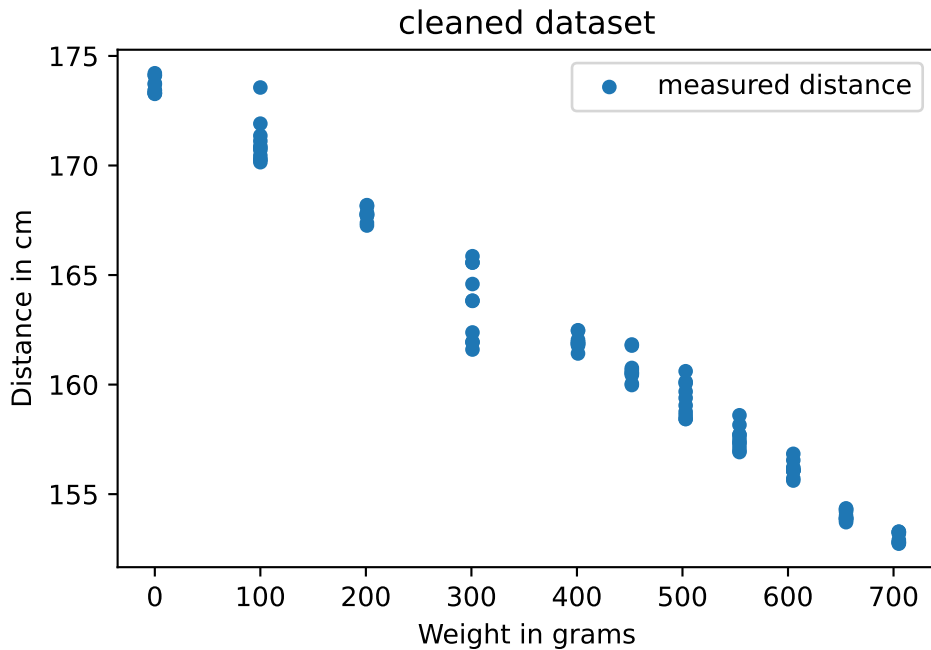
The plot of the cleaned dataset suggests a linear relationship. Additionally, the downward trend of the data points with increasing weight is striking.

```

hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'cleaned dataset', ylabel =
plt.legend()

plt.show()

```



According to the experimental setup, as the spring is stretched, the distance to the distance sensor decreases. Since the spring elongation is to be measured, it is useful to transform the data accordingly. For this purpose, the measured distance at 0 grams of weight is taken as the zero point, from which the spring elongation is measured. This means that the arithmetic mean of the elongation measured at 0 grams is subtracted from all data points, and the result is multiplied by -1.

```

zero_point = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
print(f"Zero point: {zero_point:.2f} cm")

hooke['distance'] = hooke['distance'].sub(zero_point).mul(-1)

hooke.plot(
x='mass',
y='distance',
kind='scatter',

```

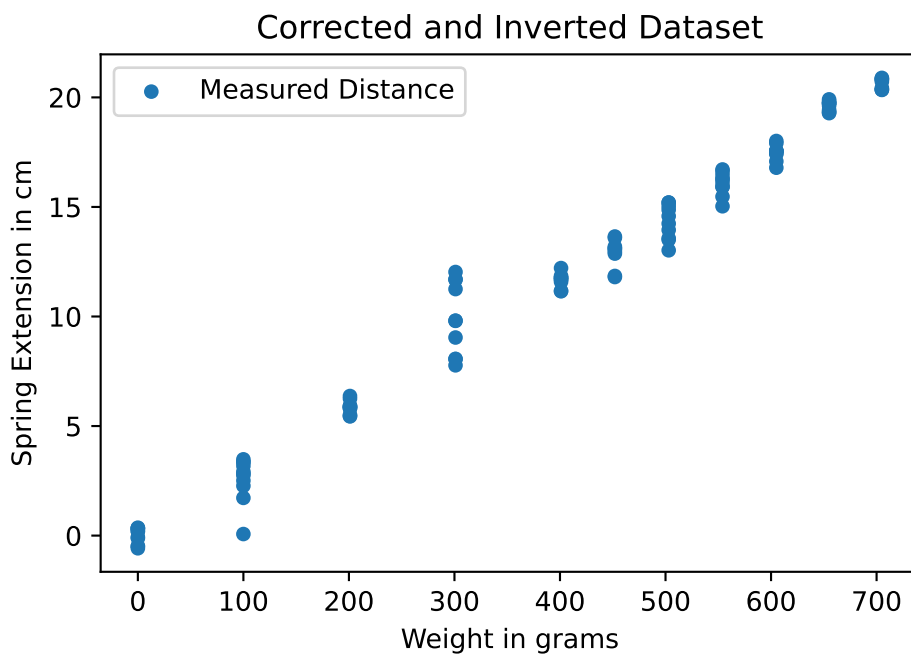
```

title='Corrected and Inverted Dataset',
ylabel='Spring Extension in cm',
xlabel='Weight in grams',
label='Measured Distance'
)
plt.legend()

plt.show()

```

Zero point: 173.63 cm



With the function `plt.errorbar()`, the means and standard errors for each weight can be displayed graphically. Since the standard errors are relatively small, horizontal lines are drawn at the ends of the error bars using the `capsize` parameter.

```

# Mean spring extension by weight

spring_means_by_weight = hooke['distance'].groupby(by=hooke['mass']).mean()
spring_means_by_weight.name = 'Spring Extension'

```

```

# Standard error by weight

spring_stderr_by_weight = hooke['distance'].groupby(by=hooke['mass']).std(ddof=1).div(np.sqrt(hooke['mass']))
spring_stderr_by_weight.name = 'Standard Error'

data_points = hooke.plot(
    x='mass', y='distance', kind='scatter',
    title='cleaned and inverted dataset',
    ylabel='Spring Extension in cm', xlabel='Weight in grams',
    alpha=0.6, label='measured spring extension'
)

# Capture legend handles

# see: [https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.get_legend_handles_labels.html#matplotlib.axes.Axes.get_legend_handles_labels]
data_points_handle, data_points_label = data_points.get_legend_handles_labels()

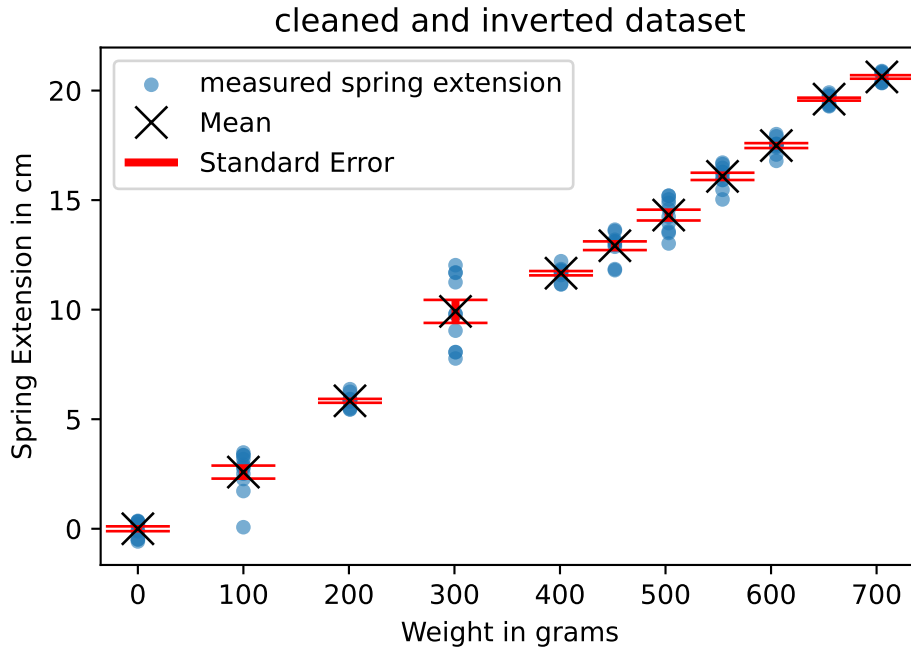
errorbar_container = plt.errorbar(
    x=spring_means_by_weight.index, y=spring_means_by_weight, yerr=spring_stderr_by_weight,
    linestyle='none', marker='x', color='black', markersize=12, elinewidth=3, ecolor='red', capsize=5
)

# Create legend manually

# see: [https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer]
plt.legend(
    [data_points_handle[0], errorbar_container.lines[0], errorbar_container.lines[2][0]],
    [data_points_label[0], 'Mean', 'Standard Error'],
    loc='upper left'
)
plt.show()

print("\n", pd.concat([spring_means_by_weight, spring_stderr_by_weight], axis=1))

```



	Spring Extension	Standard Error
mass		
0	-7.751375e-15	0.110778
100	2.586364e+00	0.297240
201	5.839091e+00	0.089339
301	9.920909e+00	0.525247
401	1.166391e+01	0.099052
452	1.291791e+01	0.198545
503	1.431691e+01	0.247005
554	1.608391e+01	0.165637
605	1.748891e+01	0.112010
655	1.960818e+01	0.067663
705	2.062202e+01	0.080475

3.2 Determining the Spring Constant

The relationship between the force F and the change in length Δx of a spring with spring constant k is described by the equation $F = k \cdot \Delta x$. Here, the force F corresponds to the weight in kilograms m multiplied by the gravitational acceleration g . On Earth, the gravitational acceleration is $9.81 \frac{m}{s^2}$.

Therefore, in the dataset, the weight recorded in the ‘mass’ column (in grams) is converted to the actual force. Similarly, the measured displacement in the ‘distance’ column is converted from centimeters to meters.

```
hooke['mass'] = hooke['mass'].div(1000).mul(9.81)
hooke.rename(columns = {'mass': 'force'}, inplace = True)

hooke['distance'] = hooke['distance'].div(100)

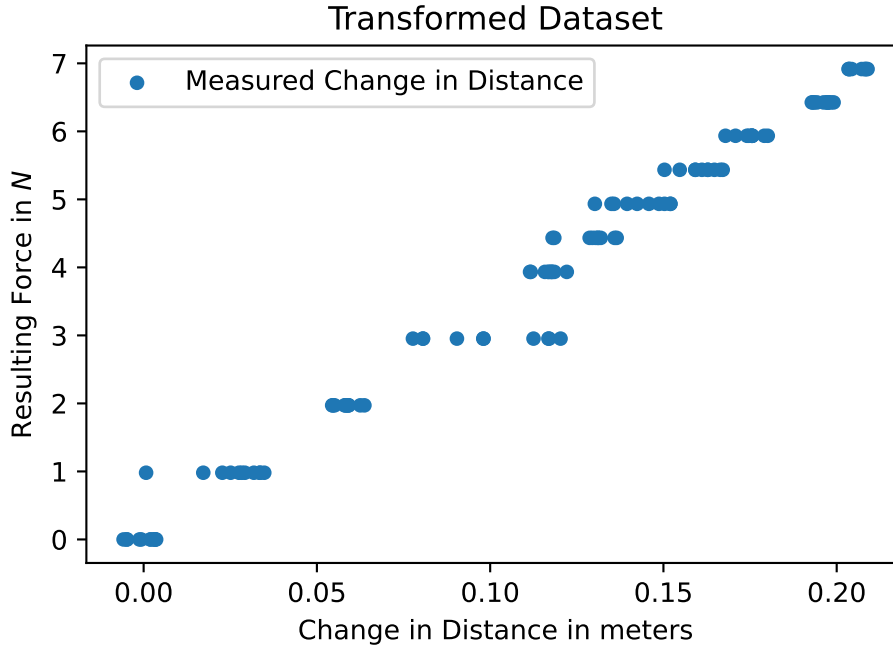
print(hooke.head())
```

	no	force	distance
0	0	6.91605	0.203409
1	1	6.91605	0.208909
2	2	6.91605	0.203609
3	3	6.91605	0.208209
4	4	6.91605	0.208609

For the graphical representation of the relationship $F = k \cdot \Delta x$, it is more practical to plot the displacement on the x-axis and the resulting force on the y-axis.

```
hooke.plot(x='distance', y='force', kind='scatter', title='Transformed Dataset', ylabel='Res')
plt.legend()

plt.show()
```



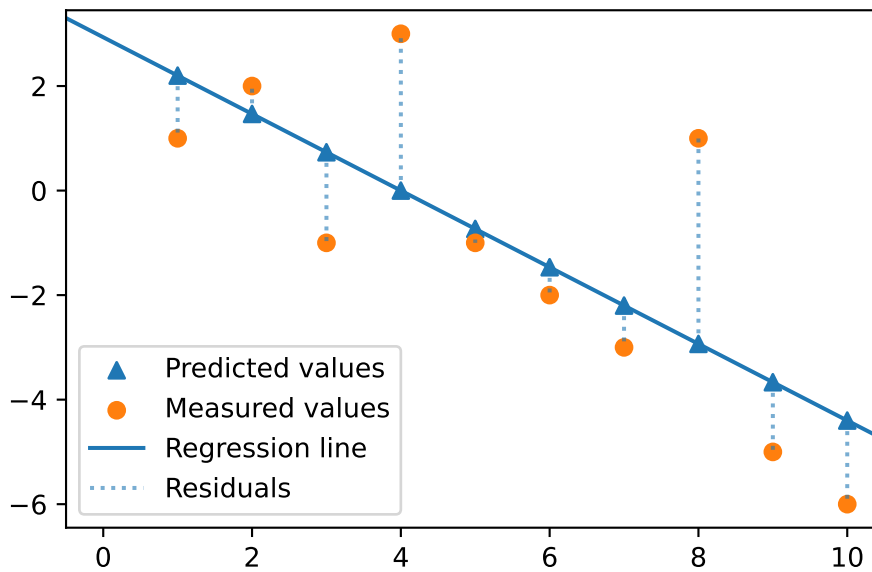
i Note 3: Least Squares Method

The least squares method aims to find the line $\hat{y} = \beta_0 + \beta_1 \cdot x$ that minimizes the squared distances between the predicted values $\hat{y}$ and the actual measured values y . The values $y_i - \hat{y}_i$ are the residuals e_i . Therefore, it holds that:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N e_i^2 = \min$$

Graphically, the minimization of the squared distances can be visualized as follows.

3.3 Plot



Regression coefficients: [2.93333333 -0.73333333]

3.4 Code

```

x = np.arange(1, 11)
y = - x.copy() + 4
y[0] -= 2
y[2] -= 2
y[3] += 3
y[-3] += 5

lm = poly.polyfit(x, y, 1)
predicted_values = poly.polyval(x, lm)

plt.scatter(x, predicted_values, label='Predicted values', marker="^", color="tab:blue")
plt.scatter(x, y, label='Measured values', marker='o', color="tab:orange")
plt.axline(xy1=(0, lm[0]), slope=lm[1], label="Regression line", color="tab:blue")
dotted = plt.vlines(x, ymin=predicted_values, ymax=y, alpha=0.6, ls='dotted', label='Residuals')

plt.legend()
plt.show()

print("Regression coefficients:", lm)

```

The drawn line corresponds to the linear function $\hat{y} = \beta_0 + \beta_1 \cdot x + e_i$. The triangle markers are the predicted values $\hat{y}_i$ of the linear model for the values $x_i = np.arange(1, 11)$. The actual measurements y are marked with circle markers. The length of the dashed lines represents the magnitude of the deviation between the observed and predicted values $y_i - \hat{y}_i$, i.e., the residuals e_i .

We are looking for the line that minimizes the sum of squared residuals. The values β_0 and β_1 we are seeking are the least squares estimators.

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

For completeness, we will derive the least squares estimators. We are searching for values of β_0 and β_1 that minimize the sum of squared residuals $\sum_{i=1}^n e_i^2$. The sum of squared residuals is the sum of the squared differences between the observed values y_i and the values predicted by the linear function.

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

We are thus examining a function that depends on two variables.

$$f(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

The summation symbol is shorthand for a sum.

$$f(\beta_0, \beta_1) = (y_1 - (\beta_0 + \beta_1 \cdot x_1))^2 + (y_2 - (\beta_0 + \beta_1 \cdot x_2))^2 + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n))^2$$

At the minimum of the function, both partial derivatives must be zero (an easy explanation is [here](#)).

Example 3.1: Partial Derivative

A partial derivative is the derivative of a function with multiple variables with respect to one variable, treating the other variables as constants.

For a function $f(x, y) = 2x + y^2$, the partial derivative with respect to x is expressed as:

$$\frac{\partial f(x, y)}{\partial x}$$

- The symbol ∂ is the cursive form of the Cyrillic lowercase letter (d) and is read as “del”. It indicates a partial derivative is being taken.
- The numerator contains the function to be differentiated. The denominator shows the variable with respect to which differentiation is performed. The term is read as “del f of x and y with respect to del x”.

The partial derivative $\frac{\partial f(x, y)}{\partial x} = 2$. y^2 is treated as a constant (e.g., 5^2) and thus its derivative is zero.

The partial derivative $\frac{\partial f(x, y)}{\partial y} = 2y$. $2x$ is treated as a constant and its derivative is zero.

In both partial derivatives, x_i and y_i are constants. In the partial derivative with respect to β_0 , β_1 is also constant; in the partial derivative with respect to β_1 , β_0 is constant.

3.5 Partial derivative with respect to the intercept

For the partial derivative with respect to β_0 , applying the chain rule for the outer function (top) and inner function (middle) gives:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (1 + 0 \cdot 0))$$

Thus, the partial derivative with respect to β_0 is:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = -2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

This can be simplified by dropping the factor -2 (since $-2 \cdot 0 = 0$) and resolving the signs. Thus:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 \cdot x_i) = 0$$

Or equivalently:

$$\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_1 \cdot x_i = 0$$

β_0 and β_1 are constants, so $\sum_{i=1}^n \beta_0 = \beta_0 \cdot \sum_{i=1}^n 1 = \beta_0 \cdot n$ and $\sum_{i=1}^n \beta_1 \cdot x_i = \beta_1 \cdot \sum_{i=1}^n 1 \cdot x_i$. Hence:

$$\sum_{i=1}^n y_i - n \cdot \beta_0 - \beta_1 \cdot \sum_{i=1}^n x_i = 0$$

Now we can divide by n . Here, $\frac{\sum_{i=1}^n y_i}{n}$ corresponds to the arithmetic mean of y , and $\frac{\sum_{i=1}^n x_i}{n}$ to the arithmetic mean of x . Thus, we obtain:

$$\bar{y} - \beta_0 - \beta_1 \cdot \bar{x} = 0$$

Rearranged:

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

3.6 Partial derivative with respect to the slope

For the partial derivative with respect to β_1 , the chain rule is applied similarly:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (0 + 1 \cdot x_i))$$

Thus:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = -2 \sum_{i=1}^n x_i \cdot (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

Dropping the factor -2 and expanding gives:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 \cdot x_i - \sum_{i=1}^n \beta_1 \cdot x_i^2 = 0$$

Factoring constants:

$$\sum_{i=1}^n x_i y_i - \beta_0 \cdot \sum_{i=1}^n x_i - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Substitute $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$ and $\sum_{i=1}^n x_i = n \cdot \bar{x}$:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \cdot \bar{x}) \cdot n \cdot \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Expanding:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - \beta_1 \cdot n\bar{x}^2 - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Combine the last two terms:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - \beta_1 \cdot (\sum_{i=1}^n x_i^2 - n\bar{x}^2) = 0$$

Solve for β_1 :

$$\beta_1 \cdot \left(\sum_{i=1}^n x_i^2 - n\bar{x}^2 \right) = \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}$$

Then:

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}}{\sum_{i=1}^n x_i^2 - n\bar{x}^2}$$

Now, we can first rewrite using $\sum_{i=1}^n x_i^2 - n\bar{x}^2 = \sum_{i=1}^n (x_i - \bar{x})^2$.

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Then—and this will be shown next—using $\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$. Thus, we obtain:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

The final step is shown starting from the result and begins by expanding the product:

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})$$

This can also be written as:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \bar{y} - \sum_{i=1}^n \bar{x} y_i + \sum_{i=1}^n \bar{x} \bar{y}$$

$\bar{x}$ and $\bar{y}$ are constants, so we can write $\bar{x} \cdot \sum_{i=1}^n y_i$ and $\bar{y} \cdot \sum_{i=1}^n x_i$. Since $\sum_{i=1}^n x_i = n \cdot \bar{x}$ (analogously for y), we obtain:

$$\sum_{i=1}^n x_i y_i - \bar{y} \cdot n \cdot \bar{x} - \bar{x} \cdot n \cdot \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Reordering:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - n\bar{x}\bar{y} + \sum_{i=1}^n \bar{x}\bar{y}$$

The last term $\sum_{i=1}^n \bar{x}\bar{y}$ can also be written as $n \cdot \bar{x}\bar{y}$, which yields:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - n\bar{x}\bar{y} + n\bar{x}\bar{y}$$

The last two terms therefore cancel out, leaving:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}$$

(Baitsch, Matthias 2019, pp. 73–74)

The functions required for this are provided both by the `numpy.polynomial` package—specifically its `numpy.polynomial.polynomial` module for polynomial functions—and by the `scipy.stats.linregress` module. In the following, the computation using NumPy is demonstrated first, followed by an introduction to the functions from the SciPy module. The way both modules work is similar.

NumPy polyfit and polyeval

```
import numpy.polynomial.polynomial as poly
```

To estimate function parameters using the least squares method, the function `poly.polyfit(x, y, deg)` is used. `x` represents the values of the independent variable, `y` the values of the dependent variable, and `deg` specifies the degree of the polynomial function to be fitted. `deg = 1` specifies a linear function.

Note 4: Explaining polyfit and polyeval

```
# Generate example data
x = np.array(list(range(0, 100)))
y = x ** 2

print(poly.polyfit(x, y, 1))
```

```
[-1617.    99.]
```

The function returns the estimated regression parameters as a NumPy array. The terms are arranged in ascending order, i.e., the intercept is at index position 0, and the slope coefficient is at index position 1. The output for a second-degree polynomial, for example, would look like this:

```
print(poly.polyfit(x, y, 2).round(2))
```

```
[-0.  0.  1.]
```

With the regression coefficients, the predicted values of the linear function can be computed. For this, the function `poly.polyeval(x, c)` is used. It calculates the function values for the values passed in `x` using the function parameters `c`. From the difference between the measured values and the predicted values, the residuals can be determined.

```
# 'manual' calculation
regressions_koeffizienten = poly.polyfit(x, y, 1)
vorhersagewerte = regressions_koeffizienten[0] + x * regressions_koeffizienten[1]
residuen = y - vorhersagewerte

# calculation with polyeval
lm = poly.polyfit(x, y, 1)
vorhersagewerte_polyval = poly.polyval(x, lm)

print("The results match:", np.equal(vorhersagewerte, vorhersagewerte_polyval).all())
print("\nExcerpt of predicted values:", vorhersagewerte[:10])
```

```
The results match: True
```

```
Excerpt of predicted values: [-1617. -1518. -1419. -1320. -1221. -1122. -
1023.  -924.  -825.  -726.]
```

The [coefficient of determination](#) R^2 indicates how well the estimated function fits the data. Its range goes from 0 to 1. A value of 1 represents a perfect fit. For a simple linear regression with only one explanatory variable, the coefficient of determination can be calculated as the square of the [Bravais–Pearson correlation coefficient](#) r . It can be obtained using the function `np.corrcoef(x, y)` (which returns a matrix of correlation coefficients).

```
print(f"r = {np.corrcoef(x, y)[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(x, y)[0, 1] ** 2:.2f}")
```

```
r = 0.97
```

```
R2 = 0.94
```

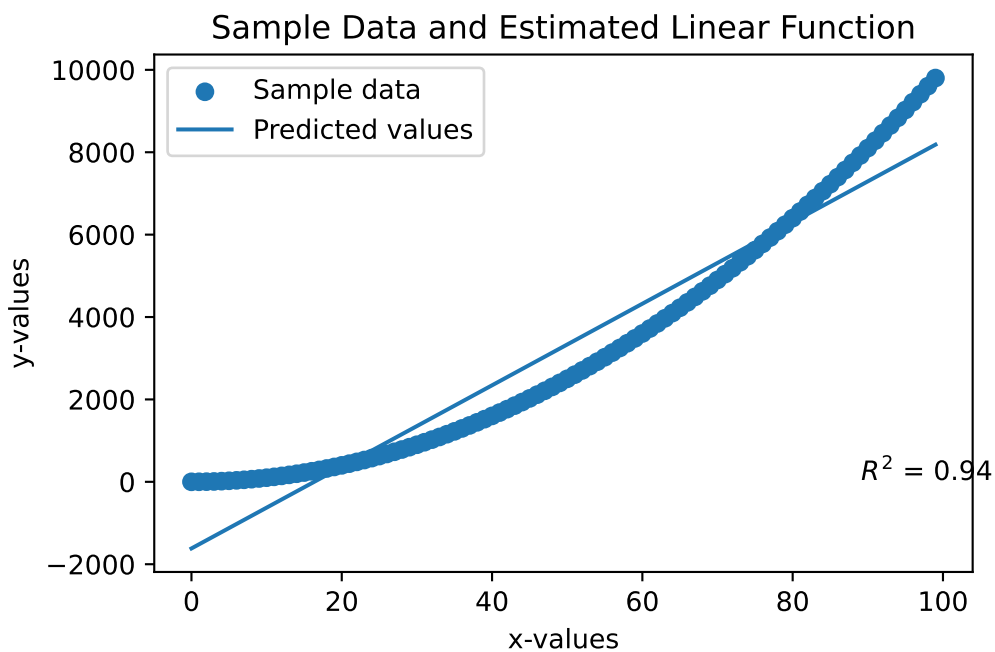
The data and the estimated line can be displayed graphically.

```
import matplotlib.pyplot as plt

plt.scatter(x, y, label='Sample data')
plt.plot(x, vorhersagewerte, label='Predicted values')
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label='Sample Data and Estimated Linear Function')
plt.xlabel('x-values')
plt.ylabel('y-values')
plt.legend()

plt.show()
```



NumPy also includes the now outdated functions `np.polyfit(x, y, deg)` and `np.polyval(p, x)`.

i Note 5: `np.polyfit` & `np.polyval`

The functions `np.polyfit(x, y, deg)` and `np.polyval(p, x)` work similarly to the functions introduced from the `numpy.polynomial.polynomial` module. However, an important difference is that **the parameters returned by `polyfit` are in reverse order**.

```
print(poly.polyfit(x, y, deg = 1))
print(np.polyfit(x, y, deg = 1))
```

```
[-1617.    99.]
[   99. -1617.]
```

Note

This forms part of the old polynomial API. Since version 1.4, the new polynomial API defined in `numpy.polynomial` is preferred. A summary of the differences can be found in the [transition guide](#).

[NumPy Documentation](#)

The parameters of the linear function fitted to the measured values and the coefficient of determination are:

```
print(poly.polyfit(hooke['distance'], hooke['force'], 1))

print(f"r = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1] ** 2:.2f}")
```

```
[ 0.05753159 33.01899551]
r = 0.99
R2 = 0.99
```

With the regression coefficients, the predicted values of the linear function can be calculated.

```
# Calculation with polyeval

lm = poly.polyfit(hooke['distance'], hooke['force'], 1)
predicted_values_hooke = poly.polyval(hooke['distance'], lm)
```

The measurement series and the linear function fitted to it can be displayed graphically.

```
# Placeholder x & y

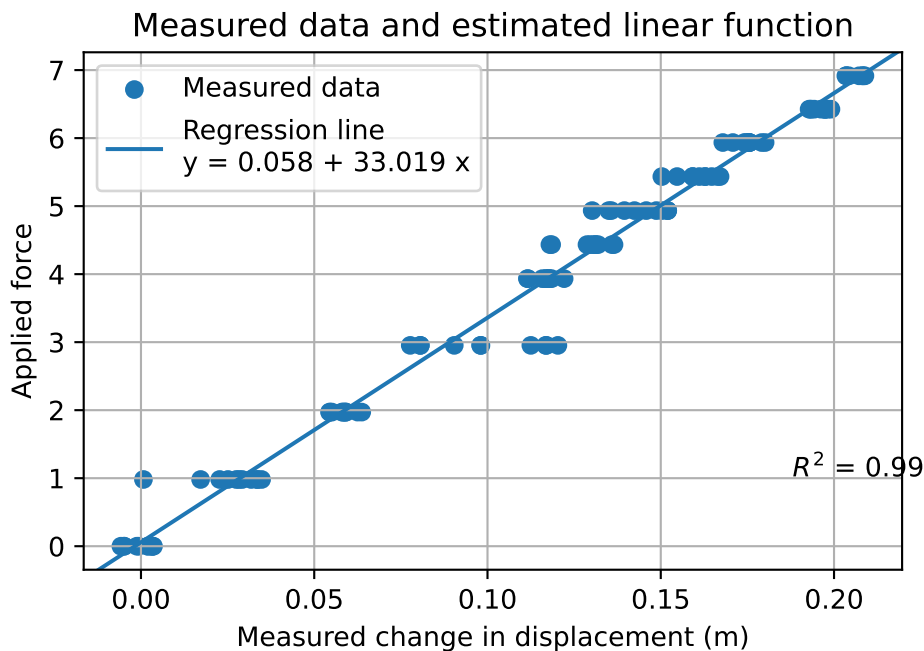
x = hooke['distance']
y = hooke['force']

# Create plot
```

```
plt.scatter(x, y, label='Measured data')
plt.axline(xy1=(0, lm[0]), slope=lm[1], label='Regression line\ny = ' + "{beta_0:.3f}".format(beta_0=lm[0]) + "{beta_1:.3f}x".format(beta_1=lm[1]))
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title('Measured data and estimated linear function')
plt.xlabel('Measured change in displacement (m)')
plt.ylabel('Applied force')
plt.legend()

plt.grid()
plt.show()
```



3.6.1 Quantifying Measurement Deviation

For the estimated regression coefficient, the standard error of the regression coefficient $SE = \hat{\sigma}_{\hat{\beta}_1}$ can be calculated for simple linear regression with one explanatory variable (see [Wikipedia](#)).

$$SE = \sqrt{\frac{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

- The numerator contains the mean **residual sum of squares** (sum of squared residuals / degrees of freedom).
- The denominator contains the **sum of squares of deviations of x** .

For a significance level α , a confidence level $1 - \alpha$ can be given as:

$$\hat{\beta}_1 \pm SE \cdot t_{1-\alpha/2} (n - 2)$$

- $t_{1-\alpha/2} (n - 2)$ is the value from the t-distribution with $n - 2$ degrees of freedom, or equivalently the return value of the function:
 - `scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)` for the upper bound of the interval.

```
print(f"Regression coefficient: {lm[1]:.4f}")

# Calculate the standard error of the regression coefficient 'manually'
standard_error_beta_1 = np.sqrt( (1 / (len(x) - 2) * sum((y - predicted_values_hooke) ** 2))

print(f"Standard error of the regression coefficient: {standard_error_beta_1:.4f}")

# Choose significance level (alpha level) 1 - 95%
alpha = 0.05
n = len(x)

t_value = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)
print(f"t-value 95% interval (two-sided): {t_value:.4f}")
print(f"95% confidence interval: {lm[1]:.4f} ± {t_value:.4f} * {standard_error_beta_1:.4f}")
print(f"lower 95% interval bound: {lm[1] - t_value * standard_error_beta_1:.4f}")
print(f"upper 95% interval bound: {lm[1] + t_value * standard_error_beta_1:.4f}")
```

```
Regression coefficient: 33.0190
Standard error of the regression coefficient: 0.3784
t-value 95% interval (two-sided): 1.9816
95% confidence interval: 33.0190 ± 1.9816 * 0.3784
lower 95% interval bound: 32.2692
upper 95% interval bound: 33.7688
```

The confidence interval can also be represented graphically.

```

# Placeholder x & y

x = hooke['distance']
y = hooke['force']

# Create plot

plt.scatter(x, y, label='Measured data')
plt.axline(xy1=(0, lm[0]), slope=lm[1], label='Regression line\ny = ' + "{beta_0:.3f}".format(beta_0=lm[0], beta_1=lm[1]) + " + {beta_1:.3f}x".format(beta_1=lm[1]))
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

# Draw 95% confidence interval

## poly.polyval(hooke['distance'], [lm[0]])

beta1_lower_boundary = lm[1] - (t_value * standard_error_beta_1)
beta1_upper_boundary = lm[1] + (t_value * standard_error_beta_1)

y_lower_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_lower_boundary])
y_upper_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_upper_boundary])

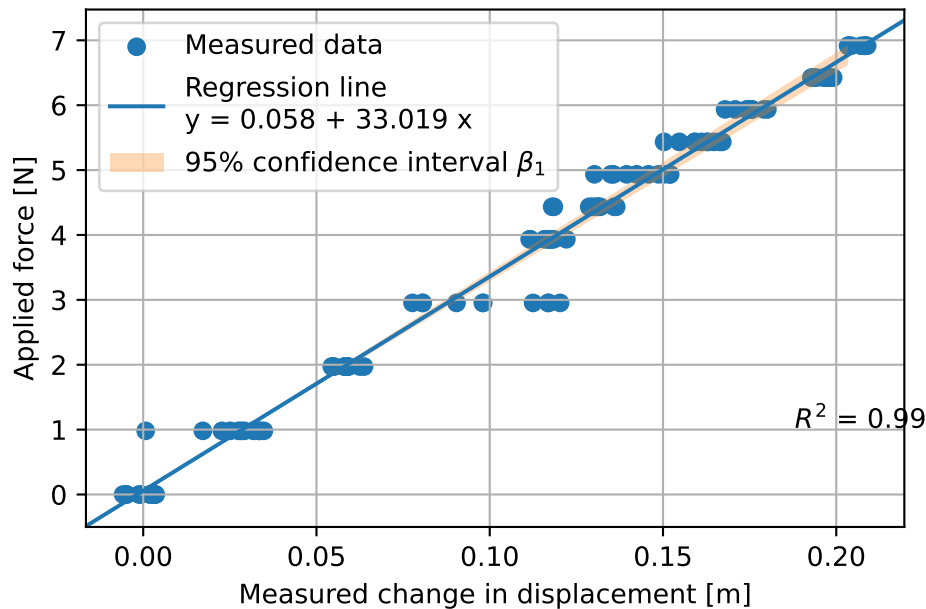
plt.fill_between(x=x, y1=y_lower_boundary, y2=y_upper_boundary, alpha=0.3, label='95% confidence interval')

plt.title(label='Measured data and estimated linear function within 95% interval')
plt.xlabel('Measured change in displacement [m]')
plt.ylabel('Applied force [N]')
plt.legend()

plt.grid()
plt.show()

```

Measured data and estimated linear function within 95% interval



3.6.2 The SciPy Module

The function `scipy.stats.linregress(x, y)` returns multiple values in a single function call:

- Slope of the regression line,
- y-intercept of the regression line,
- Bravais-Pearson correlation coefficient r ,
- p-value for the null hypothesis that the slope of the regression line is zero,
- Standard error of the slope, and
- Standard error of the y-intercept.

The standard error of the y-intercept is only available if the return values are stored in an object. The values can then be accessed as attributes.

```
# Assigning multiple objects
slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standard error of slope: {slope_stderr:.4f}", sep='')

# Assigning a single object
```

```
lm = scipy.stats.linregress(x, y)

print("\n", lm, sep='')
print(f"y-intercept: {lm.intercept:.4f}\nStandard error of y-intercept: {lm.intercept_stderr:.4f}")
```

```
y = 0.0575 + 33.0190 * x
r = 0.9928 R2 = 0.9856 p = 0.0000
Standard error of slope: 0.3784
```

```
LinregressResult(slope=np.float64(33.018995509180165), intercept=np.float64(0.05753158907970104), stderr=np.float64(0.3783732001932878), intercept_stderr=np.float64(0.05067079726769367))
y-intercept: 0.0575
Standard error of y-intercept: 0.0507
```

This allows the confidence interval to be calculated using the corresponding t-value.

```
alpha = 0.05
n = len(x)

print(f"slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f} {slope + scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}")
```

```
32.269 33.019 33.769
```

3.6.3 Spring Constant Result

The spring constant of the experimental setup lies, with 95 percent confidence, within the interval between 32.27 and 33.77. The point estimate for the spring constant is 33.02.

The task could be to calculate the 99-percent confidence interval.

→ In that case, only one number needs to be changed.

4 Error Calculation

“Experimentation is a thorny path, and one is never completely certain.”

Wagner, Paul. 2015. “Introduction to Physics I. PH I - 04 - Measurement Accuracy / Error Calculation Part 1.” University of Vienna Physics, [YouTube](#), timestamp 04:05

No measuring instrument is exact, and not all environmental influences can be controlled. The goal of error calculation is to quantify the resulting deviation of the measured values from the true value of a quantity.

4.1 Before we begin

However, the topic comes with some challenges that make initial engagement and independent study difficult.

4.1.1 The Concept of Error

Although the word “error” appears in the title of this chapter, it has not been a standardized term since 1995. DIN 1319 standardizes fundamental concepts and procedures in metrology. The standard replaces the term “error” with **measurement deviation** and **measurement uncertainty**, while also encompassing a variety of related definitions, notes, and remarks.

This chapter uses the terms commonly employed in error analysis to connect them to the standardized terminology. Generally, the standardized terms are used. To simplify the topic, this chapter is limited to measurements with:

- uncorrelated input quantities and
- negligible uncertainty in the systematic measurement deviation.

4.1.2 Confidence Level

Repeated measurements contain a random component of statistical uncertainty. Statistical uncertainty can only be expressed with a probability of confidence. The calculation of confidence intervals was covered in Chapter 2 and Chapter 3.

The first step in statistical analysis is to define the confidence level. From this, the multiples of the standard deviation s or the correction factor t to be used are derived. Different disciplines follow different conventions.

“In physics and surveying, a statistical confidence of 68.3% is sufficient, hence $t = 1$ [...] (In industry $t = 2$, in biology $t = 3$ is preferred.)” (Hochschule Aalen 2016, p. 4)

In engineering literature, this step is not always explicitly stated (since $1s$ or $1t$ is commonly used). DIN 1319 states: “It is customary to specify the confidence level $(1 - \alpha)$ as a percentage, usually $\alpha = 0.05$ (confidence level: 95%).” (Deutsches Institut für Normung 1995, p. 17)

4.1.3 Notation

The notation is complex: alongside the Latin and Greek alphabets, the Cyrillic alphabet is also used. Nevertheless, symbol duplication for formula symbols and units cannot always be avoided (the spring constant is particularly unfortunate in this respect).

Notation is also inconsistently applied. While DIN 1319 uses e for measurement deviations, literature often uses ϵ (epsilon) and Δ (Delta), with the latter also used for maximum error.

By the way: the e in DIN stands for “error” (ϵ).

4.2 Types of Errors

Measurements can exhibit different types of deviations from the true value of the quantity. The most common classification is into gross, systematic, and random *errors*. According to DIN 1319, it is better to speak of systematic and random measurement deviations. Gross errors are not covered by the standard, and the term “error” is appropriate here.

- Systematic measurement deviations: constant deviations under identical conditions, i.e., with the same magnitude and sign, caused by instrument inaccuracies, for example:
 - dependent on temperature
 - faulty calibration
 - aging of components
 - type of measurement, e.g., reading and scale errors, measurement interval in digital instruments ([quantization error](#))

- Random measurement deviations: statistical spread of values around the expectation, for example:
 - sampling error
 - reading errors
- Gross errors: distort the measurement result, for example due to:
 - incorrect experimental setup
 - unsuitable instruments
 - incorrect reading, e.g., ignoring the [settling time](#) or inattentiveness, such as weighing the container

(Hempel [2016](#), pp. 1–2; Eden and Gebhard [2024](#), pp. 17–18)

Some actions, like reading a value, can be associated with multiple error types:

- Inattentive reading and careless mistakes are gross errors,
- angle- and position-dependent errors like [parallax error](#) are systematic deviations,
- errors due to limited visual resolution lead to random deviations.

Tip 4: Handling Different Types of Measurement Deviations

1. Gross errors: discard affected values and repeat the measurement.
2. Systematic deviations: corrected if known.
3. Random deviations: quantify the statistical spread around the expectation.

Some systematic and random deviations can only be reduced with significant effort (more precise instruments, larger number of measurements). The effort must be weighed against the gain in accuracy: Is the measurement result acceptable with the stated uncertainty? (Hempel ([2016](#)), p. 2)

4.3 Fundamental Concepts According to DIN 1319

DIN 1319 standardizes fundamental concepts and procedures in metrology.

- Part 1 DIN 1319-1 and Part 2 DIN 1319-2 define basic metrology terms and measurement instruments.
- Part 3 DIN 1319-3 deals with evaluating measurements of a *single* quantity and its measurement uncertainty. It also applies when the quantity is calculated from other quantities via a given function (e.g., velocity v as distance s divided by time t).

- Part 4 DIN 1319-4 addresses reporting measurement results and their uncertainties when quantities are calculated from other quantities, e.g., via a least squares fit.

DIN 1319-4 deals with calculating a result Y through a function, e.g., a linear regression. The case in the previous chapter, determining the spring constant via regression coefficient β_1 , is not addressed in the standard and thus not presented here.

The learning goal of this chapter is to explain the following figure from DIN 1319.

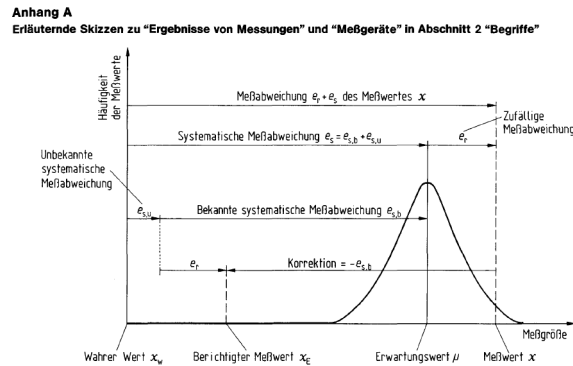


Figure 4.1: Measurement result according to DIN 1319

(Deutsches Institut für Normung [1995](#), p. 30)

The three most important fundamental concepts are:

1. Measurement deviation,
2. Measurement uncertainty, and
3. Complete measurement result.

4.3.1 Measurement Deviation

In error analysis, the term “error” generally refers to measurement deviation per DIN 1319, i.e., the *actual deviation* of a single measurement or the mean of a series of measurements (uncorrected measurement result) from the true value.

! Important 5: Measurement Deviation According to DIN 1319

Measurement deviation is: “the deviation of a value obtained from measurements and assigned to a quantity [...] from the true value [...]. If m is the value assigned to the quantity and x_w the true value, then the measurement deviation of the assigned value is

$m - x_w$ [...]” (Deutsches Institut für Normung 1995, p. 12)

The deviation of the **uncorrected measurement result** consists additively of the random deviation e_r and systematic deviation e_s .

- Systematic deviation is the difference between the expected value μ and the true value x_w : $e_s = \mu - x_w$
- The systematic deviation e_s consists of a known part $e_{s,b}$ and an unknown part $e_{s,u}$.
- The known systematic deviation $e_{s,b}$, when applied with opposite sign $-e_{s,b}$, is called a *correction*.

The **(corrected) measurement result** $\bar{x}_E$ is the arithmetic mean of a series of measurements $\bar{x}$ corrected for the known systematic measurement deviation $e_{s,b}$, and it is used as an estimator of the true value x_w . $\bar{x}_E = \bar{x} - e_{s,b}$

(Deutsches Institut für Normung 1995, pp. 11–13)

4.3.2 Measurement Uncertainty

The true value is generally unknown, so the actual deviation is also unknown. Only an *estimate from the measurements*, the measurement uncertainty, can be stated. In a broader sense, “error” refers to measurement uncertainty per DIN 1319.

! Important 6: Measurement Uncertainty According to DIN 1319

“A quantity obtained from measurements [...] used together with the measurement result [...] to characterize a **range of values** for the true value of the quantity [...]” (Deutsches Institut für Normung 1995, 14, emphasis added)

While measurement deviation is a single value, uncertainty specifies a range: “Measurement uncertainty must be distinguished from measurement deviation. The latter is only the difference between a value assigned to the quantity [...] and the true value. Measurement deviation can be zero without being known. This ignorance is expressed in a measurement uncertainty greater than zero.” (Deutsches Institut für Normung 1996, p. 3)

Measurement uncertainty of a quantity Y consists of the random deviation e_r and the uncertainty of the systematic deviation $u(e_s)$.

Random Deviation

If a measured quantity $Y = y(x)$ is determined through multiple direct measurements, the measured values x_i randomly scatter around their expected value μ with a standard deviation σ . The expected value μ and the standard deviation σ are usually unknown and are estimated from the measured values using the arithmetic mean $\bar{x}$ and the empirical standard deviation s_{n-1} . (see Chapter 1)

The uncertainty of the random scatter is represented by the empirical standard deviation of the mean:

$$u(\bar{x}) = \frac{s_{n-1}}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

(Deutsches Institut für Normung [1996](#), Equation (4), p. 5, notation adjusted)

systematic measurement error

The systematic measurement error e_s consists of a known component $e_{s,b}$ and an unknown component $e_{s,u}$. Since the unknown component cannot be determined, the total systematic measurement error is estimated using the known systematic error $e_{s,b}$. This leads to an uncertainty about the actual total systematic measurement error $u(e_s)$. This uncertainty exists even if the known systematic measurement error $e_{s,b}$ is zero, because the unknown component $e_{s,u}$ may be non-zero.

DIN 1319 distinguishes two cases, depending on whether the uncertainty $u(e_s)$ is considered negligible or not. In this chapter, we assume the uncertainty $u(e_s)$ is negligible (for the second case, see the example).

i Note 6: $u(e_s)$ Not Negligible

If the systematic deviation uncertainty is non-negligible, DIN 1319-3 refers to calculation rules in section 6.2.5, not discussed here. (Deutsches Institut für Normung ([1996](#)), p. 5-6) The uncertainty of the corrected result is the quadratic combination of random and systematic uncertainties:

$$u(y) = \sqrt{u^2(\bar{x}) + u^2(e_s)}$$

(Deutsches Institut für Normung [1996](#), Eq. (8), p. 6, notation adapted)

Note 4.1: Notation in DIN 1319

Random deviation is denoted as $u^2(x_1)$ instead of $u^2(\bar{x})$, and systematic uncertainty as $u^2(x_2)$ instead of $u^2(e_s)$.

4.3.3 Complete Measurement Result

The **corrected measurement result** $\bar{x}_E = y$ (in DIN 1319, the notation changes from $\bar{x}_E$ to y) is calculated by correcting the known systematic measurement deviation:

$$y = \bar{x} - e_{s,b}$$

(Deutsches Institut für Normung 1996, Equation (7), p. 6, notation adjusted)

The **complete measurement result** Y consists of the corrected measurement result and an indication of the measurement uncertainty, e.g., in the form:

$$Y = y \pm u(y)$$

(Deutsches Institut für Normung 1996, Equation (9), p. 6)

Additionally, a confidence interval (also called confidence range or confidence level) can be specified along with the complete measurement result. We have already introduced the calculation of confidence intervals in Chapter Chapter 2.

$$y - t \cdot u(y) \leq Y \leq y + t \cdot u(y)$$

(Deutsches Institut für Normung 1996, Equation (14), p. 7)

The choice of a confidence interval is discretionary. DIN 1319 states: “It is customary to specify the confidence level $(1 - \alpha)$ as a percentage, with $\alpha = 0.05$ (confidence level: 95 %) most commonly used.” (Deutsches Institut für Normung 1995, p. 17) “This 95 % confidence level should be used, but not a higher one.” (Deutsches Institut für Normung 1996, p. 7)

For example, this yields:

$$y - t_{\alpha/2} \cdot \frac{s_{n-1}}{\sqrt{n}} \leq Y \leq y + t_{\alpha/2} \cdot \frac{s_{n-1}}{\sqrt{n}}$$

Warning 2: Notation in DIN 1319

The arithmetic mean of a series is denoted $\bar{v}$ instead of $\bar{x}$.

4.3.4 Overview

DIN 1319 defines a number of additional terms, which are compiled in the following box together with the most important basic terms.

Important 7: Basic Terms According to DIN 1319

Measurement Deviation: “Deviation of a value obtained from measurements and assigned to the measurand [...] from the true value [...]. If m is the value assigned to the measurand and x_w is the true value, then the measurement deviation of the assigned value is $m - x_w$ [...]” (Deutsches Institut für Normung 1995, p. 12)

- The measurement deviation of the **uncorrected measurement result** consists additively of the random measurement deviation e_r and the systematic measurement deviation e_s .
- The systematic measurement deviation e_s consists of a known part $e_{s,b}$ and an unknown part $e_{s,u}$.
- The systematic measurement deviation is the difference between the expected value μ and the true value x_w . $e_s = \mu - x_w$
- The known systematic measurement deviation $e_{s,b}$, when applied with the opposite sign $-e_{s,b}$, is called a *correction*.

The **measurement result** $\bar{x}_E$ is the arithmetic mean of a series of measurements $\bar{x}$ corrected for the known systematic measurement deviation $e_{s,b}$, used as an estimator for the expected value μ .

$$\bar{x}_E = \bar{x} - e_{s,b}$$

(Deutsches Institut für Normung 1995, pp. 11–13)

Measurement Uncertainty u or $u(y)$: “Quantity derived from measurements [...] and used together with the measurement result [...] to characterize a range of values for the true value of the measurand [...]” (Deutsches Institut für Normung 1995, p. 14)

- **Standard measurement uncertainty**, or simply standard uncertainty, is used when the measurement uncertainty is expressed by a standard deviation. (Deutsches Institut für Normung 1996, p. 3)
- **Expanded measurement uncertainty** $U(y) = k \cdot u(y)$: Range that likely contains the true value of the measurand. The expansion factor k should be set between 2 and 3, with $k = 2$ preferred. The expansion factor k should be reported so that the standard measurement uncertainty $u(y) = U(y)/k$ can be determined. (Deutsches Institut für Normung 1996, p. 7)

Note: The expanded measurement uncertainty differs from a confidence interval in that no normal distribution is assumed – it is simply a factor ‘to be on the safe side’ that the true value is within the given range.

The **complete measurement result** is the measurement result with quantitative information about the measurement uncertainty u . $x = \bar{x}_E \pm u$. (Deutsches Institut für Normung 1995, p. 16)

- The **confidence range** $\bar{x} - t \cdot u(y) \leq Y \leq \bar{x} + t \cdot u(y)$ can additionally be provided along with the complete measurement result.
 - If the systematic measurement deviation is known and has been corrected, the true value of the measurand Y lies within the confidence range.
 - If the systematic measurement deviation is unknown, the expected value μ of the distribution lies within the confidence range. (Deutsches Institut für Normung 1996, p. 7)

Note: In the DIN standard, the arithmetic mean of a measurement series is written as $\bar{v}$ instead of $\bar{x}$.

4.4 Absolute and Relative Errors

In error analysis, a distinction is made between absolute and relative *errors*. It is more precise to speak of absolute and relative measurement uncertainties.

- The **absolute measurement uncertainty** u is given in the unit of the measured quantity: $u(m) = 0.2 \text{ kg}$. The complete measurement result is written as: $m = 5.0 \pm 0.2 \text{ kg}$ or $m = 5.0 \text{ kg} \pm 0.2 \text{ kg}$.
- The **relative measurement uncertainty** $u_{rel}(m) = \frac{u(m)}{|m|}$ is either given in the product form as $m \cdot (1 \pm \frac{u(m)}{|m|})$, for example:

$$m = 5.0 \text{ kg} \cdot \left(1 \pm \frac{0.2 \text{ kg}}{5.0 \text{ kg}}\right) = 5.0 \text{ kg} \cdot (1 \pm 0.04)$$

Alternatively, the relative measurement uncertainty can be expressed as a percentage: $u_{rel}(m) = \frac{u(m)}{|m|} \cdot 100 \%$, for example:

$$m = 5.0 \text{ kg} \pm 4 \%$$

(Deutsches Institut für Normung [1995](#), pp. 9–10, 16; Deutsches Institut für Normung [1996](#), p. 10)

Warning 3: (Alternative) Notations

In DIN 1319, measurement deviations are denoted by e , while measurement uncertainty is denoted by u . In error analysis literature, the term (measurement) *error* is more commonly used. The notation, however, is not consistent.

- The absolute error is often denoted by ϵx (epsilon x). Another common notation is Δx (Delta), for example (Hochschule Aalen [2016](#); Dinter, Ralf [2011](#); Schrüfer, Reindl, and Zagar [2022](#)). Δ is also used to denote the maximum error.
- The relative error is also denoted by δx (delta x), for example: $\delta x = 4 \%$.

4.5 Maximum Error

Systematic measurement deviations $e(x)_s$ cause a one-sided bias in the measurement result, so that the measured value is always either too high or too low. Random measurement deviations $e(x)_r$ lead uncontrolledly to either overestimation or underestimation of the measured value.

In general, the different measurement deviations partially cancel each other out. The **maximum error** refers to the worst-case scenario, where all measurement deviations reinforce each other at their respective maximum (or minimum) values, so that the measured value shows the largest possible deviation from the (unknown) true value. The **maximum error** is denoted as Δx (Delta x).

The maximum error of a series of measurements for a quantity is determined by summing the absolute values of the individual measurement deviations.

$$\Delta x = |e(x)_s| + |e(x)_r|$$

Since measurements with gross errors are considered invalid, the contribution of gross errors $|e(x)_{gross}|$ is omitted.

(Eden and Gebhard (2024), pp. 35-36)

When multiple quantities are considered, the maximum errors of the involved quantities add up to the maximum error of the resulting quantity. For a quantity $Y = f(x_1, x_2)$:

$$\Delta y = \Delta x_1 + \Delta x_2$$

Tip 5: Maximum Error

The maximum error is not a standardized specification according to DIN 1319, but a simplified form of error estimation and overestimates the likely measurement uncertainty. “For many considerations, especially in introductory laboratory courses of the first semesters in technical, scientific, and engineering programs, it is sufficient to determine the maximum error (*absolute maximum error*).” (Eden and Gebhard (2024), p. 35)

4.6 Significant Figures

When reporting a complete measurement result, the number of significant or reliably known digits must be taken into account.

! Important 8: Significant Figures

“Every reliably known digit, except for zeros that indicate the position of the decimal point, is called a significant figure.” (Eden and Gebhard 2024, p. 24)

“Rounded numerical uncertainty values should be given **with two (or if necessary three) significant digits. They must be rounded up.** The measurement result should be rounded to the same place as the associated uncertainty, e.g., $y = 245.5716$ mm is rounded to $y = 245.57$ mm if $u(y) = 0.4528$ mm is rounded up to $u(y) = 0.46$ mm.” (Deutsches Institut für Normung 1996, 6, emphasis added)

For example, the distance measurements used in the previous chapter are given in the format 153.29, which means the measurements have 5 significant figures. Therefore, the arithmetic mean of a series of measurements, 153.008889, should also be reported with 5 significant figures. It is rounded at the first non-significant digit, so the arithmetic mean is reported as 153.01.

However, the number of significant figures is not always straightforward to determine. The rules are:

1. All non-zero digits are always significant.
2. Leading zeros are never significant. For example, 0012 km has 2 significant figures. 0.001 mm has 1 significant figure.
3. Zeros at the end of a number are significant if they are to the right of the decimal point, e.g., 12.100 km has 5 significant figures.
4. Zeros at the end of a number without a decimal point may or may not be significant. 1200 m can have two or four significant figures.

💡 Significant Figures and Scientific Rounding

Although uncertainty values are rounded up, it is useful to know: do you know the difference between commercial rounding and scientific rounding?

<https://www.youtube.com/watch?v=s0gPyypigOs>

“Rounding and Significant Figures (Mathematics Prep Course)” by Edmund Weitz is available on [YouTube](#), 2019.

Python rounds scientifically:

```
print(round(2.5), round(3.5))
```

2 4

4.7 Methods of Error Analysis

The goal of error analysis is to determine uncertainty intervals for a measured quantity. Several methods are available:

1. Error estimation,
2. Statistical error analysis, and
3. Error propagation.

There is some discretion in choosing the method. (Hempel (2016), p. 2) For routine measurements, it is sufficient to report uncertainty in percentage terms. In calibration or when comparing with a [standard](#), more precise information is required. (Schrüfer, Reindl, and Zagar 2022, p. 33)

The choice of the appropriate method also depends on the available information. For example, the spring constant determined in the previous chapter depends on three quantities:

1. the mass m , measured with a kitchen scale, whose reading accuracy is estimated as $e_r(m) = 0.5g = 0.0005kg$.
2. the distance to the weight, or after data processing, the spring extension Δx , measured with an electronic distance sensor and an Arduino (Ultrasonic Ranging Module HC - SR04).
 - The reading accuracy of the measurement system is one unit of the measurement resolution, i.e., $e_r = 0.01cm = 0.0001m$.
 - The measurement deviation of the system is up to 3 mm according to the datasheet, symmetrically, i.e., $e_r = 3mm = 0.003m$.
 - The random component of the measurement deviation is therefore $e_r(x) = 0.0001m + 0.003m = 0.0031m$.
3. the statistical estimate of the spring constant $k = \frac{m \cdot g}{\Delta x} = 33.019 \pm 0.378$, or within the specified confidence interval $k_{95\%} = 32.269 \leq 33.019 \leq 33.769$.

Warning 4: Symbols and Units

The same symbols are used for second and displacement s , mass and meter m , gram, the constant g , and Δ for the change of a quantity and for the maximum error. Therefore, the formulas should be read carefully.

4.7.1 1. Error Estimation

Error estimation is used when

- **one quantity**
- **is measured only once.**

Since the true value of a measured quantity is unknown, error estimation aims to determine the maximum possible deviation of the measured value from the true value, i.e., the **maximum error**. For a single measurement, there is no statistical component. Therefore, the error estimation for a single measurement includes:

- the guaranteed error limit of the instrument, provided by the manufacturer on the device or in the manual (see example), i.e., the known systematic measurement deviation $e_{s,b}$, and
- an estimated component depending on the specific experimental conditions, e.g., the scale division or reading conditions. This component can consist of a systematic part (e.g., an analog scale read from an angle) and a random part (e.g., reading accuracy of the instrument).

The maximum error Δx is obtained by adding the absolute values of the known systematic deviation and the estimated random deviation: $\Delta x = |e(x)_{s,b}| + |e(x)_r|$.

(Hempel ([2016](#)), p. 3)

Exercise: Temperature Measurement

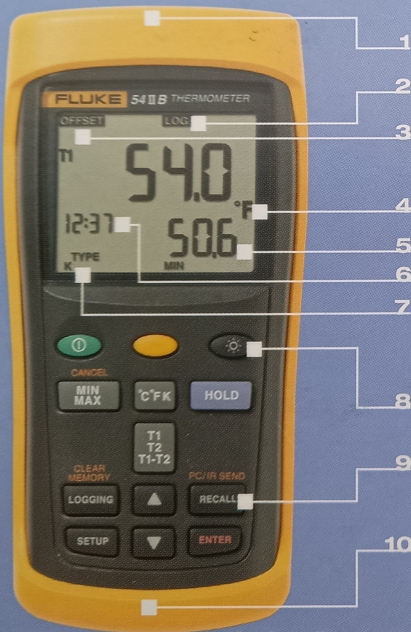
A temperature $T = 112.1\text{ }^{\circ}\text{C}$ was measured using a digital thermometer with a K-type thermocouple. Which systematic and random measurement deviations are known and can be reported (see example)?

i Note 7: Thermometer Guaranteed Error Limit

FLUKE®

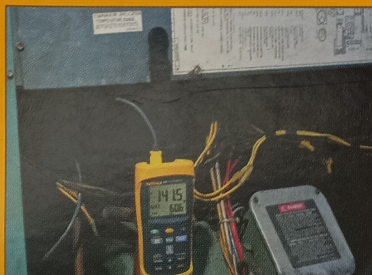
Thermocouple Thermometer

53/54 II B



Key features

- 1 Accepts magnetic hanger (optional TPAK accessory) for hands-free operation
- 2 Datalog up to 500 points to internal memory
- 3 Electronic offset compensates for thermocouple error
- 4 Selectable °C, °F, or Kelvin readout
- 5 Large dual display shows any combination of T1, T2 (54 II only), T1-T2 (54 II only), plus MIN, MAX, or AVG
- 6 Real time clock captures exact time of day when major events occur
- 7 Thermometer accepts seven different thermocouple types
- 8 Bright backlight turns on for two minutes with a touch of a button
- 9 Recall logged data on-screen or download via IR port to optional PC software
- 10 Rugged design easily withstands a 1 m (3 ft) drop



Specifications

Accuracy	
Above -100 °C	[0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types
Below -100 °C	[0.05 % + 0.4 °C (0.7 °F)] for R and S types
	[0.2 % + 0.3 °C] for J, K, E, and N types
	[0.5 % + 0.3 °C] for T type
Resolution	0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000
Battery Life	1000 hours typical
Warranty	Three-years

Figure 4.2: Thermometer

Specifications	
Accuracy	
Above -100 °C	[0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types
Below -100 °C	[0.05 % + 0.4 °C (0.7 °F)] for R and S types [0.2 % + 0.3 °C] for J, K, E, and N types [0.5 % + 0.3 °C] for T type
Resolution	0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000
Battery Life	1000 hours typical
Warranty	Three-years

Figure 4.3: Specifications

💡 Tip 6: Half or Full Scale Division?

Digital instruments make this easy: the reading accuracy is one scale unit. For analog instruments, it is up to your judgment which measurement accuracy to report.

If you are certain you can distinguish “exactly on the line” vs. “between two lines,” the reading error is half a scale division. If unsure, the reading error is one full scale division. ([Error Analysis Made Easy, FSU Jena](#))

💡 Tip 7: Sample Solution: Temperature Measurement

Systematic measurement deviation

The absolute measurement deviation is: $u(T) = 0.3\text{ °C}$ The relative deviation is: 0.05 %.

That means:

$$\frac{u(T)}{112.1\text{ °C}} \cdot 100\% = 0.05\%$$

$$\frac{u(T)}{112.1\text{ °C}} \cdot 100 = 0.05$$

$$u(T) = 0.05 \cdot \frac{112.1\text{ °C}}{100}$$

$$u(T) = 0.05605\text{ °C}$$

Therefore, the known systematic measurement deviation is: $e(T)_{s,b} = 0.3\text{ °C} + 0.05605\text{ °C} = 0.35605\text{ °C}$

Random measurement deviation

The random measurement deviation equals one scale unit: $e(T)_r = 0.1\text{ °C}$

Maximum Error

The maximum error is:

$$\Delta T = |e(T)_{s,b}| + |e(T)_r|$$

Rounding the measurement uncertainty to two decimal places:

$$\Delta T = |0.35605^{\circ}\text{C}| + |0.1^{\circ}\text{C}| = 0.45605^{\circ}\text{C} \approx 0.46^{\circ}\text{C}$$

The complete measurement result is therefore: $T = 112.1 \pm 0.46^{\circ}\text{C}$.

Example Measurement Conditions

What is meant by the concrete conditions of a measurement process? This is demonstrated using two simple measurement procedures.

You can participate here!

You will need:

- a suitable measuring instrument, e.g. a ruler, and
- a coin (ideally a 1-euro coin).

In the first measurement procedure, the diameter of the 1-euro coin is to be determined.

i Note 8: First Measurement Procedure: Coin

The measurement is carried out using a [vernier caliper](#) with a resolution of 0.05 mm according to DIN 862. DIN 862 standardizes symmetrical error limits for vernier calipers. The error limit depends on the scale resolution of the caliper as well as on the measured length and can be read from a table. (DIN 862 was replaced in 2011 by DIN EN ISO 13385-1, in which the term deviation is used. However, DIN 862 is still printed on the caliper.)

Measured value: $x = 23.25 \text{ mm}$

- The scale division is 0.05 mm, so the random reading error e_r is $\pm 0.025 \text{ mm}$.
- DIN 862 standardizes error limits that increase with the measured length, but are at least $50 \mu\text{m}$ or 0.05 mm. For the measured value of 23.25 mm, the known systematic measurement deviation is therefore $e_{s,b} = \pm 0.05 \text{ mm}$.

From the sum of the absolute values, the maximum error is obtained:

$$\Delta x = |0.05| \text{ mm} + |0.025| \text{ mm} = 0.075 \text{ mm}.$$

The complete measurement result, consisting of the measured value and the measurement uncertainty, is therefore:

$$23.25 \pm 0.075 \text{ mm}.$$

The relative maximum error is: $23.25 \text{ mm} \pm 0.32\%$.

In the second measurement procedure, the length of the thumb is to be determined. ([Anatomically, the thumb consists of only 2 phalange bones](#))

i Note 9: Second Measurement Process – Thumb

The measurement is carried out again using the vernier caliper.

Measured value: $x = 64.70$ mm

- The scale division is 0.05 mm, therefore the random reading uncertainty is ± 0.025 mm.
- According to DIN 862, the standardized error limit for measured lengths between 50 and 300 mm is 50 μm or 0.05 mm. For the measured value of 64.7 mm, the error limit is therefore ± 0.05 mm.
- In addition, a symmetric measurement deviation of 3 mm is assumed because
 - the measuring device was not applied straight,
 - the base of the thumb could not be located exactly, and
 - the elastic fingertip could not be fixed.

From the sum of the absolute values, the maximum error is obtained: $\Delta x = |0.025| \text{ mm} + |0.05| \text{ mm} + |3| \text{ mm} = 3.075 \text{ mm} \approx 3.08 \text{ mm}$.

The measurement result, consisting of the measured value and the error interval, is therefore: 64.70 ± 3.08 mm.

The relative maximum error is: $64.70 \text{ mm} \pm 4.76\%$.

4.7.2 2. Error Statistics

Error statistics are used when

- **a quantity**
- **is measured multiple times**

The error statistics are only considered reliable if the measurement deviations are random, meaning the values fluctuate in magnitude and sign. In this case, the measurement deviation is included in the calculated standard deviation. However, if a known systematic measurement error exists, the measurement result must be corrected accordingly: $\bar{x}_E = \bar{x} - e_{s,b}$.

In previous chapters, we anticipated error statistics with the statistical description of the empirical spread of the mean. To recap:

1. Set a significance level α or a two-sided confidence level $1 - \frac{\alpha}{2}$.
2. Determine the sample mean $\bar{x}$.
3. Determine the sample standard deviation s , the sample size N , and the empirical standard error $\frac{s_{n-1}}{\sqrt{N}}$.
4. Determine the values of the normal or t-distribution for the chosen confidence level — for a two-sided hypothesis test $\frac{\alpha}{2}$ and $1 - \frac{\alpha}{2}$.

5. Calculate the confidence interval $\bar{x} \pm t_{\alpha/2} \frac{s_{n-1}}{\sqrt{n}}$.

This confidence interval indicates, for a chosen probability, the range in which the mean of a repeated measurement series will lie, or, if the unknown systematic measurement error is zero (or negligible), the *true value*.

In contrast, the standard deviation s indicates the range in which a single measurement value is likely to fall. The certainty of this statement can be increased by widening the confidence interval (e.g., $2 \cdot s$).

(Hempel 2016, pp. 3–7)

The following example demonstrates the procedure. It also introduces the Pandas method `.sem()` for easy calculation of the standard error.

i Note 10: Statistical Error Analysis

From the previous chapter's experiment, select the measurement series for weight 503g.

```
filepath = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = filepath, sep = ';')
hooke.drop(index = 113, inplace = True) # missing value

hooke_503g = hooke.loc[hooke['mass'] == 503, :]
print(hooke_503g.head(), hooke_503g.describe(), sep = "\n\n")
```

	no	mass	distance
40	40	503	158.43
41	41	503	158.43
42	42	503	158.60
43	43	503	158.76
44	44	503	159.05

	no	mass	distance
count	10.00000	10.0	10.000000
mean	44.50000	503.0	159.314000
std	3.02765	0.0	0.781099
min	40.00000	503.0	158.430000
25%	42.25000	503.0	158.640000
50%	44.50000	503.0	159.220000
75%	46.75000	503.0	159.965000
max	49.00000	503.0	160.610000

Let us consider once again the arithmetic mean (the uncorrected measurement result) and the standard error of the mean (the statistical measure of dispersion of the mean) for

this series of measurements. The method `.sem()` computes the empirical standard error of the mean (by default, the parameter `ddof = 1` is used).

Output accurate to 3 decimal places.

```
print(f"Weight in grams: {hooke_503g['mass'].unique()}",  
      f"Mean of measurements: {hooke_503g['distance'].mean():.3f} cm",  
      f"Empirical standard error: {hooke_503g['distance'].sem():.3f} cm",  
      sep = "\n")
```

```
Weight in grams: [503]  
Mean of measurements: 159.314 cm  
Empirical standard error: 0.247 cm
```

The measurement uncertainty is rounded up to 2 decimal places. For this, the NumPy function `np.ceil()` can be used. However, it rounds to the next integer, and there is no parameter to round to a specific decimal place. Therefore, the decimal place must be shifted by multiplying with the corresponding power of ten. For 2 decimal places, multiply by 10^2 : `np.ceil(value * 100) / 100`.

```
value = pd.Series([0.247])  
print(np.ceil(value * 100) / 100)  
  
# using Pandas methods  
print(value.mul(100).apply(np.ceil).div(100))
```

```
0    0.25  
dtype: float64  
0    0.25  
dtype: float64
```

For chaining with further methods in Pandas, the return value of the function, a NumPy scalar or array, must be converted back into a `pd.Series`.

```
print(f"Weight in grams: {hooke_503g['mass'].unique()}",  
      f"Mean of measurements: {hooke_503g['distance'].mean():.3f} cm",  
      f"Empirical standard error: { ( sem_hooke_503g:= pd.Series(hooke_503g['distance'].sem()) ):.3f} cm",  
      f"Complete measurement result: {hooke_503g['distance'].mean().round(2):.2f} cm ± {sem_hooke_503g:.2f} cm",  
      sep = "\n")
```

```
Weight in grams: [503]  
Mean of measurements: 159.314 cm  
Empirical standard error: 0.250 cm  
Complete measurement result: 159.31 cm ± 0.25 cm
```

Additional specification of the 95% confidence interval.

```
# Determine t-value for significance level (choose alpha-level 1 - 95%)
alpha = 0.05
n = hooke_503g.shape[0] # Number of rows
t_value = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 1)

print(f"95% confidence interval : {hooke_503g['distance'].mean().round(2)} cm ± {t_value:
      f"95% confidence interval : {hooke_503g['distance'].mean().round(2)} cm ± {(t_value
      f"lower 95% interval limit: {hooke_503g['distance'].mean().round(2)} - round(t_value
      f"upper 95% interval limit: {hooke_503g['distance'].mean().round(2)} + round(t_value
      sep="\n")

95% confidence interval : 159.31 cm ± 2.2622 * 0.25 cm
95% confidence interval : 159.31 cm ± 0.57 cm
lower 95% interval limit: 158.74 cm
upper 95% interval limit: 159.88 cm
```

4.7.3 3. Error Propagation

Error propagation is used when

- multiple input quantities,
- that determine the output quantity through a function,

are measured.

To determine the measurement uncertainty of a quantity Y , which is estimated by a function $y = f(x_1, x_2, \dots, x_m)$, there are two methods:

- **linear error propagation** is used when the uncertainties of the involved quantities are determined from the **error estimation of a single measurement**.
- **Gaussian error propagation** is used when the uncertainties are **statistically determined from repeated measurements** and given with confidence intervals.

(Hempel 2016, pp. 8–9)

Linear Error Propagation

Linear error propagation considers how the **maximum errors** of several measured quantities affect the result in the worst case. In other words, the **maximum error** of the desired quantity Y is calculated. (cf. Deutsches Institut für Normung (1996), p. 9)

For a function with two input quantities $f = f(x_1, x_2)$, the maximum error Δf is determined from the maximum errors Δx_1 and Δx_2 .

$$f + \Delta f = f(x_1 + \Delta x_1, x_2 + \Delta x_2)$$

Assuming small errors relative to the measured values, the error Δf can be calculated using the absolute values of the partial derivatives of the first term of the Taylor series (see example).

$$f + \Delta f = f(x_1, x_2) + \left| \frac{\partial f(x_1, x_2)}{\partial x_1} \right| \cdot \Delta x_1 + \left| \frac{\partial f(x_1, x_2)}{\partial x_2} \right| \cdot \Delta x_2$$

When are the errors considered small relative to the measured values? Unfortunately, there is no absolute limit. It depends on whether, in the Taylor expansion, higher-order polynomials are significantly smaller than the linear term and can thus be neglected if x is small. The decision ultimately depends on the required accuracy.

i Note 11: Taylor Series

<https://www.youtube.com/watch?v=F2IKRaUguBM>

The Taylor series - simply explained by Denkbar is available on [YouTube](#), 2015.

i Note 12: Partial Derivative

A partial derivative is the derivative of a function with multiple variables with respect to one variable, treating all other variables as constants.

For a function $f(x, y) = 2x + y^2$, the partial derivative with respect to x is expressed as:

$$\frac{\partial f(x, y)}{\partial x}$$

- The symbol ∂ is the italic form of the Cyrillic lowercase letter (d) and is read as “del”. It indicates that a partial derivative is being taken.
- In the numerator is the function to be differentiated. In the denominator is the variable with respect to which the derivative is taken. The term is read as “del f of x and y with respect to del x”.

The partial derivative $\frac{\partial f(x, y)}{\partial x} = 2$. y^2 is treated as a constant (e.g., 5^2) and differentiates to zero.

The partial derivative $\frac{\partial f(x, y)}{\partial y} = 2y$. $2x$ is treated as a constant (e.g., $2 \cdot 3$) and differentiates to zero.

The maximum error is calculated as:

$$\Delta f = \left| \frac{\partial f(x_1, x_2)}{\partial x_1} \right| \cdot \Delta x_1 + \left| \frac{\partial f(x_1, x_2)}{\partial x_2} \right| \cdot \Delta x_2$$

(Hempel 2016, pp. 9–10)

Or in shorthand notation for $i = N$ input quantities:

$$\Delta f(x_i) = \pm \sum_{i=1}^N \left| \frac{\partial f}{\partial x_i} \right| \cdot \Delta x_i$$

(after Eden and Gebhard 2024, 35, notation adjusted)

i Note 13: Example: Spring Constant

As an example, we consider the first measurement from the series with a weight of 503 grams.

```
hooke_503g_single_value = hooke_503g.iloc[0, :].copy()
print(hooke_503g_single_value)
```

```
no          40.00
mass        503.00
distance    158.43
Name: 40, dtype: float64
```

Since multiple magnitudes are considered, the measurements are converted to SI units. The zero point is defined as the first measurement in the series with a weight of 0 grams.

```
hooke_503g_single_value['mass'] = hooke_503g_single_value['mass'] / 1000
hooke_503g_single_value['distance'] = hooke_503g_single_value['distance'] / 100

zero_point = hooke[hooke['mass'] == 0].loc[:, 'distance'].head(n=1) / 100
print(f"Zero point in meters: {zero_point.item():.3f}\n")

hooke_503g_single_value['extension'] = (hooke_503g_single_value['distance'].item() - zero_point.item())
print(hooke_503g_single_value)
```

```
Zero point in meters: 1.733
```

```
no          40.0000
mass         0.5030
distance     1.5843
extension    0.1489
Name: 40, dtype: float64
```

For the spring constant (the change in displacement Δx is denoted as s):

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

Inserting the values (s now stands for seconds, m for meters):

$$k = \frac{0,503 \text{ kg} \cdot 9,81 \text{ m}}{0,1489 \text{ m} \cdot \text{s}^2} = \frac{0,503 \cdot 9,81 \text{ N}}{0,1489 \text{ m}} = 33.139 \frac{\text{N}}{\text{m}}$$

The maximum error of the function is given by:

$$\Delta k(m, s) = \left| \frac{\partial f(m, s)}{\partial m} \right| \cdot \Delta m + \left| \frac{\partial f(m, s)}{\partial s} \right| \cdot \Delta s$$

In the first step, the partial derivatives of the function $k = \frac{m \cdot g}{s}$ are calculated. The partial derivative with respect to m using the constant factor rule is:

$$\frac{\partial f(m, s)}{\partial m} = \frac{\partial}{\partial m} \frac{m \cdot g}{s} = \frac{\partial}{\partial m} m \cdot \frac{g}{s} = 1 \cdot \frac{g}{s} = \frac{g}{s}$$

The partial derivative with respect to s is:

$$\frac{\partial f(m, s)}{\partial s} = \frac{\partial}{\partial s} \frac{m \cdot g}{s} = -\frac{m \cdot g}{s^2}$$

Thus, the maximum error is:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| -\frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

In the second step, the maximum errors of the quantities involved are determined.

- The mass m of the weights was measured with a kitchen scale, whose reading error is estimated at $e_r(m) = 0.5 \text{ g} = 0.0005 \text{ kg}$. No systematic measurement error is known. Therefore, the maximum error of the mass is $\Delta m = 0.0005 \text{ kg}$.
- The maximum error of the displacement measurement is $e_r(s) = 0.01 \text{ cm} + 0.3 \text{ cm} = 0.0031 \text{ m}$.

The measured values, constant g , and maximum errors are inserted:

$$\Delta k = \left| \frac{9.81 \text{ m/s}^2}{0.1489 \text{ m}} \right| \cdot 0.0005 \text{ kg} + \left| -\frac{0.503 \text{ kg} \cdot 9.81 \text{ m/s}^2}{(0.1489 \text{ m})^2} \right| \cdot 0.0031 \text{ m}$$

Separate values and units:

$$\Delta k = \left| \frac{9.81 \cdot 0.0005 \text{ kg m}}{0.1489 \text{ s}^2 \cdot \text{m}} \right| + \left| -\frac{0.503 \cdot 9.81 \cdot 0.0031 \text{ kg m}^2}{0.02217121 \text{ s}^2 \cdot \text{m}^2} \right|$$

Calculate and convert to Newtons ($\frac{kg \cdot m}{s^2}$):

$$\Delta k = |0.0329 \frac{N}{m}| + |-0.6899 \frac{N \cdot m}{m^2}|$$

Cancel meters in the second term:

$$\Delta k = |0.0329 \frac{N}{m}| + |-0.6899 \frac{N}{m}|$$

Round the measurement uncertainty to two decimal places:

$$\Delta k = 0.7228 \frac{N}{m} \approx 0.73 \frac{N}{m}$$

The complete measurement result is thus:

$$k = 33.14 \frac{N}{m}; \Delta k = 0.73 \frac{N}{m}$$

Gaussian Error Propagation

In Gaussian error propagation, it is assumed that measurement uncertainties of the involved quantities may partially compensate each other. This allows a more realistic estimate of the measurement uncertainty.

The determination of measurement uncertainty is regulated in DIN 1319-3. For a function $Y = f(x_1, x_2, \dots, x_m)$, the measurement uncertainty is determined from the uncertainties of the quantities involved. The variances of the mean values are used (Deutsches Institut für Normung (1996), Equation (29), p. 11):

$$u^2(x_i) = s^2(\bar{x}_i) = s_i^2/n_i; \quad s_i^2 = \frac{1}{n_i - 1} \sum_{j=1}^{n_i} (x_{ij} - \bar{x}_i)^2$$

Note: Typically, the empirical standard deviation s_{n-1} is used. In the DIN standard, the arithmetic mean of a series of measurements is denoted as $\bar{v}$ instead of $\bar{x}$.

The measurement uncertainty of the quantities is thus determined via the standard error of the mean of the quantities:

$$u(x_i) = s_i / \sqrt{n_i}$$

Note: Typically, the empirical standard deviation $s_{n-1} / \sqrt{N}$ is used.

(Deutsches Institut für Normung 1996, p. 11)

The uncertainty of the function $y = f(x_1, x_2, \dots, x_m)$ is calculated as follows (DIN 1319-3, Equation (43)):

$$u(y) = \sqrt{\sum_{i=1}^m \left(\frac{\partial f}{\partial x_i} \right)^2 \cdot u^2(x_i)}$$

The squared terms can also be combined:

$$u(y) = \sqrt{\sum_{i=1}^m \left(\frac{\partial f}{\partial x_i} \cdot u(x_i) \right)^2}$$

For a function with two input quantities $y = f(x_1, x_2)$, the uncertainty of the result is:

$$u(y) = \sqrt{\left(\frac{\partial f}{\partial x_1} \cdot \sigma_{x_1} \right)^2 + \left(\frac{\partial f}{\partial x_2} \cdot \sigma_{x_2} \right)^2}$$

Note: Typically, the empirical standard deviation $s_{n-1}/\sqrt{N}$ is used.

i Note 14: Example: Measurement Series 503 Grams

Let's revisit the measurement series for the 503-gram weight. The agreed confidence level of 95 % is assumed.

```

# Restore measurement data from hooke.qmd
## Read file and remove error
filepath = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer=filepath, sep=';')
hooke.drop(index=113, inplace=True)

## Normalize displacement to zero point
zero_point = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
hooke['extension'] = hooke['distance'].sub(zero_point).mul(-1)

# Select measurement series for 503 grams
hooke_503g = hooke.loc[hooke['mass'] == 503, :].copy()

## Convert mass to kg, distance and extension to meters
hooke_503g['mass'] = hooke_503g['mass'].div(1000)
hooke_503g['distance'] = hooke_503g['distance'].div(100)
hooke_503g['extension'] = hooke_503g['extension'].div(100)

# Determine t-value for significance level (alpha) 1 - 95%
alpha = 0.05
n_extension = hooke_503g.shape[0] # number of rows
t_value_extension = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_extension - 1)

# Output
print(f"Weight in kilograms: {hooke_503g['mass'].unique()}",
      f"Mean extension: {hooke_503g['extension'].mean():.4f} m",
      f"empirical standard error: {(sem_hooke_503g := pd.Series(hooke_503g['extension']).sem):.4f} m",
      f"t-value of extension: {t_value_extension:.4f}",
      f"95% confidence interval: {(hooke_503g['extension'].mean() - t_value_extension * sem_hooke_503g, hooke_503g['extension'].mean() + t_value_extension * sem_hooke_503g):.4f} m",
      sep="\n")

```

```

Weight in kilograms: [0.503]
Mean extension: 0.1432 m
empirical standard error: 0.002470 m
t-value of extension: 2.2622
95% confidence interval: 0.1376 m  0.1432 m  0.1488 m

```

For the spring constant (the change in displacement Δx is denoted by s), the formula is:

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

$$k = \frac{0.503 \text{ kg} \cdot 9.81 \text{ m/s}^2}{0.1432 \text{ m}} = \frac{0.503 \cdot 9.81 \text{ N}}{0.1432 \text{ m}} = 34.458 \frac{\text{N}}{\text{m}}$$

To determine the uncertainty of the resulting quantity k using Gaussian error propagation, the mass of the weight used would also need to be determined through a series of measurements. (How measurement series and single measurements can be combined will be shown in the next section.) Therefore, it is assumed that multiple measurements were performed to determine the weight. The measurement series, along with its mean and standard error, could look as follows:

```
measurement_series_503g = pd.Series([503, 503, 504, 502, 503, 502, 503, 504])
measurement_series_503g /= 1000 # convert to kilograms

# Determine t-value for significance level (choose alpha = 1 - 95%)

alpha = 0.05
n_mass = measurement_series_503g.shape[0] # number of elements
t_value_mass = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_mass - 1)

# Output

print(f"arithmetic mean of mass: {measurement_series_503g.mean()} kg",
      f"empirical standard error: {(sem_measurement_series_503g := pd.Series(measurement_series_503g.
f"t-value of mass: {t_value_mass:.4f}",
f"95% confidence interval: {(measurement_series_503g.mean() - t_value_mass * sem_measurement_
sep="\n")

arithmetic mean of mass: 0.503 kg
empirical standard error: 0.000267 kg
t-value of mass: 2.3646
95% confidence interval: 0.5024 kg - 0.5036 kg
```

The Gaussian error propagation is calculated similarly to linear error propagation.

$$u(k(m, s)) = \sqrt{\left(\frac{\partial f(m, s)}{\partial m} \cdot \sigma_m\right)^2 + \left(\frac{\partial f(m, s)}{\partial s} \cdot \sigma_s\right)^2}$$

The partial derivatives are inserted:

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2}$$

4.8 Complete Measurement Result

The measured values, constant g , and standard errors of the means are inserted:

$$u(k(m, s)) = \sqrt{\left(\frac{9.81m}{s^2 \cdot 0.1432m} \cdot 0.000267kg\right)^2 + \left((-)\frac{0.503kg \cdot 9.81m}{s^2 \cdot (0.1432m)^2} \cdot 0.00247m\right)^2}$$

Separate values and units:

$$u(k(m, s)) = \sqrt{\left(\frac{9.81 \cdot 0.000267}{0.1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 + \left((-)\frac{0.503 \cdot 9.81 \cdot 0.00247}{0.02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2}$$

Calculation, inserting Newton $\frac{kg \cdot m}{s^2}$ and canceling meters in the 2nd term:

$$u(k(m, s)) = \sqrt{\left(\frac{9.81 \cdot 0.000267}{0.1432} \frac{N}{m}\right)^2 + \left((-)\frac{0.503 \cdot 9.81 \cdot 0.00247}{0.02051} \frac{N}{m}\right)^2}$$

Combine terms:

$$u(k(m, s)) = \sqrt{\left(0.0183 \frac{N}{m}\right)^2 + \left((-)0.594 \frac{N}{m}\right)^2}$$

Final result:

$$u(k(m, s)) = 0.595 \frac{N}{m} \approx 0.60 \frac{N}{m}$$

Thus, the complete measurement result is:

$$k = 34.46 \frac{N}{m} \pm 0.60 \frac{N}{m}$$

4.9 95% Confidence Interval

In addition to the complete measurement result, a confidence interval can be given. According to the agreed confidence level, the corresponding t-values are used as correction factors.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

For the mass, $t = 2.3646$ and for the extension, $t = 2.2622$.

Insert measured values, constant g , standard error of the mean, and t-value in the first term:

$$\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 = \left(\frac{9.81m}{s^2 \cdot 0.1432m} \cdot 2.3646 \cdot 0.000267kg\right)^2$$

Separate values and units, inserting Newton $\frac{kg \cdot m}{s^2}$:

$$\left(\frac{9.81 \cdot 2.3646 \cdot 0.000267}{0.1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 = \left(0.0433 \frac{N}{m}\right)^2$$

Insert measured values, constant g , standard error of the mean, and t-value in the second term:

$$\left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2 = \left((-)\frac{0.503kg \cdot 9.81m}{s^2 \cdot (0.1432m)^2} \cdot 2.2622 \cdot 0.00247m\right)^2$$

Separate values and units, inserting Newton $\frac{kg \cdot m}{s^2}$ and cancel meters:

$$\left((-)\frac{0.503 \cdot 9.81 \cdot 2.2622 \cdot 0.00247}{0.02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)1.344 \frac{N}{m}\right)^2$$

The formula becomes:

$$u(k(m, s)) = \sqrt{\left(0.0433 \frac{N}{m}\right)^2 + \left((-)1.344 \frac{N}{m}\right)^2}$$

Calculate:

$$u(k(m, s)) = 1.345 \frac{N}{m} \approx 1.35 \frac{N}{m}$$

The interval limits are thus:

$$k = 34.46 \frac{N}{m} \pm 1.35 \frac{N}{m}$$

So:

$$k_{95\%} = 33, 11 \frac{N}{m} \leq 34, 46 \frac{N}{m} \leq 35, 81 \frac{N}{m}$$

Overview of Error Propagation for Basic Operations and Powers

Some functions are particularly easy to calculate. The following table lists calculation methods for determining measurement uncertainty according to Gaussian error propagation (standard error) and linear error propagation (maximum error).

Tabelle 3.1: Berechnung der Größfehler für Potenzprodukte und Grundrechenarten

$y = f(x_1, x_2)$	Standardfehler	Größfehler
$y = x_1 \pm x_2$	$\sigma_y = \pm \sqrt{\sigma_{x_1}^2 + \sigma_{x_2}^2}$	$\sigma_y = \sigma_{x_1} + \sigma_{x_2}$ absolute Fehler werden addiert
$y = x_1 \cdot x_2$	$\sigma_y = \pm xy \sqrt{(\frac{\sigma_{x_1}}{x_1})^2 + (\frac{\sigma_{x_2}}{x_2})^2}$ $\delta y = \pm x \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$	$\delta y = \delta x_1 + \delta x_2 = \frac{\delta x_1}{x_1} + \frac{\delta x_2}{x_2}$
$y = \frac{x_1}{x_2}$	$\sigma_y = \pm xy \sqrt{(\frac{\sigma_{x_1}}{x_1})^2 + (\frac{\sigma_{x_2}}{x_2})^2}$ $\delta y = \pm x \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$	$\delta y = \delta x_1 + \delta x_2$
$y = x_1^n \cdot x_2^m$	$\sigma_y = \pm \sqrt{(n \delta x_1)^2 + (m \delta x_2)^2}$	$\delta y = n \delta x_1 + m \delta x_2$

Figure 4.4: Error propagation for basic operations and powers

(Eden and Gebhard 2024, p. 36)

Special Case: Error Propagation with Single Values

A special case is determining measurement uncertainty when it arises from statistical confidence intervals and estimated error contributions from a single measurement.

To calculate a total error, a compromise must be made:

1. Linear error propagation is applied using the estimated error and statistical confidence interval.
2. Gaussian error propagation is applied with the statistical confidence interval, and an error component is added, determined from the linear error propagation of the estimated error.

“Both methods are probably equally suitable (or unsuitable) as a compromise in laboratory practice.” (Hempel (2016), p. 9)

(Hempel (2016), pp. 8–9)

DIN 1319 only deals with Gaussian error propagation. If only a single value is available for an input quantity, it is used as an estimate for that quantity. The uncertainty is based on available information or experience, e.g., from previous measurements. (Deutsches Institut für Normung 1996, p. 11)

As an example, we again use the measurement series for the 503-gram weight.

Note 15: Example Spring Constant

```
# Output
print(f"Weight in kilograms: {hooke_503g['mass'].unique()}",
      f"Mean extension: {hooke_503g['extension'].mean():.4f} m",
      f"Empirical standard error: { (sem_hooke_503g:= pd.Series(hooke_503g['extension'].ser
      f"t-value of the extension: {t_value_extension:.4f}",
      f"95%-confidence interval: {(hooke_503g['extension'].mean() - t_value_extension * ser
      sep = "\n")
```

```
Weight in kilograms: [0.503]
Mean extension: 0.1432 m
Empirical standard error: 0.002470 m
t-value of the extension: 2.2622
95%-confidence interval: 0.1376 m   0.1432 m   0.1488 m
```

4.10 Linear Error Propagation

Analogous to error estimation, the maximum error of the spring constant is given by:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| (-) \frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

- The reading error of the kitchen scale is estimated as $e_r(m) = 0.5g = 0.0005kg$. No systematic measurement deviation is known. Therefore, the maximum error of the mass is $\Delta m = 0.0005kg$.
- The empirical standard error of the extension is $\sigma_x = 0.00247m$, corresponding to the agreed confidence level of $\sigma_{x95\%} = 0.00247m \cdot 2.2622 = 0.00559m$.

Insert the measured values, the constant g , and the maximum errors:

$$\Delta k = \left| \frac{9.81m}{s^2 \cdot 0.1432m} \right| \cdot 0.0005kg + \left| (-) \frac{0.503kg \cdot 9.81m}{s^2 \cdot (0.1432m)^2} \right| \cdot 0.00247m$$

Separate values and units:

$$\Delta k = \left| \frac{9.81 \cdot 0.0005}{0.1432} \frac{m \cdot kg}{s^2 \cdot m} \right| + \left| (-) \frac{0.503 \cdot 9.81 \cdot 0.00247}{0.020506} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right|$$

Compute and insert Newton $\frac{kg \cdot m}{s^2}$:

$$\Delta k = \left| 0.0342 \frac{N}{m} \right| + \left| (-) 0.594 \frac{N \cdot m}{m^2} \right|$$

Cancel meters in the second term:

$$\Delta k = \left| 0.0342 \frac{N}{m} \right| + \left| (-) 0.594 \frac{N}{m} \right|$$

Round the measurement uncertainty to two decimal places:

$$\Delta k = 0.628 \frac{N}{m} \approx 0.63 \frac{N}{m}$$

Thus, the measured value and maximum error are:

$$k = 34.46 \frac{N}{m}; \Delta k = 0.63 \frac{N}{m}$$

4.11 Gaussian Error Propagation

The uncertainty of the spring constant is calculated as:

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m \right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s \right)^2}$$

Insert the measured values, the constant g , and the estimated error portion (maximum error) of the mass into the first term:

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9.81m}{s^2 \cdot 0.1432m} \cdot 0.0005kg\right)^2$$

Separate values and units, insert Newton $\frac{kg \cdot m}{s^2}$:

$$\left(\frac{9.81 \cdot 0.0005}{0.1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 = \left(0.0342 \frac{N}{m}\right)^2$$

Insert the mass of the weight, constant g , and standard error of the mean of the extension into the second term:

$$\left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2 = \left((-) \frac{0.503kg \cdot 9.81m}{s^2 \cdot (0.1432m)^2} \cdot 0.00247m\right)^2$$

Separate values and units, insert Newton $\frac{kg \cdot m}{s^2}$, and cancel m :

$$\left((-) \frac{0.503 \cdot 9.81 \cdot 0.00247}{0.02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-) 0.594 \frac{N}{m}\right)^2$$

The formula becomes:

$$u(k(m, s)) = \sqrt{\left(0.0342 \frac{N}{m}\right)^2 + \left((-) 0.594 \frac{N}{m}\right)^2}$$

Compute:

$$u(k(m, s)) = 0.595 \frac{N}{m} \approx 0.60 \frac{N}{m}$$

The complete measurement result is:

$$k = 34.46 \frac{N}{m}; \Delta k = 0.60 \frac{N}{m}$$

For the 95% confidence interval, the t-value of the extension $t = 2.2622$ is used:

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

The first term remains unchanged:

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9.81m}{s^2 \cdot 0.1432m} \cdot 0.0005kg\right)^2 = \left(0.0342 \frac{N}{m}\right)^2$$

Insert the mass, constant g , standard error of the mean of the extension, and the t-value $t = 2.2622$ into the second term:

$$\left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2 = \left((-) \frac{0.503kg \cdot 9.81m}{s^2 \cdot (0.1432m)^2} \cdot 2.2622 \cdot 0.00247m\right)^2$$

Separate values and units, insert Newton $\frac{kg \cdot m}{s^2}$, and cancel m :

$$\left((-)\frac{0.503 \cdot 9.81 \cdot 2.2622 \cdot 0.00247}{0.02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)1.344 \frac{N}{m}\right)^2$$

The formula becomes:

$$u(k(m, s)) = \sqrt{(0.0342 \frac{N}{m})^2 + ((-)1.344 \frac{N}{m})^2}$$

Compute:

$$u(k(m, s)) = 1.344 \frac{N}{m} \approx 1.35 \frac{N}{m}$$

The interval boundaries are:

$$k_{95\%} = 33.11 \frac{N}{m} \leq 34.46 \frac{N}{m} \leq 35.81 \frac{N}{m}$$

5 Exercise: Hooke's Law

Sensor data often come in multiple files. Possible reasons for this include measurements being taken

- by different people,
- at different locations,
- at different times,
- using different devices, or
- for different measurement quantities.

In the folder '01-daten/hooke', there are several txt files containing measurements of spring extension. In this chapter, you will apply what you have learned so far and answer the following questions:

1. Are there any invalid measurements?
2. What values can be determined for the spring constants?
3. Were the measurements taken with the same spring (or with springs of the same spring constant) if a confidence level of 90% or 95% is assumed?

In section [Section 5.1](#) you will find guidance, and in section [Section 5.2](#) a sample solution for reading the files. Afterwards, you are expected to work on the tasks independently. Section [Section 5.3](#) provides a sample solution for data preprocessing. Section [Section 5.10](#) contains the sample solution for determining the spring constants.

Note: Depending on the approach chosen, different results may arise (for example, when evaluating and handling extreme values / outliers).

5.1 Reading Files

To read files, you can use the `glob` module (see [Module on Reading Structured Datasets](#)).

First, you can pass the argument `pathname = *` to the function `glob.glob()`. The wildcard `*` stands for any sequence of characters (except path elements like `/` or `.`), allowing you to list all files stored in the specified folder. This way, you can determine the number of files and their extensions if they are not yet known.

```

folder_path = '01-daten/hooke'

file_path_list = glob.glob(pathname='*', root_dir=folder_path, recursive=False)
print(file_path_list)
print(f"Number of files: {len(file_path_list)}")

```

```

['team_fabi.txt', 'team_ma.txt', 'team_die_ahnungslosen.txt', 'team_kreativkoepfe.txt']
Number of files: 4

```

Using the file paths, the files can be read into a list using a loop. Initially, only the first 3 rows of each file are read to get an impression of the file structure.

```

for file_path in file_path_list:
    temp_storage = pd.read_csv(filepath_or_buffer = folder_path + '/' + file_path, nrows = 3)
    print(file_path, "\n", temp_storage, "\n", sep = '')

```

```

team_fabi.txt
    09:17:54\t110.31 cm\t0
0  09:17:57\t110.29 cm\t0
1  09:18:03\t110.74 cm\t0
2  09:18:06\t109.95 cm\t0

```

```

team_ma.txt
    10:09:38\t109.26 cm\t0
0  10:09:41\t109.26 cm\t0
1  10:09:44\t109.28 cm\t0
2  10:09:47\t109.18 cm\t0

```

```

team_die_ahnungslosen.txt
    11:03:23\t109.66 cm\t0
0  11:03:23\t109.62 cm\t0
1  11:03:23\t109.73 cm\t0
2  11:03:23\t109.55 cm\t0

```

```

team_kreativkoepfe.txt
    10:33:02\t109.64 cm\t0
0  10:33:05\t109.62 cm\t0
1  10:33:08\t109.64 cm\t0
2  10:33:11\t109.62 cm\t0

```

The files do not contain column headers and use the tab character `\t` as a delimiter. The first column contains a timestamp, the second the measured spring extension, and the third (presumably) the attached weight.

5.2 Task: Reading Files

Now read in the files. While doing so, check:

- whether the data types are correctly interpreted, and
- for missing values.

You can:

- a) Read each file individually.
- b) Use the `glob` module to read the files automatically and store each in a separate object (*Note:* This approach is demonstrated in the [Module on Reading Structured Datasets](#)).
- c) Use the `glob` module to read the files automatically and merge them into a single file.

These options require increasing levels of programming effort. The more separate files you want to analyze, the more automation is needed. Since sensor data analysis often involves numerous files, the model solution shows variant c).

Tip 8: Step-by-Step Approach

Reading the files is likely to be the most time-consuming and error-prone step. Develop your solution step by step. Start with variant a). Once you have read the files, you can familiarize yourself with variant b) using the `glob` module. Building on that, variant c) allows you to automate the process for any number of files.

Tip 9: Model Solution: Reading Files

The first attempt to read the files may fail with an error message. Therefore, the instructions are embedded in a try-except structure to catch the problematic file. (If multiple files cause errors, they would need to be stored in a list and processed later with a loop, if possible.)

```

hooke = pd.DataFrame(columns = ['Time', 'Distance', 'Weight', 'Team']) # an empty DataFrame

for path in file_path_list:

    try:
        temp_storage = pd.read_csv(filepath_or_buffer = folder_path + '/' + path, sep = '\t', n

        # Add filename as a column
        temp_storage['Team'] = path[5:-4] # remove 'team_' and the file extension

        hooke = pd.concat([hooke, temp_storage], ignore_index = True)

    except Exception as error:
        print("Problematic file path: ", path, error, sep = "\n")
        problem_file_path = path

print(hooke.info(), "\n")
print("Successfully read:\n", hooke['Team'].unique(), sep = '')

```

Problematic file path:

team_ma.txt

Error tokenizing data. C error: Expected 3 fields in line 135, saw 4

```

<class 'pandas.DataFrame'>
RangeIndex: 359 entries, 0 to 358
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        355 non-null    object
1   Distance    355 non-null    object
2   Weight      355 non-null    object
3   Team        359 non-null    object
dtypes: object(4)
memory usage: 11.3+ KB
None

```

Successfully read:

```
['fabi' 'die_ahnungslosen' 'kreativkoepfe']
```

According to the error message, line 135 contains 4 columns instead of 3. Therefore, the file causing the error is processed line by line, and each line with more than 3 entries is printed. For verification, the first 5 lines are also displayed.

```

# create an empty DataFrame with 3 columns

df = pd.DataFrame(data = [], columns = ['Time', 'Distance', 'Weight'])

problem_file_object = open(file = folder_path + '/' + problem_file_path, mode = 'r')

index = 0
for line in problem_file_object:
    try:
        buffer = line.split(sep = "\t")
        if len(buffer) > 3:
            print("Index =", index, ":", buffer)
        elif index <= 5:
            print(line)
        index += 1
    except Exception as error:
        print(error)

problem_file_object.close()

```

```
10:09:38    109.26 cm    0
```

```
10:09:41    109.26 cm    0
```

```
10:09:44    109.28 cm    0
```

```
Index = 134 : ['10:29:27', '105.49 cm', '300', '\n']
```

Every second line is empty. In line 134, a newline ‘\n’ is read. Therefore, the file is processed again using an adjusted loop. The extra newline ‘\n’ is removed from the relevant line. Empty lines are skipped. The correct lines are appended to the DataFrame ‘hooke’.

```

problem_file_object = open(file = folder_path + '/' + problem_file_path, mode = 'r')

for line in problem_file_object:
    try:
        buffer = line.split(sep = "\t")
        if len(buffer) > 3:
            buffer = buffer[:3]
        elif len(buffer) < 3: # skip empty lines
            continue
        # Append team name as 4th column
        buffer.append(problem_file_path[5:-4]) # remove 'team_' prefix and file extension

        hooke.loc[len(hooke)] = pd.Series(buffer).values

    except Exception as error:
        print(error)
        print(pd.Series(buffer).values)

problem_file_object.close()

print("Successfully read:\n", hooke['Team'].unique(), "\n", sep = '')
print(hooke.info())

```

Successfully read:

```
['fabi' 'die_ahnungslosen' 'kreativkoepfe' 'ma']
```

```

<class 'pandas.DataFrame'>
RangeIndex: 537 entries, 0 to 536
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        533 non-null    object
1   Distance    533 non-null    object
2   Weight      533 non-null    object
3   Team        537 non-null    object
dtypes: object(4)
memory usage: 16.9+ KB
None

```

Next, we look at the rows that contain null values ...

```
print(hooke.loc[hooke.apply(pd.isna).any(axis=1), :])
```

	Time	Distance	Weight	Team
207	NaN	NaN	NaN	die_ahnungslosen
247	NaN	NaN	NaN	kreativkoepfe
267	NaN	NaN	NaN	kreativkoepfe
272	NaN	NaN	NaN	kreativkoepfe

... and removed.

```
hooke.drop(np.where(hooke.apply(pd.isna).any(axis=1))[0], inplace=True)
```

Next, the data types are checked.

- The time can remain as a string, since it is not needed for the analysis.
- The measured distance is noted with ' cm' – this string will be removed. Afterwards, the column should be recognized as numeric.
- The weight should contain numeric values, but is read in as an object type and needs further investigation.
- The Team column could be assigned the [Pandas category datatype](#), but this is not necessary.

Remove the string ' cm'.

```
hooke.replace(' cm', '', regex = True, inplace = True)
```

	Time	Distance	Weight	Team
0	09:17:54	110.31	0	fabi
1	09:17:57	110.29	0	fabi
2	09:18:03	110.74	0	fabi
3	09:18:06	109.95	0	fabi
4	09:18:23	110.67	0	fabi
...	...	...	...	...
532	11:05:14	95.80	850\n	ma
533	11:05:17	95.61	850\n	ma
534	11:05:20	95.87	850\n	ma
535	11:05:23	95.35	850\n	ma
536	11:05:26	95.90	850\n	ma

You can check if all elements of a column are numeric using the Pandas method `pd.Series.str.isnumeric()`. A look at the data shows the cause.

```
print(hooke['Weight'].str.isnumeric().sum())

print(hooke['Weight'].head())
print(hooke['Weight'].tail())
```

```
1
0    0
1    0
2    0
3    0
4    0
Name: Weight, dtype: object
532    850\n
533    850\n
534    850\n
535    850\n
536    850\n
Name: Weight, dtype: object
```

The line breaks are also removed.

```
hooke.replace('\n', '', regex=True, inplace=True)
print(hooke['Weight'].str.isnumeric().sum())
print(hooke['Weight'].tail())
```

```
178
532    850
533    850
534    850
535    850
536    850
Name: Weight, dtype: object
```

```

print(hooke.info(), "\n")

# explicit assignment

hooke['Distance'] = hooke['Distance'].astype('float')
hooke['Weight'] = hooke['Weight'].astype('float')

print(hooke.info())

```

```

<class 'pandas.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        533 non-null    object
1   Distance    533 non-null    object
2   Weight      533 non-null    object
3   Team        533 non-null    object
dtypes: object(4)
memory usage: 20.8+ KB
None

```

```

<class 'pandas.DataFrame'>
Index: 533 entries, 0 to 536
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        533 non-null    object
1   Distance    533 non-null    float64
2   Weight      533 non-null    float64
3   Team        533 non-null    object
dtypes: float64(2), object(2)
memory usage: 20.8+ KB
None

```

The result might look like this:

```

hooke.groupby(by = ['Team', 'Weight'], sort = False)['Distance'].describe()

```

Team	Weight	count	mean	std	min	25%	50%	75%	max
fabi	0.0	10.0	110.366000	0.253693	109.95	110.2900	110.310	110.5800	110.74
	50.0	12.0	110.425833	0.210992	110.22	110.3050	110.310	110.6225	110.77
	101.0	9.0	108.238889	0.231163	107.96	107.9900	108.410	108.4200	108.47
	152.0	10.0	106.902000	0.335685	106.45	106.6000	106.980	106.9950	107.43
	201.0	10.0	105.253000	1.052416	102.30	105.4600	105.470	105.5775	105.90
	253.0	11.0	104.279091	0.466036	103.55	103.9400	104.100	104.6600	104.91
	302.0	12.0	102.650000	0.390943	102.09	102.4650	102.560	103.0100	103.35
	353.0	11.0	100.673636	0.199613	100.50	100.5500	100.580	100.7800	101.03
	403.0	14.0	99.447143	0.424815	98.97	99.1025	99.410	99.5900	100.26
	455.0	11.0	97.839091	0.228099	97.51	97.6200	97.880	98.0350	98.15
die_ahnungslosen	0.0	10.0	109.759000	0.180705	109.55	109.6300	109.730	109.7925	110.09
	50.0	8.0	108.912500	0.180930	108.53	108.8800	108.930	109.0250	109.11
	100.0	12.0	107.725833	0.704446	106.76	107.1975	107.735	108.0725	108.92
	150.0	19.0	106.265789	0.759505	104.67	105.7050	106.120	106.7100	107.87
	200.0	18.0	104.616111	0.392955	103.96	104.2650	104.590	104.8150	105.23
	250.0	18.0	102.906667	0.221519	102.35	102.8025	102.890	102.9750	103.33
	300.0	11.0	101.840000	0.894561	101.00	101.3650	101.460	101.8150	104.15
	350.0	14.0	100.264286	0.265930	99.98	100.1475	100.170	100.2625	101.06
	400.0	11.0	99.239091	0.232786	98.94	99.0500	99.130	99.4150	99.57
	450.0	12.0	98.042500	0.468016	97.51	97.6750	97.970	98.2850	99.08
kreativkoepfe	0.0	9.0	109.676667	0.053619	109.62	109.6400	109.640	109.7300	109.74
	50.0	8.0	108.557500	0.489949	107.84	108.2325	108.495	108.8675	109.33
	100.0	10.0	107.093000	0.922726	106.30	106.3800	106.875	107.2700	109.11
	150.0	10.0	105.934000	0.599967	104.87	105.6125	106.020	106.3075	106.84
	200.0	10.0	105.453000	1.618278	103.52	104.0750	105.035	106.7700	108.27
	250.0	10.0	102.527000	0.796228	101.10	102.1700	102.410	102.7625	103.89
	300.0	10.0	101.236000	0.702095	100.29	100.7625	101.010	101.8350	102.45
	350.0	10.0	98.804000	1.471629	97.43	97.5725	98.155	100.3600	100.84
	400.0	9.0	96.827778	1.036783	95.80	95.8900	96.900	97.4100	98.78
	450.0	9.0	95.515556	1.115954	93.91	94.6300	95.750	95.8300	97.24
ma	500.0	8.0	95.336250	1.564298	93.30	94.4575	95.140	96.5625	97.69
	550.0	9.0	93.155556	0.352956	92.76	92.8700	92.950	93.4000	93.76
	0.0	10.0	109.258000	0.042374	109.18	109.2600	109.260	109.2800	109.31
	50.0	10.0	107.179000	1.809588	105.83	106.0875	106.185	107.9575	110.81
	100.0	10.0	102.101000	2.958804	99.37	99.6725	101.120	104.4825	106.67
	150.0	11.0	106.656364	4.727985	100.29	100.9900	109.730	110.1600	110.60
	200.0	9.0	108.456667	1.666816	106.74	107.2900	107.740	110.1700	111.06
	250.0	10.0	106.931000	0.957838	105.66	105.9325	107.540	107.6975	107.80
	300.0	10.0	105.696000	0.194376	105.46	105.5075	105.715	105.8650	105.94
	350.0	10.0	104.838000	0.530949	104.10	104.5150	104.685	105.1475	105.94

Team	Weight	count	mean	std	min	25%	50%	75%	max
	400.0	10.0	104.790000	0.602974	104.31	104.4775	104.550	104.6925	106.09
	450.0	10.0	104.517000	0.475513	103.91	104.2225	104.510	104.7125	105.56
	500.0	10.0	104.456000	0.569097	103.65	104.0925	104.420	104.6400	105.71
	550.0	10.0	103.004000	0.800475	101.55	102.7650	103.065	103.5000	104.20
	600.0	10.0	101.403000	0.621183	100.22	101.5000	101.580	101.7800	101.94
	650.0	10.0	100.109000	0.082523	99.97	100.0475	100.110	100.1875	100.21
	700.0	10.0	98.871000	0.534155	98.24	98.5425	98.715	99.1850	100.02
	750.0	9.0	97.966667	0.729315	96.86	97.5000	97.980	98.6800	98.77
	800.0	11.0	97.115455	0.540210	96.19	96.7000	97.330	97.5050	97.72
	850.0	8.0	95.715000	0.245822	95.35	95.5625	95.750	95.8775	96.07

5.3 Sample Solution: Data Preparation

In the next step, the data will be checked and, if necessary, cleaned. This includes the following steps:

- checking for outliers (using studentized z-scores and visually) and cleaning if necessary,
- normalizing the distance measurement to the zero point and converting the units used to SI units, and
- graphical representation.

Warning 5: glob, pd.groupby, and SciPy

The execution of the following code may vary locally, depending on the order in which files are read using the glob module. In the following, index positions in a groupby object are determined where a condition is true. These index positions are then used to access the original DataFrame. Pandas sorts the groups in a groupby object alphabetically by default (`sort=True`). If the files read with glob are not in alphabetical order, incorrect index positions may be used to access the DataFrame. This can be avoided either by sorting the DataFrame according to the groupby criteria or, as shown here, by using the argument `pd.groupby(by=condition, sort=False)`. The latter approach is more flexible.

Also, `scipy.stats.zscore()` may return either a `pd.Series` or a `np.array` depending on the version. Accordingly, methods from Pandas or NumPy should be used.

1. Are there invalid measurements?

5.3.1 Checking for Outliers

Outliers can be checked (among other methods) using studentized z-scores and visually.

5.4 Determining Outliers

1. Determine the number of outliers and display their positions.

```
# Find z-scores greater than or equal to abs(3)
## Solution for SciPy 1.14.1 (returns pd.Series)
## z_values_ge3_sum = hooke.groupby(by=['Team', 'Weight'])['Distance'].apply(lambda x: scipy

## Solution for SciPy 1.15 and later (returns np.array)
z_values_ge3_sum = hooke.groupby(by=['Team', 'Weight'], sort=False)['Distance'].apply(scipy.s

print("Number of studentized z-scores with absolute value 3:", z_values_ge3_sum, "\n")

# Find z-scores greater than or equal to abs(2.5)
## Solution for SciPy 1.14.1 (returns pd.Series)
## z_values_ge25_sum = hooke.groupby(by=['Team', 'Weight'])['Distance'].apply(lambda x: scipy

## Solution for SciPy 1.15 and later (returns np.array)
z_values_ge25_sum = hooke.groupby(by=['Team', 'Weight'], sort=False)['Distance'].apply(scipy.s
print("Number of studentized z-scores with absolute value 2.5:", z_values_ge25_sum)

# Display rows with z-scores greater than abs(2.5)
## Solution for SciPy 1.14.1 (returns pd.Series)
## bool_index = hooke.groupby(by=['Team', 'Weight'])['Distance'].apply(lambda x: scipy.stats

## Solution for SciPy 1.15 and later (returns np.array)
### Step 1: Capture boolean values
bool_index = hooke.groupby(by=['Team', 'Weight'], sort=False)['Distance'].apply(scipy.stats.s
### Step 2: Flatten to a 1D array
bool_index = np.hstack(bool_index)

print("\nOutliers are located at these index positions:", np.nonzero(bool_index))
```

Number of studentized z-scores with absolute value 3: 0

Number of studentized z-scores with absolute value 2.5: 4

Outliers are located at these index positions: (array([44, 186, 201, 206]),)

2. Output of the lines from the measurement series

```
# Selecting rows with z-values greater than or equal to 2.5
z_values_ge_25 = hooke.iloc[bool_index , :] # --> The problem is likely here
print(z_values_ge_25, "\n")

# Determine combinations of Weight & Team

print("Determining combinations of Weight & Team")
teams = z_values_ge_25['Team'].unique()

## Iterate through teams and store corresponding weights

team_weights = [] # empty list
for i in range(len(teams)):
    print(teams[i])
    print(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Weight'], "\n")
    team_weights.append(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Weight'].values)

# print("As a list of arrays:")

# print(team_weights, "\n")
```

	Time	Distance	Weight	Team
44	09:27:18	102.30	201.0	fabi
186	11:10:38	102.35	250.0	die_ahnungslosen
201	11:11:42	104.15	300.0	die_ahnungslosen
206	11:12:43	101.06	350.0	die_ahnungslosen

Determining combinations of Weight & Team

fabi

44 201.0

Name: Weight, dtype: float64

die_ahnungslosen

186 250.0

201 300.0

206 350.0

Name: Weight, dtype: float64

5.5 Combined Output of Measurement Series

This output is intended for illustration and verification.

```
# Select measurement series
measurement_series = pd.DataFrame()
for i in range(len(teams)):
    for j in range(len(team_weights[i])):

        print(teams[i], team_weights[i][j])

        measurement_series = pd.concat([measurement_series, hooke.loc[(hooke['Team'] == teams[i]) & (hooke['Weight'] == team_weights[i][j])]['Distance']]

# Compute studentized z-scores of the measurement series
## Solution for SciPy 1.14.1 (returns pd.Series)
## measurement_series_z_scores = measurement_series.groupby(by=['Team', 'Weight'])['Distance'].apply(lambda x: scipy.stats.zscore(x, ddof=1))

## Solution for SciPy 1.15 and newer (returns np.array)
## Convert row-wise return to a one-dimensional array:
measurement_series_z_scores = np.hstack(
    measurement_series.groupby(by=['Team', 'Weight'], sort=False)['Distance'].apply(lambda x: scipy.stats.zscore(x, ddof=1))
    .reset_index(drop=True)
)

# Combined output of the data
## Solution for SciPy 1.14.1 (returns pd.Series)
## measurement_series.insert(loc=2, column='z-Score Distance', value=measurement_series_z_scores)

## Solution for SciPy 1.15 and newer (returns np.array)
measurement_series.insert(loc=2, column='z-Score Distance', value=measurement_series_z_scores)
print(measurement_series)
```

```
fabi 201.0
die_ahnungslosen 250.0
die_ahnungslosen 300.0
die_ahnungslosen 350.0
```

	Time	Distance	z-Score Distance	Weight	Team
41	09:27:09	105.47	0.206192	201.0	fabi
42	09:27:12	105.90	0.614776	201.0	fabi
43	09:27:15	105.54	0.272706	201.0	fabi
44	09:27:18	102.30	-2.805925	201.0	fabi

45	09:27:21	105.90	0.614776	201.0	fabi
46	09:27:24	105.47	0.206192	201.0	fabi
47	09:27:27	105.44	0.177686	201.0	fabi
48	09:27:30	105.59	0.320216	201.0	fabi
49	09:27:33	105.46	0.196690	201.0	fabi
50	09:27:36	105.46	0.196690	201.0	fabi
177	11:10:11	102.99	0.376191	250.0	die_ahnungslosen
178	11:10:13	102.99	0.376191	250.0	die_ahnungslosen
179	11:10:17	102.93	0.105333	250.0	die_ahnungslosen
180	11:10:20	102.83	-0.346095	250.0	die_ahnungslosen
181	11:10:23	103.16	1.143620	250.0	die_ahnungslosen
182	11:10:26	102.80	-0.481524	250.0	die_ahnungslosen
183	11:10:29	102.90	-0.030095	250.0	die_ahnungslosen
184	11:10:32	103.33	1.911049	250.0	die_ahnungslosen
185	11:10:35	102.78	-0.571810	250.0	die_ahnungslosen
186	11:10:38	102.35	-2.512954	250.0	die_ahnungslosen
187	11:10:41	102.93	0.105333	250.0	die_ahnungslosen
188	11:10:44	102.88	-0.120381	250.0	die_ahnungslosen
189	11:10:47	102.90	-0.030095	250.0	die_ahnungslosen
190	11:10:50	102.88	-0.120381	250.0	die_ahnungslosen
191	11:10:53	102.73	-0.797524	250.0	die_ahnungslosen
192	11:10:56	102.81	-0.436381	250.0	die_ahnungslosen
193	11:10:59	102.80	-0.481524	250.0	die_ahnungslosen
194	11:11:02	103.33	1.911049	250.0	die_ahnungslosen
195	11:11:24	101.37	-0.525397	300.0	die_ahnungslosen
196	11:11:27	101.43	-0.458325	300.0	die_ahnungslosen
197	11:11:30	102.80	1.073152	300.0	die_ahnungslosen
198	11:11:33	101.72	-0.134144	300.0	die_ahnungslosen
199	11:11:36	101.36	-0.536576	300.0	die_ahnungslosen
200	11:11:39	101.46	-0.424789	300.0	die_ahnungslosen
201	11:11:42	104.15	2.582271	300.0	die_ahnungslosen
202	11:11:45	101.36	-0.536576	300.0	die_ahnungslosen
203	11:11:48	101.00	-0.939008	300.0	die_ahnungslosen
204	11:11:51	101.91	0.078251	300.0	die_ahnungslosen
205	11:11:54	101.68	-0.178859	300.0	die_ahnungslosen
206	11:12:43	101.06	2.992196	350.0	die_ahnungslosen
208	11:12:46	100.17	-0.354551	350.0	die_ahnungslosen
209	11:12:49	99.98	-1.069025	350.0	die_ahnungslosen
210	11:12:52	100.07	-0.730590	350.0	die_ahnungslosen
211	11:12:55	100.21	-0.204135	350.0	die_ahnungslosen
212	11:12:58	100.21	-0.204135	350.0	die_ahnungslosen
213	11:13:01	100.14	-0.467363	350.0	die_ahnungslosen
214	11:13:04	100.17	-0.354551	350.0	die_ahnungslosen

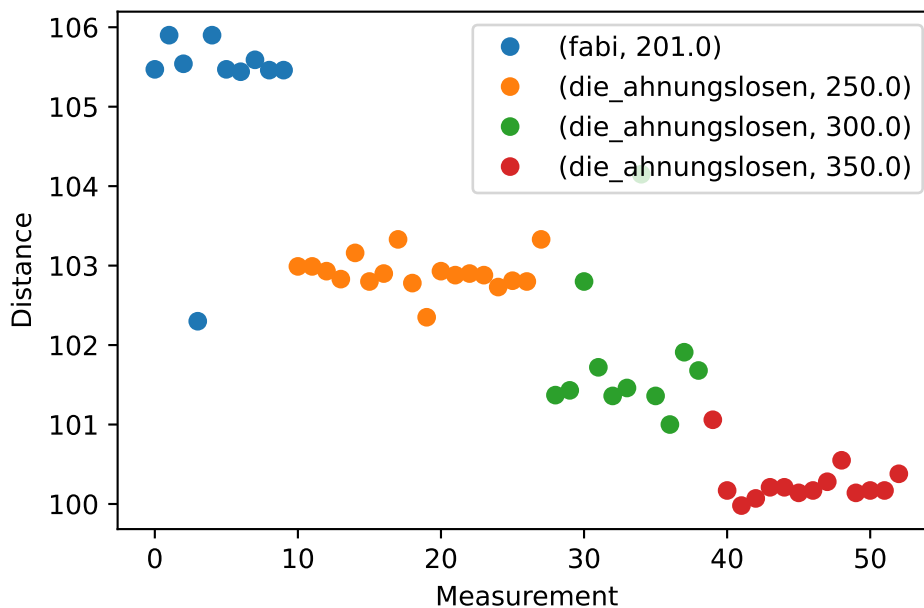
215	11:13:07	100.28	0.059092	350.0	die_ahnungslosen
216	11:13:10	100.55	1.074397	350.0	die_ahnungslosen
217	11:13:13	100.14	-0.467363	350.0	die_ahnungslosen
218	11:13:16	100.17	-0.354551	350.0	die_ahnungslosen
219	11:13:19	100.17	-0.354551	350.0	die_ahnungslosen
220	11:13:22	100.38	0.435131	350.0	die_ahnungslosen

The values can be visualized easily using the Pandas method `pd.plot()`. However, this method is not as flexible as the matplotlib package. For example, a scatter plot (`kind = 'scatter'`) is only available for DataFrames, not for groupby objects. This can be worked around by setting a marker and setting the line width to 0.

```
measurement_series.reset_index(drop = True).groupby(by = ['Team', 'Weight'], sort = False)['Distance']

plt.xlabel('Measurement')
plt.ylabel('Distance')
plt.legend()

plt.show()
```



Values with absolute studentized z-scores ≥ 2.5 could be considered outliers and removed. In this case, they are retained.

5.5.1 Conversion of Measurement Quantities

In the next step, the distance measurements are normalized to the zero point to represent the spring elongation. Similarly, the weight in g is converted into the acting force in N .

Tip 10: Example Solution

Normalize Distance Measurements to Meters and Zero Point

Normalize distance measurements to the zero point. Rename the column `Distance` to `Distance_Change`.

```
zero_points = hooke.loc[hooke['Weight'] == 0, :].groupby(by = 'Team', sort = False)['Distance'].min()

print("Zero points", zero_points, sep = '\n')

teams = zero_points.index

for i in range(len(teams)):

    hooke.loc[hooke['Team'] == teams[i], 'Distance'] = hooke.loc[hooke['Team'] == teams[i], 'Distance'] - zero_points[teams[i]]

hooke.rename(columns = {'Distance': 'Distance_Change'}, inplace = True)
hooke['Distance_Change'] = hooke['Distance_Change'].div(100)
```

Zero points

Team	
fabi	110.366000
die_ahnungslosen	109.759000
kreativkoepfe	109.676667
ma	109.258000

Name: Distance, dtype: float64

Convert Weight to Acting Force

Convert weight in g to the acting force in N . The column will be added to the dataset.

```
hooke['Force'] = hooke['Weight'].div(1000).mul(9.81)
```

The result might look like this. The column `Distance` was renamed to `Distance_Change`.

```
hooke.groupby(by = ['Team', 'Force'], sort = False)['Distance_Change'].describe()
```

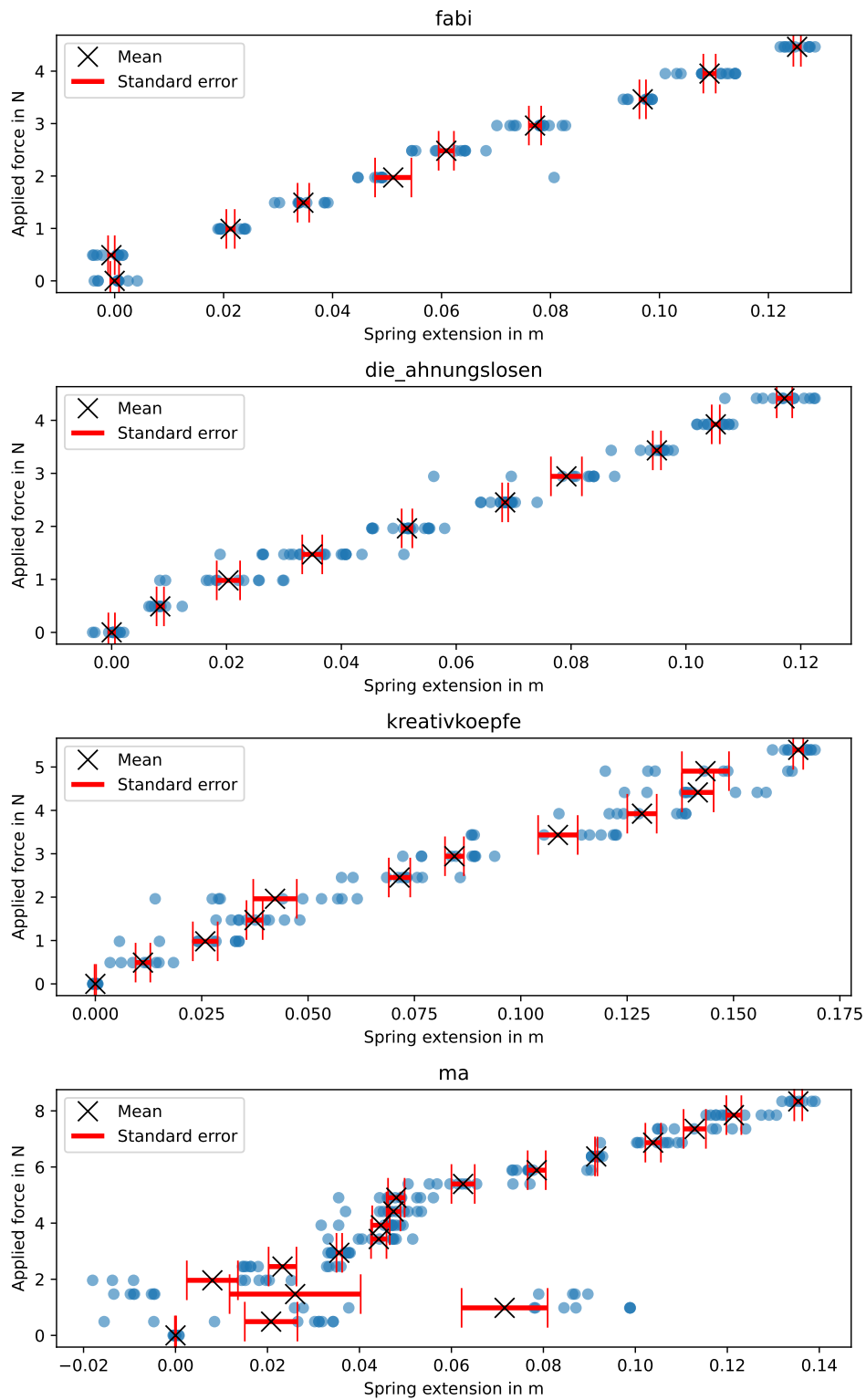
Team	Force	count	mean	std	min	25%	50%	75%
fabi	0.00000	10.0	1.137111e-16	0.002537	-0.003740	-0.002140	0.000560	0.000760
	0.49050	12.0	-5.983333e-04	0.002110	-0.004040	-0.002565	0.000560	0.000610
	0.99081	9.0	2.127111e-02	0.002312	0.018960	0.019460	0.019560	0.023760
	1.49112	10.0	3.464000e-02	0.003357	0.029360	0.033710	0.033860	0.037660
	1.97181	10.0	5.113000e-02	0.010524	0.044660	0.047885	0.048960	0.049060
	2.48193	11.0	6.086909e-02	0.004660	0.054560	0.057060	0.062660	0.064260
	2.96262	12.0	7.716000e-02	0.003909	0.070160	0.073560	0.078060	0.079010
	3.46293	11.0	9.692364e-02	0.001996	0.093360	0.095860	0.097860	0.098160
	3.95343	14.0	1.091886e-01	0.004248	0.101060	0.107760	0.109560	0.112635
	4.46355	11.0	1.252691e-01	0.002281	0.122160	0.123310	0.124860	0.127460
die_ahnungslosen	0.00000	10.0	-1.420088e-16	0.001807	-0.003310	-0.000335	0.000290	0.001290
	0.49050	8.0	8.465000e-03	0.001809	0.006490	0.007340	0.008290	0.008790
	0.98100	12.0	2.033167e-02	0.007044	0.008390	0.016865	0.020240	0.025615
	1.47150	19.0	3.493211e-02	0.007595	0.018890	0.030490	0.036390	0.040540
	1.96200	18.0	5.142889e-02	0.003930	0.045290	0.049440	0.051690	0.054940
	2.45250	18.0	6.852333e-02	0.002215	0.064290	0.067840	0.068690	0.069565
	2.94300	11.0	7.919000e-02	0.008946	0.056090	0.079440	0.082990	0.083940
	3.43350	14.0	9.494714e-02	0.002659	0.086990	0.094965	0.095890	0.096115
	3.92400	11.0	1.051991e-01	0.002328	0.101890	0.103440	0.106290	0.107090
	4.41450	12.0	1.171650e-01	0.004680	0.106790	0.114740	0.117890	0.120840
kreativkoepfe	0.00000	9.0	7.894196e-17	0.000536	-0.000633	-0.000533	0.000367	0.000367
	0.49050	8.0	1.119167e-02	0.004899	0.003467	0.008092	0.011817	0.014442
	0.98100	10.0	2.583667e-02	0.009227	0.005667	0.024067	0.028017	0.032967
	1.47150	10.0	3.742667e-02	0.006000	0.028367	0.033692	0.036567	0.040642
	1.96200	10.0	4.223667e-02	0.016183	0.014067	0.029067	0.046417	0.056017
	2.45250	10.0	7.149667e-02	0.007962	0.057867	0.069142	0.072667	0.075067
	2.94300	10.0	8.440667e-02	0.007021	0.072267	0.078417	0.086667	0.089142
	3.43350	10.0	1.087267e-01	0.014716	0.088367	0.093167	0.115217	0.121042
	3.92400	9.0	1.284889e-01	0.010368	0.108967	0.122667	0.127767	0.137867
	4.41450	9.0	1.416111e-01	0.011160	0.124367	0.138467	0.139267	0.150467
ma	4.90500	8.0	1.434042e-01	0.015643	0.119867	0.131142	0.145367	0.152192
	5.39550	9.0	1.652111e-01	0.003530	0.159167	0.162767	0.167267	0.168067
	0.00000	10.0	-7.105997e-17	0.000424	-0.000520	-0.000220	-0.000020	-0.000020
	0.49050	10.0	2.079000e-02	0.018096	-0.015520	0.013005	0.030730	0.031705
	0.98100	10.0	7.157000e-02	0.029588	0.025880	0.047755	0.081380	0.095855
	1.47150	11.0	2.601636e-02	0.047280	-0.013420	-0.009020	-0.004720	0.082680
	1.96200	9.0	8.013333e-03	0.016668	-0.018020	-0.009120	0.015180	0.019680
	2.45250	10.0	2.327000e-02	0.009578	0.014580	0.015605	0.017180	0.033255
	2.94300	10.0	3.562000e-02	0.001944	0.033180	0.033930	0.035430	0.037505
	3.43350	10.0	4.420000e-02	0.005309	0.033180	0.041105	0.045730	0.047430

Team	Force	count	mean	std	min	25%	50%	75%
	3.92400	10.0	4.468000e-02	0.006030	0.031680	0.045655	0.047080	0.047805
	4.41450	10.0	4.741000e-02	0.004755	0.036980	0.045455	0.047480	0.050355
	4.90500	10.0	4.802000e-02	0.005691	0.035480	0.046180	0.048380	0.051655
	5.39550	10.0	6.254000e-02	0.008005	0.050580	0.057580	0.061930	0.064930
	5.88600	10.0	7.855000e-02	0.006212	0.073180	0.074780	0.076780	0.077580
	6.37650	10.0	9.149000e-02	0.000825	0.090480	0.090705	0.091480	0.092105
	6.86700	10.0	1.038700e-01	0.005342	0.092380	0.100730	0.105430	0.107155
	7.35750	9.0	1.129133e-01	0.007293	0.104880	0.105780	0.112780	0.117580
	7.84800	11.0	1.214255e-01	0.005402	0.115380	0.117530	0.119280	0.125580
	8.33850	8.0	1.354300e-01	0.002458	0.131880	0.133805	0.135080	0.136955

5.5.2 Graphical Representation

Since there are only four teams, the measurements can be visualized graphically. One possible representation can be found in the first tab, with intermediate steps and conclusions in the following tabs.

5.6 Graphical Representation



5.7 Calculating Means and Standard Errors

The output is limited to the first few rows for space reasons.

Means of distance change by team and force:

Team	Force	
fabi	0.00000	1.137111e-16
	0.49050	-5.983333e-04
	0.99081	2.127111e-02
	1.49112	3.464000e-02
	1.97181	5.113000e-02

Name: SpringExtension, dtype: float64

Standard errors of means by team and force

Team	Force	
fabi	0.00000	0.000802
	0.49050	0.000609
	0.99081	0.000771
	1.49112	0.001062
	1.97181	0.003328

Name: StandardError, dtype: float64

5.8 Code

```
# Mean values of teams by force

distance_means_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Force']], sort = False)
distance_means_by_team_and_force.name = 'SpringExtension'

print("Mean values of distance change by team and force:", distance_means_by_team_and_force.

print() # empty line

# Standard errors of teams by force

distance_stderrors_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Force']], sort = False)
distance_stderrors_by_team_and_force.name = 'StandardError'

print("Standard errors of the means by team and force", distance_stderrors_by_team_and_force
```

```

# graphical representation

num_teams = hooke['Team'].unique().size

plt.figure(figsize = (7.5, 12))
for i in range(num_teams):

    plt.subplot(4, 1, i + 1) # plt.subplot counts from 1

    # scatter plot

    plotting_data = hooke.loc[hook['Team'] == hooke['Team'].unique()[i], :]
    plt.scatter(x = plotting_data['Distance_Change'], y = plotting_data['Force'], alpha = 0.6)

    plt.title(label = hooke['Team'].unique()[i])
    plt.xlabel("Spring extension in m")
    plt.ylabel("Applied force in N")

    # # error bars

    distance_means_by_force = plotting_data.groupby(by = plotting_data['Force'], sort = False)
    distance_stderrors_by_force = plotting_data.groupby(by = plotting_data['Force'], sort = False)

    errorbar_container = plt.errorbar(x = distance_means_by_force, y = distance_means_by_force,
    linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor = 'black')

    # see: [https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer]

    plt.legend([errorbar_container.lines[0], errorbar_container.lines[2][0]],
    ['Mean', 'Standard Error'],
    loc = 'upper left')

plt.tight_layout()
plt.show()

```

5.9 Evaluation

- The measurement series of the team *die_ahnungslosen* matches the expected linear trend.
- For team *fabi*, there seems to be an error in data collection for the first attached weight (50 grams). It was probably measured as 0 grams by mistake.
- The measurement series of the team *kreativköpfe* largely follows the expected linear trend.

- The measurement series of the team *ma* appears to be affected by significant measurement errors, at least for the first four attached weights.

The measurement series of team *ma* will be removed from the dataset due to major measurement errors. For team *fabi*, the measurement for the 50-gram weight will be removed.

```
hooke.drop(index = hooke.loc[hooke['Team'] == 'ma', :].index, inplace = True)
hooke.drop(index = hooke.loc[(hooke['Team'] == 'fabi') & (hooke['Weight'] == 50), :].index, inplace = True)
```

💡 Tip 11: Approach for Large Numbers of Datasets

When dealing with a large number of datasets, graphical inspection alone can reach its limits. In this case, it is recommended to combine visual and metric-based methods to identify patterns and verify them across many measurements. For example, after a visual inspection of measurement series with extreme values or outliers, all series could be checked to see if the mean spring elongation increases with increasing weight, as expected. Deviating series could then be further checked graphically.

5.10 Sample Solution: Determining Spring Constants

The next step is to determine the spring constants using linear regression.

2. What values can be determined for the spring constants?
3. Were the measurements performed with the same spring, assuming a confidence level of 90% or 95%?

5.11 $\alpha = 0.10$

```
fabi Confidence Level: 0.9
y = 0.2373 + 34.1048 * x
r = 0.9907 R2 = 0.9814 p = 0.0000
Standard error of the slope: 0.4792
33.309    34.105    34.901
```

```
die_ahnungslosen Confidence Level: 0.9
y = 0.1798 + 34.9183 * x
```

r = 0.9886 R2 = 0.9773 p = 0.0000
Standard error of the slope: 0.4651
34.148 34.918 35.689

kreativkoepfe Confidence Level: 0.9
y = 0.3180 + 29.7643 * x
r = 0.9773 R2 = 0.9552 p = 0.0000
Standard error of the slope: 0.6148
28.744 29.764 30.784

5.12 $\alpha = 0.05$

fabi Confidence level: 0.95
y = 0.2373 + 34.1048 * x
r = 0.9907 R2 = 0.9814 p = 0.0000
Standard error of the slope: 0.4792
33.154 34.105 35.056

die_ahnungslosen Confidence level: 0.95
y = 0.1798 + 34.9183 * x
r = 0.9886 R2 = 0.9773 p = 0.0000
Standard error of the slope: 0.4651
33.998 34.918 35.838

kreativkoepfe Confidence level: 0.95
y = 0.3180 + 29.7643 * x
r = 0.9773 R2 = 0.9552 p = 0.0000
Standard error of the slope: 0.6148
28.546 29.764 30.983

5.13 Code

```
num_teams = hooke['Team'].unique().size  
alpha = 0.10  
  
for i in range(num_teams):  
  
    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
```

```

x = reg_data['Distance_Change']
y = reg_data['Force']
n = len(x)

print("\n", hooke['Team'].unique()[i], " Confidence Level: ", 1 - alpha, sep='')

slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
f"Standard Error of Slope: {slope_stderr:.4f}", sep='')

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

alpha = 0.05

for i in range(num_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Distance_Change']
    y = reg_data['Force']
    n = len(x)

    print("\n", hooke['Team'].unique()[i], " Confidence Level: ", 1 - alpha, sep='')

    slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
    print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
f"Standard Error of Slope: {slope_stderr:.4f}", sep='')

    print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

```

5.14 Evaluation

The point estimate of the spring constant for Team fabi 34.105 lies within the 95% confidence interval of Team die_ahnungslosen' measurement 33.998 34.918 35.838. However, the point estimate of the spring constant for Team fabi 34.105 **does not lie within the 90% confidence interval** of Team die_ahnungslosen' measurement 34.148 34.918 35.689.

Regardless of the chosen confidence level, the point estimate of the spring constant for Team kreativkoepfe 29.764 does not lie within the confidence intervals of the other two teams.

 Warning 6: Results

Depending on the chosen approach, different outcomes are possible, for example by removing individual values classified as outliers or by treating Team fabi's measurement series for the attached 50-gram weight differently.

6 Calibration

The accuracy of measurements is improved by correcting systematic measurement deviations. In this chapter, typical *types of errors* in measurements and practical calibration methods are introduced:

- additive errors,
- multiplicative errors, and
- nonlinearity.

In this section, we use the established terminology, even though the errors would more precisely be described as systematic measurement deviations.

As an exercise, use cooked ice. (You have to use the zero point and 100 degrees as reference values). The problem is that cooking ice is also nonlinear (above 50 °C). Note that you do not have a reference value for the zero point (which lies at -273 °C), so you have to estimate it using linear regression. Presumably, it is not necessary to convert to Kelvin because of the absolute shift. → We have already shown for the calculation without measurement values for the zero point that no shift is required.

- But does this also apply to temperature sensors that cannot measure down to absolute zero at all? Perhaps by conceptualizing the lower limit of the measurement range as the zero point.
- The nonlinearity is not an error of the sensor system, but a physical effect. How do we deal with this? → One could say that the effect is approximated via the linearly estimated span error. Between the nonlinear measurement curve and the linearly estimated span error, a linearity error then arises.

The problem is that the data must be cleaned without interpolating or smoothing it → work with a linear trend and standard deviation, or refer to the module on data fitting and data optimization.

6.1 Calibration and Adjustment

The correction of systematic measurement deviations can be carried out in two ways: by calibration or by adjustment of the measuring instrument.

In the narrower sense, the term calibration refers to the determination of the relationship between measured values or their arithmetic mean and the agreed correct value of the measurand.

! Important 9: Calibration according to DIN 1319

“Determination of the relationship between measured values [...] or expected value [...] of the output quantity [...] and the associated true [...] or correct value [...] of the measurand present as the input quantity for a given measuring device [...]” (Deutsches Institut für Normung 1995, p. 22)

In a broader sense, calibration also means “the creation of a correction table [...], the determination of calibration factors or a (empirical) calibration function” (Deutsches Institut für Normung 1995, p. 22). These are methods used to correct measurement values affected by systematic measurement deviations without changing the measuring instrument.

In contrast, adjustment permanently changes the measuring instrument.

! Important 10: Adjustment according to DIN 1319

“Setting or aligning a measuring instrument [...], in order to eliminate systematic measurement deviations [...] as far as required for the intended application.” (Deutsches Institut für Normung 1995, p. 22)

6.2 Calibration Methods According to DIN 1319

DIN 1319 lists three calibration methods, which are only briefly discussed here:

1. Correction table,
2. Determination of calibration factors, and
3. Determination of a (empirical) calibration function.

Which method is used depends on the field of application, the availability of data, and the desired accuracy.

Method	Application for ...	Advantages	Disadvantages
Correction table	nonlinear, difficult to model	easy to use	many data points required, otherwise interpolation inaccurate
Calibration factors	linear deviation	easy to implement	insufficient for nonlinearity
Calibration function	nonlinear, modelable	very accurate	higher effort

6.2.1 Correction Table

A correction table can represent arbitrary corrections, provided that sufficient data points are available. In Section 6.6.1 we will learn about an example of a correction table.

6.2.2 Determination of Calibration Factors

A calibration factor can be specified when a constant relative deviation is present. A practical example can be found [here on page 1](#).

6.2.3 Determination of a (Empirical) Calibration Function

A calibration function allows the modeling of complex relationships. The methods for this are presented in the [module Data Fitting and Data Optimization](#).

6.3 Additive Errors: The Zero Offset Error

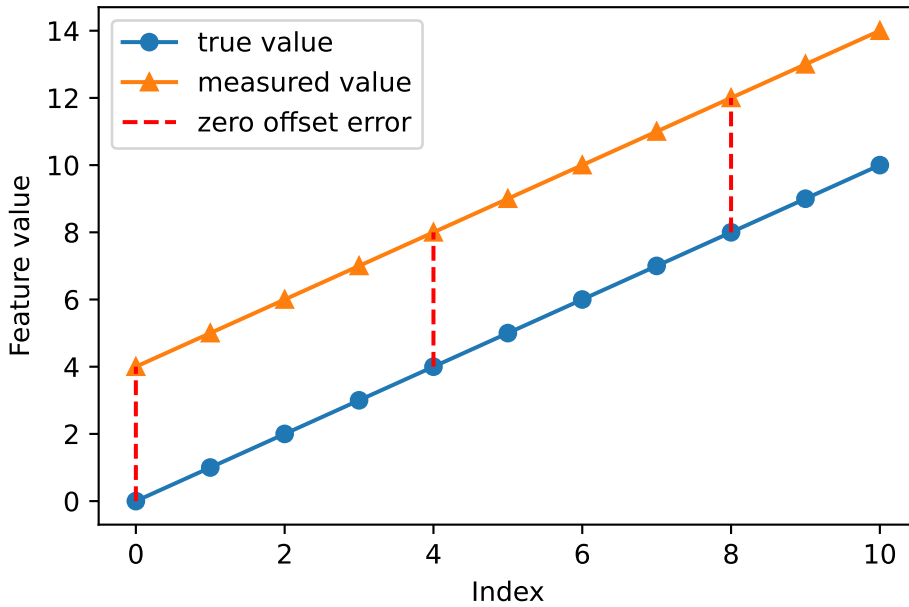
The zero offset error describes the deviation of the measured quantity from zero when the true value is actually zero. The zero offset error is also referred to as zero adjustment error, zero shift, or offset error. The zero offset error is constant and acts additively on every measured value. (ICS Schneider Messtechnik)

```
# Generate data
measured_quantity = np.arange(0, 11)
measured_values = measured_quantity + 4

# Plot
plt.plot(measured_quantity, marker='o', label='true value')
plt.plot(measured_values, marker='^', label='measured value')
plt.plot([0, 0], [measured_quantity[0], measured_values[0]], linestyle='dashed', label='zero offset')
plt.plot([4, 4], [measured_quantity[4], measured_values[4]], linestyle='dashed', color='red', label='zero offset')
plt.plot([8, 8], [measured_quantity[8], measured_values[8]], linestyle='dashed', color='red', label='zero offset')

# Axis labels
plt.xlabel('Index')
plt.ylabel('Feature value')

plt.legend()
plt.show()
```



6.3.1 Quantifying the Zero-Offset Error

The calculations here are not yet correct. With simple values it seems to work by chance, but with shifted values it no longer does. The correct approach is shown in the test using Kelvin.

The zero-offset error can be easily determined if measurement values for the zero point are available and the true value is known. (If it is also known that no further systematic measurement deviations are present, the zero-offset error can be determined using any true reference value.) This is the case for the data created for graphical representation.

```
print(f"The zero-offset error is: {measured_values[0] - measured_quantity[0]}")
```

The zero-offset error is: 4

If no data for the zero point is available, the zero-point error can be estimated using linear regression from at least two known reference points. This involves estimating an ideal calibration curve and determining its intersection with the y-axis. Likewise, a linear function is estimated for the measured values. The difference between the y-axis intercepts gives an estimate of the zero-point error.

```

# known reference points

x1 = 4
x2 = 8
y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# estimate ideal calibration curve

# print(linear_model) returns intercept + slope

lm_curve = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
print("The first value is the intersection of the ideal calibration curve with the y-axis:",

# estimate linear function for the measured values

## at least 2 points are needed

lm_measurements = poly.polyfit(x = [x1, x2], y = [measured_values[x1], measured_values[x2]]),
print("Linear function estimated from 2 measured values:", lm_measurements.round(2), sep = "

## estimation using all measured values is more reliable

lm_measurements_all = poly.polyfit(x = np.arange(measured_values.size), y = measured_values,
print("Linear function estimated from all measured values:", lm_measurements_all.round(2), s

print("\nThe difference between the estimated zero-point of the measured values and the\nest.

```

The first value is the intersection of the ideal calibration curve with the y-axis:

```
[-0.  1.]
```

Linear function estimated from 2 measured values:

```
[4.  1.]
```

Linear function estimated from all measured values:

```
[4.  1.]
```

The difference between the estimated zero-point of the measured values and the estimated zero-point of the ideal calibration curve is the zero-point error:

```
4.0
```

The procedure is illustrated graphically.

```

# calculate linear functions

estimated_ideal_curve = poly.polyval(x = np.arange(measured_values.size), c = lm_curve)
linear_function_measurements = poly.polyval(x = np.arange(measured_values.size), c = lm_meas

# plotting

## points

plt.plot([x1, x2], [y1, y2], marker = 'o', linestyle = 'none', color = 'C0', label = 'known r
plt.plot([x1, x2], [measured_values[x1], measured_values[x2]], marker = '^', linestyle = 'no

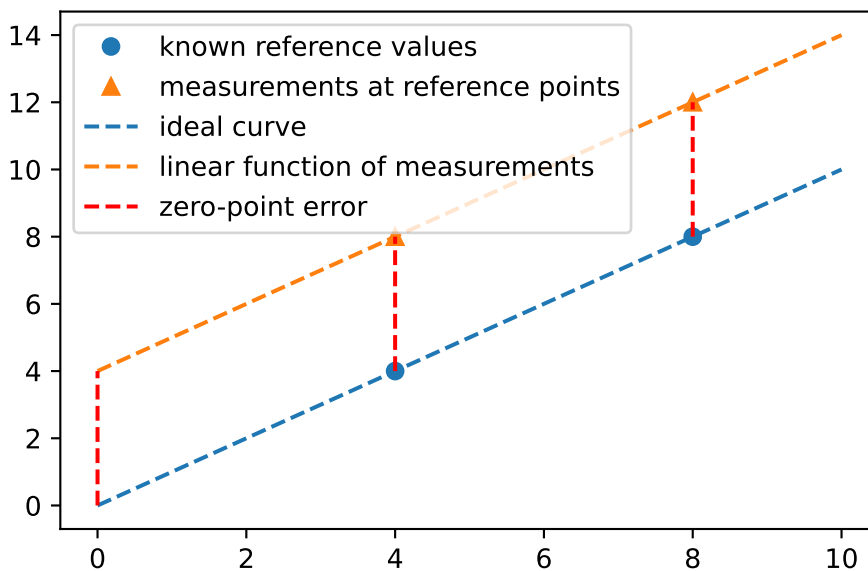
## lines

plt.plot(np.arange(measured_values.size), estimated_ideal_curve, marker = 'none', linestyle =
plt.plot(np.arange(measured_values.size), linear_function_measurements, marker = 'none', lin

plt.plot([0, 0], [measured_quantity[0], measured_values[0]], linestyle = 'dashed', label = '
plt.plot([4, 4], [measured_quantity[4], measured_values[4]], linestyle = 'dashed', color = '
plt.plot([8, 8], [measured_quantity[8], measured_values[8]], linestyle = 'dashed', color = '

plt.legend()
plt.show()

```



Are there other methods?

6.3.2 Correcting the zero-point error

The zero-point error is corrected by subtracting it from the measured values. The procedure for determining the zero-point error is illustrated in the following example with different values.

```
# Generate data
measured_quantity = np.arange(0, 11) + 5
measured_values = measured_quantity + 2.5

# known reference points
x1 = 2
x2 = 5
y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# estimate ideal characteristic line
ideal_line = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)

# estimate linear function of the measurements
## estimation over all measurements is more reliable
measured_line = poly.polyfit(x = np.arange(measured_values.size), y = measured_values, deg = 1)

# determine zero-point error
zero_point_error = measured_line[0] - ideal_line[0]
print(f"The zero-point error is: {zero_point_error.round(2)}")

# correct zero-point error
corrected_measurements = measured_values - zero_point_error
```

The zero-point error is: 2.5

6.3.3 Exercise: Zero-Point Error

Andres' bike sensor always measures 3 degrees too high because the display generates heat (offset error). Here are the values with the display off and the known reference values.

6.4 Multiplicative Error: Span Error

Span error seems to be a rarely used term.

A span error is an error that increases or decreases over the measurement range (the span) of the quantity being measured. The relative measurement deviation δ remains constant, while the absolute measurement deviation Δ depends on the measured value. (ics [Schneider Messtechnik](#))

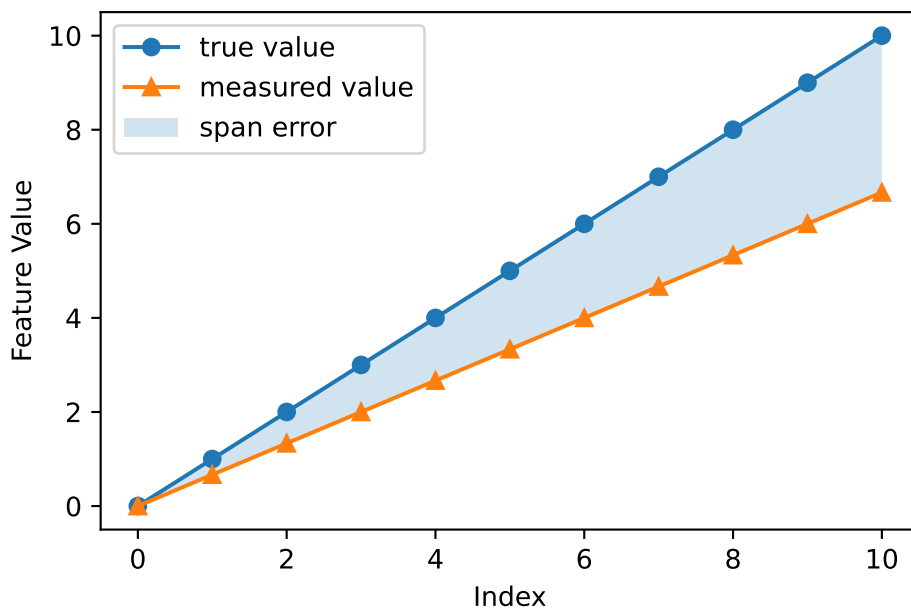
```
# Generate data
measured_quantity = np.arange(0, 11)
measured_values = measured_quantity - measured_quantity / 3

# Plot
plt.plot(measured_quantity, marker='o', label='true value')
plt.plot(measured_values, marker='^', label='measured value')

# Fill area
plt.fill_between(x=np.arange(measured_values.size), y1=measured_quantity, y2=measured_values)

# Axis labels
plt.xlabel('Index')
plt.ylabel('Feature Value')

plt.legend()
plt.show()
```



6.4.1 Quantifying Span Error

The span error can be estimated if reference values are known. With the complete example data, it's straightforward (in this example, a warning due to division by 0 is suppressed):

```
print(f"The span error is: {round((np.nanmean((measured_values / measured_quantity)) - 1) * 100, 2)} %")
```

The span error is: -33.33 %

For sparse data points, an ideal curve is estimated to approximate the span error.

```
# known reference values

x1 = 4
x2 = 8
y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# estimate ideal curve

lm_curve = poly.polyfit(x=[x1, x2], y=[y1, y2], deg=1)
estimated_ideal_curve = poly.polyval(x=np.arange(measured_values.size), c=lm_curve)

# linear regression of the measurements

lm_measurements = poly.polyfit(x=np.arange(measured_values.size), y=measured_values, deg=1)

# output

print(f"Slope of the measurements: {round(lm_measurements[1], 2)}")
print(f"Slope of the ideal curve: {round(lm_curve[1], 2)}")
```

Slope of the measurements: 0.67

Slope of the ideal curve: 1.0

The procedure is illustrated graphically.

```
# Points and lines

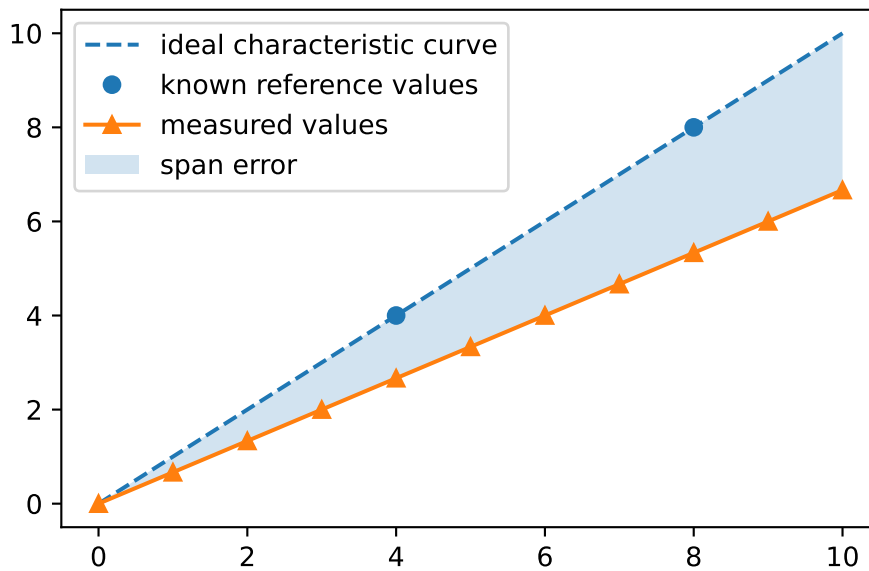
plt.plot([0, 10], [measured_quantity[0], measured_quantity[-1]], marker='none', linestyle='dashed')
```

```
plt.plot([x1, x2], [y1, y2], marker='o', linestyle='none', color='C0', label='known reference values')
plt.plot(measured_values, marker='^', color='C1', label='measured values')

# fill

plt.fill_between(x=np.arange(measured_values.size), y1=measured_quantity, y2=measured_values)

plt.legend()
plt.show()
```



6.4.2 Correcting Span Error

The span error is corrected by dividing the measured values by 1 plus the span error. The procedure for determining the span error is illustrated in the following example with different values.

```
# Generate data
measured_quantity = np.arange(0, 11) + 5
measured_values = measured_quantity - measured_quantity / 5

# Known reference points
x1 = 2
x2 = 5
```

```

y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# Estimate ideal characteristic line
ideal_line = poly.polyfit(x=[x1, x2], y=[y1, y2], deg=1)

# Estimate linear function of measured values
measured_line = poly.polyfit(x=np.arange(measured_values.size), y=measured_values, deg=1)

# Determine span error
span_error = measured_line[1] - ideal_line[1]
print(f"The span error is: {((measured_line[1] - ideal_line[1]) * 100).round(2)} %")

# Correct span error
corrected_values = measured_values / (1 + span_error)

print(f"True values:\n{measured_quantity}")
print(f"Measured values:\n{measured_values}")
print(f"Corrected measured values:\n{corrected_values}")

```

```

The span error is: -20.0 %
True values:
[ 5  6  7  8  9 10 11 12 13 14 15]
Measured values:
[ 4.   4.8  5.6  6.4  7.2  8.   8.8  9.6 10.4 11.2 12. ]
Corrected measured values:
[ 5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15.]

```

6.4.3 Exercise: Span Error

to do

6.5 Zero-Point and Span Errors

Absolute and multiplicative deviations can occur simultaneously. In this example, the absolute measurement deviation decreases over the displayed value range, because the zero-point error has a positive sign and the span error has a negative sign.

```

# Generate data
measured_quantity = np.arange(0, 11)
measured_values = measured_quantity + 4 - measured_quantity / 3

# Plot
plt.plot(measured_quantity, marker='o', label='true value')
plt.plot(measured_values, marker='^', label='measured value')
plt.plot([0, 0], [measured_quantity[0], measured_values[0]], linestyle='dashed', label='zero-point error')

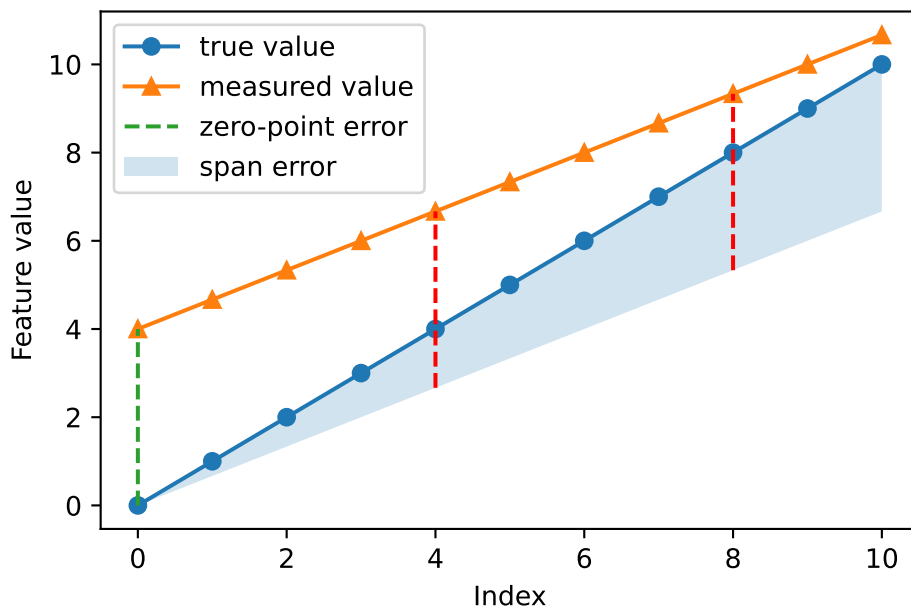
# Fill span error
span_error_line = measured_values - 4
plt.fill_between(x=np.arange(measured_values.size), y1=measured_quantity, y2=span_error_line)

# Show zero-point error at specific points
plt.plot([4, 4], [span_error_line[4], measured_values[4]], linestyle='dashed', color='red')
plt.plot([8, 8], [span_error_line[8], measured_values[8]], linestyle='dashed', color='red')

# Axis labels
plt.xlabel('Index')
plt.ylabel('Feature value')

plt.legend()
plt.show()

```



6.5.1 Quantifying Zero-Point and Span Errors

Using at least two known reference points, the zero-point and span errors can be estimated with linear regression. In linear regression, an index starting at 0 is used for the x-values. To correctly determine the zero-point error, this index is shifted by the estimated true zero point.

This is necessary because if the measured quantity does not include a value for 0, the span error would also be applied to the value at index 0. Therefore, the span error must be corrected first.

(If the values include 0, this step is not noticeable.)

```
# Generate data
measured_quantity = np.arange(0, 11) + 7
measured_values = measured_quantity + 4 - measured_quantity / 3

# Known reference points
x1 = 4
x2 = 8
y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# Estimate ideal characteristic line
lm_characteristic = poly.polyfit(x = [x1, x2], y = [y1, y2], deg = 1)
print(f"lm_characteristic: {lm_characteristic}")

# Linear regression of measurements
lm_measurements = poly.polyfit(x = np.arange(measured_values.size), y = measured_values, deg = 1)

# Linear regression of measurements plus y-intercept of ideal characteristic line
lm_measurements_plus_intercept = poly.polyfit(np.arange(measured_values.size) + lm_characteristic[0], y = measured_values, deg = 1)

# Determine zero-point error
zero_point_error = lm_measurements_plus_intercept[0]
print(f"The zero-point error is: {zero_point_error.round(2)}")

# Determine span error
span_error = lm_measurements[1] - lm_characteristic[1]
print(f"The span error is: {(lm_measurements[1] - lm_characteristic[1]) * 100}.round(2)} %")
```

```
lm_characteristic: [7. 1.]
```

The zero-point error is: 4.0
The span error is: -33.33 %

Graphical Representation **to-do**

```
# Optional: graphical representation
# estimated_ideal_curve = poly.polyval(x = np.arange(measured_values.size), c = lm_curve)
```

6.5.2 Correcting Zero-Point and Span Errors

The order of correction matters.

1. First, the zero-point error is corrected.
2. Then, the span error is corrected.

```
# Generate data

measured_quantity = np.arange(0, 11) + 7
measured_values = measured_quantity + 4 - measured_quantity / 3

# known reference points

x1 = 4
x2 = 8
y1 = measured_quantity[x1]
y2 = measured_quantity[x2]

# Output

print(f"True values:\n{measured_quantity}")
print(f"Measurements:\n{measured_values}")
print()

# first correct zero-point error, then span error

## Correct zero-point error

corrected_measurements = measured_values - zero_point_error

## Correct span error
```

```

corrected_measurements = corrected_measurements / (1 + span_error)

print(f"Zero-point corrected first, then span error corrected\n{corrected_measurements}")

# first correct span error, then zero-point error

## Correct span error

corrected_measurements = measured_values / (1 + span_error)

## Correct zero-point error

corrected_measurements = corrected_measurements - zero_point_error

print(f"Span error corrected first, then zero-point error corrected\n{corrected_measurements}")
print()

# Arnold's approach

true_minimum = measured_quantity.min()
true_maximum = measured_quantity.max()

a = (true_maximum - true_minimum) / (measured_values.max() - measured_values.min())
b = true_minimum - a * measured_values.min()

corrected_measurements = a * measured_values + b

print(f"Arnold's approach\n",
      f"a = {a:.1f}\n",
      f"b = {b:.1f}\n",
      f"\na * measurements + b\n{corrected_measurements}",
      sep = ' ')

```

True values:

```
[ 7  8  9 10 11 12 13 14 15 16 17]
```

Measurements:

```
[ 8.66666667  9.33333333 10.          10.66666667 11.33333333 12.
 12.66666667 13.33333333 14.          14.66666667 15.33333333]
```

Zero-point corrected first, then span error corrected

```
[ 7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17.]
```

Span error corrected first, then zero-point error corrected

```
[ 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19.]
```

Arnold's approach

```
a = 1.5
```

```
b = -6.0
```

```
a * measurements + b
```

```
[ 7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17.]
```

Simone's approach leads to correctly calibrated measurements. The variable **a** is the correction factor for the span error. However, the variable **b** has no intrinsic meaning. But why does it work then?! Because **b** represents the zero-point correction when the span error is corrected first.

This has to do with the structure of the data and the correction. **b** is the correction of the zero-point error * span error. This means that from the Arnold formula, the span error can be calculated as $-1 * \frac{b}{a}$.

My approach is more flexible because it can estimate the correction from arbitrary points, rather than just the minimum and maximum.

6.6 Nonlinearity: Linearity Error

6.6.1 Example Pt100

The Pt100 is a platinum resistance thermometer (Pt = Platinum) with a defined resistance of 100Ω at a temperature of 0°C (hence the name **Pt100**). The resistance of a Pt100 increases with temperature. At 100°C , for example, the resistance is 138.51Ω . However, the relationship between the input quantity, the electrical resistance, and the measured temperature is only approximately linear.

The characteristics of a Pt100 resistor are specified in the standard DIN EN IEC 60751 (Deutsches Institut für Normung [2023](#)). The relationship is described by two polynomials: one for the temperature range from -200°C to 0°C , and one for 0°C to 850°C .

For the temperature range -200°C to 0°C :

$$R_T = R_0 \cdot (1 + A \cdot T + B \cdot T^2 + C \cdot (T - 100^\circ\text{C}) \cdot T^3)$$

For the temperature range 0°C to $+850^\circ\text{C}$:

$$R_T = R_0 \cdot (1 + A \cdot T + B \cdot T^2)$$

Where:

- R_T is the resistance at temperature T ,
- R_0 is the resistance at 0°C ,
- A , B , and C are constants describing the sensor's specific characteristics. Specifically:

$$\begin{aligned}
- A &= 3.9083 \times 10^{-3} \text{ }^\circ\text{C}^{-1} \\
- B &= -5.775 \times 10^{-7} \text{ }^\circ\text{C}^{-2} \\
- C &= -4.183 \times 10^{-12} \text{ }^\circ\text{C}^{-4}
\end{aligned}$$

(Deutsches Institut für Normung [2023](#), p. 13)

The appendix of the standard contains tables showing the relationship between temperature and measured resistance. The table format allows quickly determining the measured temperature from a measured resistance value. The first column lists temperatures in tens of degrees, while the next ten columns represent the unit digits. For temperatures below zero, the unit digit is subtracted (indicated by the negative sign $-$), and for temperatures above zero, it is added (indicated by the positive sign $+$). A consolidated version can be found, for example, [here](#).

Tabelle A.1 – Beziehung zwischen Temperatur und Widerstand unter 0°C ; $R_0 = 100,00 \Omega$

$t_{90}/^\circ\text{C}$	Widerstand in Ohm bei der Temperatur $t_{90}/^\circ\text{C}$										$t_{90}/^\circ\text{C}$
	0	-1	-2	-3	-4	-5	-6	-7	-8	-9	
-200	18,52										-200
-190	22,83	22,40	21,97	21,54	21,11	20,68	20,25	19,82	19,38	18,95	-190
-180	27,10	26,67	26,24	25,82	25,39	24,97	24,54	24,11	23,68	23,25	-180
-170	31,34	30,91	30,49	30,07	29,64	29,22	28,80	28,37	27,95	27,52	-170

Figure 6.1: Excerpt from Table A.1

(Deutsches Institut für Normung [2023](#), p. 25)

The notation $t_{90}/^\circ\text{C}$ is based on the International Temperature Scale ITS-90 of 1990, which defines temperatures T_{90} in Kelvin and t_{90} in degrees Celsius. The notation indicates that the table values are in degrees Celsius. (Deutsches Institut für Normung ([2023](#)), p. 10)

6.6.2 Task: Reading the Pt100 Table

For data analysis, the tabular format is less suitable. The [consolidated version](#) was copied into two CSV files. Read the files so that the electrical resistance and temperature each form ascending sequences (e.g., as a list, NumPy array, or a Pandas data structure). The file paths are:

```
'skript/01-daten/pt100-table-below-zero.csv'
'skript/01-daten/pt100-table-above-zero.csv'
```

💡 Tip 12: Read Sample Solution Pt100 Table

```
file_path_below_zero = '01-daten/pt100-table-below-zero.csv'
file_path_above_zero = '01-daten/pt100-table-above-zero.csv'
```

Import and view files. First, the file with temperatures below 0 degrees Celsius.

```
# below_zero

below_zero = pd.read_csv(filepath_or_buffer = file_path_below_zero, sep = ',')

print(below_zero.head(), "\n")
print(below_zero.tail())
```

	Temperatur in °C	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	\
0	NaN	0	-1	-2	-3	-	
4							
1	-200.0	18,52	NaN	NaN	NaN	NaN	
2	-190.0	22,83	22,4	21,97	21,54	21,11	
3	-180.0	27,1	26,67	26,24	25,82	25,39	
4	-170.0	31,34	30,91	30,49	30,07	29,64	

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	
0	-5	-6	-7	-8	-9	
1	NaN	NaN	NaN	NaN	NaN	
2	20,68	20,25	19,82	19,38	18,95	
3	24,97	24,54	24,11	23,68	23,25	
4	29,22	28,8	28,37	27,95	27,52	

	Temperatur in °C	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	\
17	-40.0	84,27	83,87	83,48	83,08	82,69	
18	-30.0	88,22	87,83	87,43	87,04	86,64	
19	-20.0	92,16	91,77	91,37	90,98	90,59	
20	-10.0	96,09	95,69	95,3	94,91	94,52	
21	0.0	100	99,61	99,22	98,83	98,44	

	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	
17	82,29	81,89	81,5	81,1	80,7	
18	86,25	85,85	85,46	85,06	84,67	
19	90,19	89,8	89,4	89,01	88,62	
20	94,12	93,73	93,34	92,95	92,55	
21	98,04	97,65	97,26	96,87	96,48	

There are 201 values (from -200 to 0). The data starts at the row with index 1 (if the first row is read as header). This row contains only one entry in column '0'. The following rows should be read from right to left. The decimal separator is ,.

Reading the data from right to left can be done in many ways. A one-line solution starts by passing a negative step size to the method `pd.iloc[::-1]`.

```
df = pd.DataFrame(np.array([[3, 2, 1], [6, 5, 4]]))
print(df, "\n")
print(df.iloc[::-1])
```

```
   0  1  2
0  3  2  1
1  6  5  4
```

```
   0  1  2
1  6  5  4
0  3  2  1
```

This causes the *rows* of the DataFrame to be displayed in reverse order. To display the *columns* in reverse order, the DataFrame is transposed twice using the `pd.T` method.

```
print(df.T.iloc[::-1].T)
```

```
   2  1  0
0  1  2  3
1  4  5  6
```

The NumPy method `np.flatten()` can be used to reduce an array to a one-dimensional structure. To do this, the DataFrame is converted to a NumPy array using the `pd.to_numpy()` method.

```
print(df.T.iloc[::-1].T.to_numpy())
print() # empty line
print(df.T.iloc[::-1].T.to_numpy().flatten())
```

```
[[1 2 3]
 [4 5 6]]
```

```
[1 2 3 4 5 6]
```

Let's try looking at the head of the Pt100 data.

```
below_zero = pd.read_csv(filepath_or_buffer=file_path_below_zero, sep=',', decimal=',', header=0)

print(below_zero.head())
print() # empty line
print(below_zero.head().T.iloc[:, :-1].T.to_numpy().flatten())
```

```

          0      -1      -2      -3      -4      -5      -6      -7      -8      -
9
-200  18.52   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
-190  22.83  22.40  21.97  21.54  21.11  20.68  20.25  19.82  19.38  18.95
-180  27.10  26.67  26.24  25.82  25.39  24.97  24.54  24.11  23.68  23.25
-170  31.34  30.91  30.49  30.07  29.64  29.22  28.80  28.37  27.95  27.52
-160  35.54  35.12  34.70  34.28  33.86  33.44  33.02  32.60  32.18  31.76

[ nan   nan   nan   nan   nan   nan   nan   nan   nan  18.52  18.95  19.38
  19.82  20.25  20.68  21.11  21.54  21.97  22.4   22.83  23.25  23.68  24.11  24.54
  24.97  25.39  25.82  26.24  26.67  27.1   27.52  27.95  28.37  28.8   29.22  29.64
  30.07  30.49  30.91  31.34  31.76  32.18  32.6   33.02  33.44  33.86  34.28  34.7
  35.12  35.54]
```

To skip the missing values, we convert the current result back into a `pd.Series()` and use the `pd.Series.dropna()` method.

```
pd.Series(below_zero.head().T.iloc[:, :-1].T.to_numpy().flatten()).dropna()
```

```

9      18.52
10     18.95
11     19.38
12     19.82
13     20.25
14     20.68
15     21.11
16     21.54
17     21.97
18     22.40
19     22.83
20     23.25
21     23.68
22     24.11
23     24.54
24     24.97
```

```
25    25.39
26    25.82
27    26.24
28    26.67
29    27.10
30    27.52
31    27.95
32    28.37
33    28.80
34    29.22
35    29.64
36    30.07
37    30.49
38    30.91
39    31.34
40    31.76
41    32.18
42    32.60
43    33.02
44    33.44
45    33.86
46    34.28
47    34.70
48    35.12
49    35.54
dtype: float64
```

And now for the entire file:

```
below_zero_ohm = pd.Series(below_zero.T.iloc[:-1].T.to_numpy().flatten()).dropna()
print(below_zero_ohm.head(), below_zero_ohm.tail(), sep = '\n')
```

```
9      18.52
10     18.95
11     19.38
12     19.82
13     20.25
dtype: float64
205    98.44
206    98.83
207    99.22
208    99.61
```

```
209    100.00
dtype: float64
```

We can simply generate the temperature using a range object

```
below_zero_temperature = pd.Series(range(-200, 1)) # range stop is exclusive
print(below_zero_temperature.head(), below_zero_temperature.tail(), sep='\n')
```

```
0    -200
1    -199
2    -198
3    -197
4    -196
dtype: int64
196    -4
197    -3
198    -2
199    -1
200     0
dtype: int64
```

For temperatures above 0 degrees, the procedure can be simplified, since the file has the same structure and the values are sorted in ascending order.

```
above_zero = pd.read_csv(filepath_or_buffer = file_path_above_zero, sep = ',', decimal = '.')
above_zero_ohm = pd.Series(above_zero.to_numpy().flatten()).dropna()
print(above_zero_ohm.head(), above_zero_ohm.tail(), sep = '\n')
```

```
0    100.00
1    100.39
2    100.78
3    101.17
4    101.56
dtype: float64
846    389.31
847    389.60
848    389.90
849    390.19
850    390.48
dtype: float64
```

Both datasets contain an entry for 0 degrees, so it can be removed from the second file

```
above_zero_ohm = above_zero_ohm[1:]  
print(above_zero_ohm.head(), above_zero_ohm.tail(), sep='\n')
```

```
1    100.39  
2    100.78  
3    101.17  
4    101.56  
5    101.95  
dtype: float64  
846    389.31  
847    389.60  
848    389.90  
849    390.19  
850    390.48  
dtype: float64
```

The temperature is generated again using a range object.

```
abovezero_temperature = pd.Series(range(1, 851)) # range stop is exclusive
```

The Series generated from both datasets can be combined:

```
pt100 = pd.DataFrame({  
    'Temperature': pd.concat([below_zero_temperature, abovezero_temperature], ignore_index=True),  
    'Ohm': pd.concat([below_zero_ohm, above_zero_ohm], ignore_index=True)  
})  
  
# Show the first and last rows of the combined DataFrame  
  
print(pt100.head(), pt100.tail(), sep='\n')  
  
# Display information about the DataFrame  
  
print(pt100.info())
```

```
      Temperature  Ohm  
0            -200  18.52  
1            -199  18.95  
2            -198  19.38  
3            -197  19.82
```

```

4          -196  20.25
      Temperature      Ohm
1046          846  389.31
1047          847  389.60
1048          848  389.90
1049          849  390.19
1050          850  390.48
<class 'pandas.DataFrame'>
RangeIndex: 1051 entries, 0 to 1050
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Temperature  1051 non-null    int64
1   Ohm          1051 non-null    float64
dtypes: float64(1), int64(1)
memory usage: 16.6 KB
None

```

6.6.3 Representing Nonlinearity

Using the data from [Tip 12](#), the nonlinearity of the sensor can be visualized.

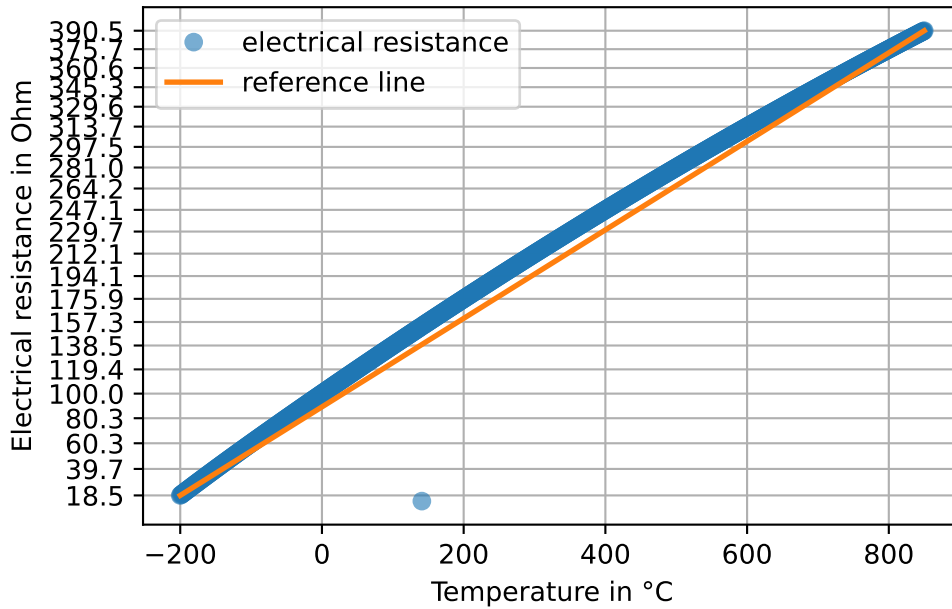
```

plt.plot(pt100['Temperature'], pt100['Ohm'], marker='o', linestyle='', label='electrical res')
plt.plot([pt100['Temperature'].iloc[0], pt100['Temperature'].iloc[-1]],
         [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
         linewidth=2, label='reference line')
plt.yticks(pt100['Ohm'][:50])

plt.xlabel('Temperature in °C')
plt.ylabel('Electrical resistance in Ohm')
plt.grid()
plt.legend()

plt.show()

```



It is noticeable that the data contains a missing value. The position of the missing value is determined using two Pandas methods. `pd.diff()` returns the difference between each value and its predecessor. `pd.idxmin()` returns the row index (specifically the label) of the smallest value. (If the missing value were above the line, `pd.idxmax()` would be used.)

```
# := is the so-called walrus operator
print(( missing_value_position := pt100['Ohm'].diff().idxmin() ))
print(pt100.iloc[list(range(missing_value_position - 2, missing_value_position + 3))])
```

```
341
    Temperature    Ohm
339          139  153.21
340          140  153.58
341          141   13.96
342          142  154.33
343          143  154.71
```

The correct value can be read from the DIN standard (Deutsches Institut für Normung [2023](#), p. 26) (153.96). However, the value can also be interpolated (see example).

Note 16: Interpolation

The simplest form of interpolation is linear interpolation.

```
interpolate_values = pt100.loc[missing_value_position - 1 : missing_value_position + 1, 'Ohm']
interpolate_values[missing_value_position] = np.nan
print(interpolate_values, "\n")

print("linearly interpolated value:", interpolate_values.interpolate(), sep="\n")
```

```
340    153.58
341         NaN
342    154.33
Name: Ohm, dtype: float64
```

```
linearly interpolated value:
340    153.580
341    153.955
342    154.330
Name: Ohm, dtype: float64
```

Rounding produces the correct result (but one should not rely on it)

```
print("linearly interpolated value:", interpolate_values.interpolate().round(2), sep = "\n")
correct_value = interpolate_values.interpolate().round(2)[missing_value_position]
```

```
linearly interpolated value:
340    153.58
341    153.96
342    154.33
Name: Ohm, dtype: float64
```

Another option is nonlinear interpolation (which is not the focus of this module).

Example 6.1: Nonlinear Interpolation

Nonlinear interpolation requires more data points. Let's try cubic interpolation:

```
interpolate_values_again = pt100.loc[missing_value_position - 3 : missing_value_position + 3, 'Ohm']
interpolate_values_again[missing_value_position] = np.nan
print("polynomial interpolated value:", interpolate_values_again.interpolate(method = "cubic"), sep = "\n")
```

polynomial interpolated value:

```
338    152.830000
339    153.210000
340    153.580000
341    153.952213
342    154.330000
343    154.710000
344    155.080000
345    155.460000
```

Name: Ohm, dtype: float64

The result is less accurate. You can learn why in the [Data Fitting and Optimization Methods Module](#).

The missing value (whose position is determined in the example above) is corrected:

```
pt100.loc[missing_value_position, 'Ohm'] = correct_value
print(pt100.loc[missing_value_position - 1 : missing_value_position + 1, 'Ohm'])
```

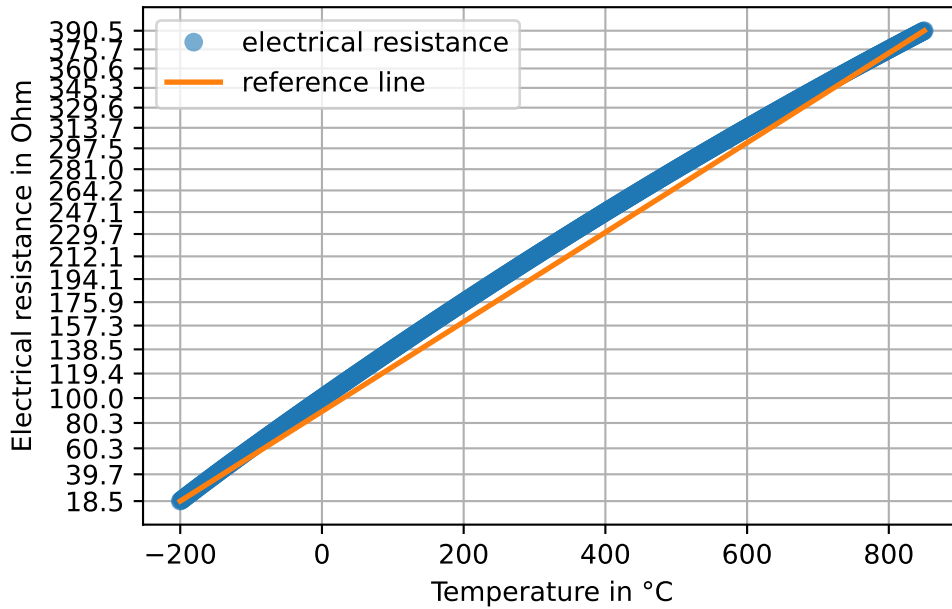
```
340    153.58
341    153.96
342    154.33
Name: Ohm, dtype: float64
```

The nonlinearity can then be represented again:

```
plt.plot(pt100['Temperature'], pt100['Ohm'], marker='o', linestyle='', label='electrical res')
plt.plot([pt100['Temperature'].iloc[0], pt100['Temperature'].iloc[-1]],
[pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
linewidth=2, label='reference line')
plt.yticks(pt100['Ohm'][:, :50])

plt.xlabel('Temperature in °C')
plt.ylabel('Electrical resistance in Ohm')
plt.grid()
plt.legend()

plt.show()
```



6.6.4 Quantifying Nonlinearity

Two methods for quantifying nonlinearity are the fixed-point method and the tolerance band method.

Fixed-Point Method

! Important 11: Fixed-Point Method

The endpoints of the actual characteristic curve are connected by a straight line. The linearity error is the maximum deviation of the characteristic curve from this line. (Grote and Feldhusen 2011, W2-W3 (pp. 1661-1662))

First, we determine the predicted values of a line defined by the endpoints.

```
lm = poly.polyfit(
    x = [pt100['Temperature'].iloc[0], pt100['Temperature'].iloc[-1]],
    y = [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
    deg = 1)
print(lm.round(2)) # intercept + slope

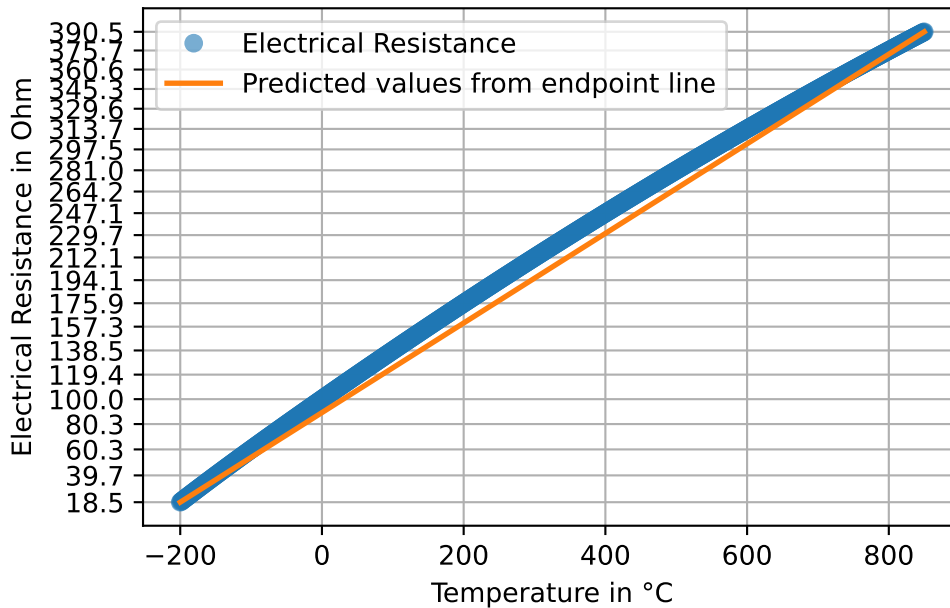
predicted_values = poly.polyval(x = pt100['Temperature'], c = lm)
```

```
plt.plot(pt100['Temperature'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'Electrical Resistance')
plt.plot(pt100['Temperature'], predicted_values, linewidth = 2, label = 'Predicted values from endpoint line')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperature in °C')
plt.ylabel('Electrical Resistance in Ohm')
plt.grid()
plt.legend()

plt.show()
```

[89.37 0.35]



The maximum linearity error can be determined from the difference between the measured electrical resistance and the linear predicted values.

```
linearity_error = (pt100['Ohm'] - predicted_values).abs().max()
print(f"Linearity error using fixed-point method: {linearity_error:.2f} Ohm.")
```

Linearity error using fixed-point method: 16.43 Ohm.

Tolerance Band Method

! Important 12: Tolerance Band Method

In the tolerance band method, a line is fitted through the measurement points such that either the sum of the squared deviations of the points from this line (least squares method) or the largest single deviation ([Chebyshev approximation](#)) is minimized. The size of the linearity error is the maximum vertical distance of the characteristic curve from this fit line. (Grote and Feldhusen [2011](#), W3 (p. 1662))

We have already learned the principle of the least squares method.

```
lm = poly.polyfit(
    x = pt100['Temperature'],
    y = pt100['Ohm'],
    deg = 1)
print(lm.round(2)) # intercept + slope

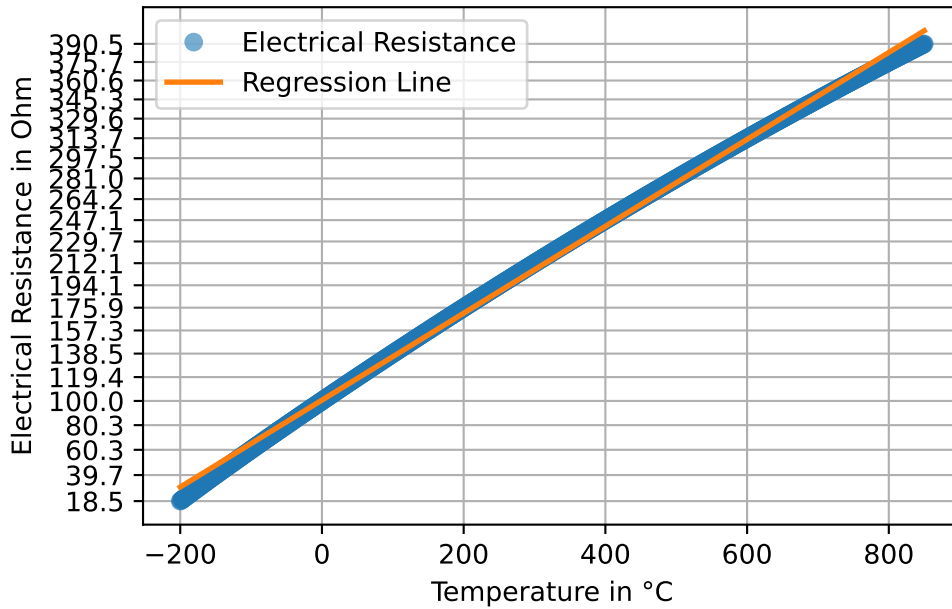
predicted_values = poly.polyval(x = pt100['Temperature'], c = lm)

plt.plot(pt100['Temperature'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'Electric')
plt.plot(pt100['Temperature'], predicted_values, linewidth = 2, label = 'Regression Line')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperature in °C')
plt.ylabel('Electrical Resistance in Ohm')
plt.grid()
plt.legend()

plt.show()
```

```
[100.67  0.35]
```



The maximum linearity error (i.e., the largest residual) can be determined from the difference between the measured electrical resistance and the linear predicted values.

```
linearity_error = (pt100['Ohm'] - predicted_values).abs().max()
print(f"Linearity error according to the tolerance band method: {linearity_error:.2f} Ohm.")
```

Linearity error according to the tolerance band method: 11.45 Ohm.

Problem: The Tolerance Band Method Produces a Zero-Point Error

6.7 Types of Errors and Practical Calibration Methods

6.7.1 Nonlinearity

In an ideal measurement, the value of interest depends linearly on the measured value. Many analog sensors, due to material properties or temperature dependence, respond nonlinearly to the measured quantity. This means that the measured values are not directly proportional to the physical quantity being measured. This so-called **linearity error** must be corrected in order to achieve accurate measurements.

There are two main approaches. One is to use nonlinear parameter estimation methods, which are covered in the [Data Fitting and Optimization Methods Module](#). The other is to correct the linearity error so that simpler linear parameter estimation methods can be used. Correction of linearity errors is the focus of this chapter.

6.8 Other

Hysteresis: Delay between the change of the input value and the response speed of the measured value

Drift: Slow change of sensor output over time

Noise: Random fluctuations in sensor output

6.9 Occurrence Together with Nonlinearity

6.10 Residual Ramp

(Additional error sources: [Hysteresis](#), Drift, Noise, or Zero-Point Shift)

Methods for Correction: Where is the variable transformation needed for linear parameter estimation?

1. **Creating a Calibration Curve/Table:** The sensor is measured at a series of known reference values (calibration points). These points are used to create an individual curve or table that represents the actual behavior of the sensor.
2. **Applying Mathematical Corrections:**
 - **Nonlinear Regression:** A function (e.g., a polynomial) is fitted to the calibration data. The difference between the measured values and the function is minimized to achieve the best possible fit. This method is shown in the [Data Fitting and Optimization Methods Module](#).
 - **Specific Coefficients:** Certain sensor types have standard models with specialized coefficients.
 - ITS-90 and Callendar–van Dusen coefficients: For the calibration of platinum sensors.
 - Steinhart–Hart coefficients: A common method for thermistors to calculate temperature.

A sufficient number of calibration points should be collected across the entire measurement range to accurately represent the nonlinearity.

Error Compensation: By applying these corrections, zero-point and span errors are also addressed, improving the accuracy of the sensor across its full measurement range.

References

- Baitsch, Matthias (2019). *Vorlesungsskript Mathematik B (Master) Stochastik*.
- Deutsches Institut für Normung (1995). *Grundlagen der Meßtechnik - Teil 1: Grundbegriffe*. DOI: [10.31030/2713411](https://doi.org/10.31030/2713411).
- (1996). *Grundlagen der Meßtechnik - Teil 3: Auswertung von Messungen einer einzelnen Meßgröße, Meßunsicherheit*. DOI: [10.31030/7204542](https://doi.org/10.31030/7204542).
 - (2023). *Industrielle Platin-Widerstandsthermometer und Platin-Temperatursensoren (IEC 60751:2022). Deutsche Fassung EN IEC 60751:2022*. DOI: [10.31030/3405985](https://doi.org/10.31030/3405985).
- Dinter, Ralf (2011). *Fehlerrechnung für Einsteiger. Eine beispielorientierte Einführung für Studierende der TUHH*. URL: <https://www2.physnet.uni-hamburg.de/TUHH/Versuchsanleitung/Fehlerrechnung.pdf>.
- Eden, Klaus and Hermann Gebhard (2024). *Dokumentation in der Mess- und Prüftechnik. Messen - Auswerten - Darstellen Protokolle - Berichte - Präsentationen*. 3. Auflage. DOI: [10.1007/978-3-658-44779-3](https://doi.org/10.1007/978-3-658-44779-3).
- Grote, Karl-Heinrich and Jörg Feldhusen, eds. (2011). *Dubbel: Taschenbuch für den Maschinenbau*. 23rd ed. Springer Berlin, Heidelberg. ISBN: 978-3-642-17306-6. DOI: [10.1007/978-3-642-17306-6](https://doi.org/10.1007/978-3-642-17306-6). URL: <https://doi.org/10.1007/978-3-642-17306-6>.
- Hempel, Rainer (2016). *Script zur Einführung in die Grundlagen der Fehlerrechnung*. URL: <https://www.tu-braunschweig.de/index.php?eID=dumpFile&t=f&f=45992&token=b041478add5f7a3e136b69905b72bfa0e192d82b>.
- Hochschule Aalen (2016). *Anleitung zur Fehlerrechnung im Physikpraktikum*. URL: https://www.hs-aalen.de/uploads/mediapool/media/file/13430/F_Fehlerrechnung.pdf.
- Schrüfer, Elmar, Leonhard M. Reindl, and Bernhard Zagar (2022). *Elektrische Messtechnik: Messung elektrischer und nichtelektrischer Größen*. 13th ed. München: Carl Hanser Verlag.